

多邻国 Android

一本按 *Duolingo* 真实轮次组织的面经

重点在“查错”与“设计”：17 道查错题 + 9 道架构设计题

SZY

2026 年 8 月 13 日

前言

这本书不按“数组、链表、动态规划、系统设计”来编排，而按多邻国真实面试轮次来编排。原因很简单：多邻国的 Android 面试不是一场换皮的 FAANG 算法考试。你当然仍然需要会写代码、会分析复杂度、会在共享编辑器里把一个函数做对；但真正拉开差距的，往往不是那道中等难度题，而是候选人是否知道自己即将进入怎样的一天。

多邻国的循环有两个决定性的不同。第一，它有一轮独立的 Code Review，也就是很多中文候选人口中的“查错”。这不是让你从零写一个算法，而是让你读一段别人已经写好的代码，找出隐藏 bug、可维护性问题、边界条件、异步状态和 Kotlin 惯用性问题，然后像真实团队评审一样说清楚“为什么这是问题、应该怎么改、风险在哪里”。第二，对 Android 岗位来说，Design Interview 不是后端分布式系统设计，不是让你画一个全球 feed、支付系统或短链服务；它更接近 Android 架构设计：模块边界、状态流、离线与同步、Repository、ViewModel、依赖注入、Server-Driven UI、性能与可测试性。这两轮都很像真实工作，也都很不像 LeetCode。

这正是很多候选人的准备错位之处。大家把最多时间花在算法题库上，因为题库清晰、反馈快、刷题有成就感；却把 Code Review 和 Android 架构设计留到最后，用几篇零散面经临时补。结果到了现场，才发现“查错”轮要求的是在陌生代码里快速定位与表达，设计轮要求的是把移动端工程复杂度讲清楚，而不是背几个缓存和消息队列名词。本书因此把 17 道调试/查错题和 9 道 Android 设计题放在中心位置。算法仍然会讲，但它不是这本书的全部；更准确地说，算法只是你进入面试的门票，Code Review 和 Design 才是多邻国 Android 面试最值得提前投资的地方。

【设计思想】讲怎么想，而不是背答案

这本书的目标不是给你一组“标准答案”。面试官真正想知道的是：你面对不完整需求时怎样澄清范围；面对陌生代码时怎样建立假设；发现 bug 后怎样判断严重性；提出设计后怎样承认取舍。你要练的不是把答案背熟，而是把推理路径说得让别人愿意相信。

本书会非常强调来源可靠性。凡是来自多邻国官方博客、官方招聘说明或官方工程文章的信息，我会标成 **[官方]**；来自公开面经、候选人复盘、面试平台汇总的信息，会标成 **[面经报告]**；根据 Android 行业常识、多邻国公开技术栈和题型逻辑做出的判断，会标成 **[高置信推断]**。请把这三类信息分开看。任何一本面经书如果不区分“官方确认”“候选人报告”和“作者推断”，都是危险的：它会让你把传闻当事实，把某一年某个人遇到的题当成固定流程，甚至在行为面里引用并不存在的公司价值观。

尤其要提醒你：多邻国官方工程博客不是背景读物，而几乎就是题库。Android Reboot、Kotlin 迁移、Server-Driven UI、Frontend Prediction 这些文章，直接暴露了他们关心的工程问题：全局可变状态、主线程阻塞、模块化、Kotlin 风格、服务端驱动 UI、乐观更新、一致性和公平性。后

面的章节会反复回到这些材料。读这本书之前，至少先把第零章列出的三篇核心官方工程文章读一遍；读完后你会发现，很多所谓“面经题”并不是凭空出现的，它们只是这些真实工程经历在面试里的压缩版本。

最后，把这本书当成一本训练“面试现场表达”的书，而不是一本速查手册。每一轮你都要练三件事：先说清楚目标，再说清楚约束，最后说清楚取舍。如果你能让面试官感觉“这个人今天就在我的代码库里也能推进事情”，那才是多邻国 Android 面试真正想筛出来的信号。

目录

前言	i
I 面试全景	1
1 多邻国面试全景	2
1.1 官方流程与轮次	2
1.2 各轮在考什么 (Android 岗视角)	3
1.3 权重与准备时间的分配建议	4
1.4 Duolingo Android 技术底座	5
1.4.1 Android Reboot (2020)	5
1.4.2 100% Kotlin 迁移	5
1.4.3 Server-Driven UI	6
1.4.4 Frontend Prediction / 乐观更新	6
1.4.5 技术栈速查	7
1.5 官方 12 条 Operating Principles	7
1.6 级别差异	8
1.7 官方给的准备建议	9
II 编码轮：算法与结对编程	11
2 算法轮：用 Kotlin 写出“像 Kotlin”的代码	12
2.1 Kotlin 编码心法	12
2.2 答题节奏：60 分钟怎么用	15
2.3 题目精解	16
2.4 数据流结构判定 (DataStream)	16
2.5 二叉树前序遍历与迭代优化	18
2.6 图遍历与边界压力测试	21
2.7 链表操作	23
2.8 设计一个带过期与容量上限的本地缓存	26

3 结对编程轮：75 分钟在别人的代码里干活	28
3.1 赛前准备：环境是最被低估的失分点	28
3.2 读陌生代码的 20 分钟法则	29
3.3 通用打法：先跑通，再做对，最后做好	29
3.4 已报告与高置信的题型	30
3.4.1 新增一个 API 调用并展示数据	30
3.4.2 补全 RecyclerView Adapter 的增删与自动刷新	32
3.4.3 在既有 SDUI 解析框架中新增组件类型	34
3.4.4 在课程完成流程中实现乐观更新	35
3.5 评分维度与自查表	37
III 查错轮：Code Review	38
4 查错轮方法论：60 分钟怎么评审	39
4.1 这一轮的形式与官方说明	39
4.2 评审的优先级阶梯	40
4.3 一遍读、两遍读、三遍读	41
4.4 说出口的模板	41
4.5 七类 Checklist	42
4.6 一个完整的评审演练	43
5 查错题集 A：协程、Flow 与并发	48
5.1 在 GlobalScope 里发起网络请求	48
5.2 在主线程上做 Room 查询与磁盘 IO	50
5.3 不可取消的轮询循环	53
5.4 async 的异常击穿了父作用域	56
5.5 在 Fragment 里收集 Flow 却没有绑定生命周期	58
5.6 被多个协程并发修改的共享可变状态	61
6 查错题集 B：生命周期、泄漏与列表	65
6.1 Fragment 的 binding 没有在 onDestroyView 置空	65
6.2 observe 用了 this 而不是 viewLifecycleOwner	68
6.3 ViewModel 持有 Activity Context	70
6.4 ViewHolder 在 bind 里起协程却不在回收时取消	73
6.5 每次数据变化都 notifyDataSetChanged，且 stable ID 不稳定	77
6.6 注册了监听器却没有解注册	80
7 查错题集 C：Compose、空安全与 Kotlin 惯用性	84
7.1 把 MutableStateFlow 暴露出去，并用 SharedFlow 存持久状态	84
7.2 LaunchedEffect 的 key 用错，副作用写在 composable 函数体里	87
7.3 collectAsState 而非 collectAsStateWithLifecycle，以及不稳定参数引发的过度重组	90

7.4	!! 强制解包、Java 平台类型与未初始化的 lateinit	93
7.5	一整段 Java 味的 Kotlin	96
IV	设计轮：Android 架构设计	100
8	Android 架构设计方法论	101
8.1	六步答题框架	102
8.1.1	第一步：澄清需求（5 分钟）	102
8.1.2	第二步：分层与数据流（8 分钟）	103
8.1.3	第三步：本地存储选型（5 分钟）	103
8.1.4	第四步：同步与写路径（10 分钟）	104
8.1.5	第五步：状态管理与乐观更新（10 分钟）	105
8.1.6	第六步：边界与失败（10 分钟）	105
8.2	什么时候该用乐观更新	106
8.3	画图与表达	107
8.4	自查表	108
9	设计题精讲（上）：离线、连胜、排行榜与实验	109
9.1	离线课程下载与进度同步	109
9.1.1	题面与常见变体	109
9.1.2	需求澄清：你该问什么	109
9.1.3	架构总览	110
9.1.4	数据模型	110
9.1.5	关键机制逐个击破	111
9.1.6	边界情况	113
9.2	连胜（Streak）客户端	114
9.2.1	题面与常见变体	114
9.2.2	需求澄清：你该问什么	114
9.2.3	架构总览	114
9.2.4	数据模型	115
9.2.5	关键机制逐个击破	115
9.2.6	边界情况	116
9.3	排行榜客户端与 XP 预测	117
9.3.1	题面与常见变体	117
9.3.2	需求澄清：你该问什么	118
9.3.3	架构总览	118
9.3.4	数据模型	118
9.3.5	关键机制逐个击破	119
9.3.6	边界情况	120
9.4	客户端 A/B 实验 SDK	121

9.4.1	题面与常见变体	121
9.4.2	需求澄清：你该问什么	122
9.4.3	架构总览	122
9.4.4	数据模型与 API	122
9.4.5	关键机制逐个击破	123
9.4.6	边界情况	124
10	设计题精讲（下）：SDUI、资产、通知与状态重构	126
10.1	Server-Driven UI 客户端解析与渲染框架	126
10.1.1	题面与常见变体	126
10.1.2	需求澄清：你该问什么	126
10.1.3	架构总览	127
10.1.4	数据模型 / 关键接口	127
10.1.5	关键机制逐个击破	128
10.1.6	边界情况	130
10.2	图片与音频资产的缓存系统	131
10.2.1	题面与常见变体	131
10.2.2	需求澄清：你该问什么	131
10.2.3	架构总览	132
10.2.4	数据模型 / 关键接口	132
10.2.5	关键机制逐个击破	132
10.2.6	边界情况	134
10.3	推送通知的客户端接收、调度与落地	134
10.3.1	题面与常见变体	135
10.3.2	需求澄清：你该问什么	135
10.3.3	架构总览	135
10.3.4	数据模型 / 关键接口	135
10.3.5	关键机制逐个击破	136
10.3.6	边界情况	137
10.4	间隔重复练习的客户端存储与调度	138
10.4.1	题面与常见变体	138
10.4.2	需求澄清：你该问什么	138
10.4.3	架构总览	138
10.4.4	数据模型 / 关键接口	139
10.4.5	关键机制逐个击破	139
10.4.6	边界情况	140
10.5	把一个 DuoState 式全局状态重构成 MVVM	141
10.5.1	题面与常见变体	141
10.5.2	需求澄清：你该问什么	141
10.5.3	架构总览	141

10.5.4 数据模型 / 关键接口	142
10.5.5 关键机制逐个击破	143
10.5.6 边界情况	144
V 项目、行为面与冲刺	146
11 项目深挖、行为面与两周冲刺	147
11.1 项目深挖轮	147
11.1.1 Android 项目示范讲法骨架	148
11.2 12 条 Operating Principles 与 STAR 故事	148
11.2.1 故事矩阵	149
11.2.2 STAR 范例骨架	150
11.3 高频行为问题与答法	150
11.3.1 Learners First 类	150
11.3.2 实验文化类	150
11.3.3 协作类	151
11.3.4 AI 成熟度类	151
11.3.5 Android 专项	151
11.4 反问环节	152
11.5 两周冲刺计划	152
11.6 资源清单	154

Part I

面试全景

Chapter 1

多邻国面试全景

多邻国的面试准备，最怕用错参照物。很多候选人默认“科技公司面试都差不多”：先刷算法，再背系统设计，再准备几段行为故事。这个框架并非完全错误，却会漏掉多邻国最有辨识度的部分。多邻国公开说过，他们希望面试让候选人感受到成为一名 Duolingo 工程师是什么样子；对应到 Android 岗，就是读现有代码、和工程师一起改代码、评审别人写的 Kotlin、解释一个客户端架构为什么能长期演进。

因此，本章不是泛泛介绍流程，而是帮你建立一张地图：哪些信息来自官方，哪些只是社区面经；每一轮到底在考什么；Android 岗和后端岗的设计轮有什么本质差别；以及为什么多邻国的几篇官方工程博客值得反复读。

1.1 官方流程与轮次

[官方] 多邻国官方工程面试文章¹把流程拆成 Pre-Onsite 和 Virtual Onsite 两大阶段。Pre-Onsite 用来确认基本编码能力、沟通方式和岗位匹配；Virtual Onsite 则更接近真实工作日，包含结对编程、代码评审、设计和行为沟通。官方给出的轮次可以整理如下：

面试类型	适用范围	允许语言	时长
Online Coding Task	仅大学/应届	Python, Java, JavaScript	60 min
Technical Video Interview	所有职位	Python, Java, JavaScript, Kotlin, Swift	60 min
Pair Programming	所有职位	Python 首选/Java; iOS → Swift; Android → Kotlin	75 min
Code Review	所有职位	同上, Android 有专属 Kotlin 版本	60 min
Whiteboard Interview	仅大学/应届	任意语言，伪代码可	60 min
Design Interview	行业职位	后端 System Design; iOS/Android 为架构设计，不写代码	60 min

¹<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

这张表最重要的不是时长，而是它明确告诉你：Android 候选人的设计轮不是默认的后端分布式系统设计。你当然可以了解缓存、队列、数据库和一致性，因为移动端应用终究要和后端系统协作；但如果你把 60 分钟全部讲成 API Gateway、Kafka 和分库分表，就很可能错过题目真正想看的移动端架构能力。多邻国 Android 面试更关心的是：一个复杂 App 如何组织状态，如何把业务逻辑从 UI 中移走，如何在客户端做预测又不破坏公平性，如何让代码可测试、可评审、可长期迁移。

[面经报告] 公开面经和面试平台复盘²³⁴给出的端到端顺序大致是：Recruiter Screen，通常是 60 分钟 Zoom；Technical Phone Screen，在 CoderPad 或 CodeSignal 中完成，常见配置是两名工程师在场，一位主问、一位 shadow；Pair Programming，约 75 分钟，常用 VS Code 加 Live Share，候选人需要提前配置环境；然后是 3-4 小时 Virtual Onsite，包含 manager 主导的 Behavioral、Design、通过 CodeSignal 进行的 Code Review、可能追加的 coding，以及 2025 年以后一些候选人报告过的实验性 AI-Assisted Assessment。完整时间线常见为 4-6 周。另一个细节是，多邻国使用自建 scheduling portal，候选人可能提前看到面试官姓名，这意味着你可以更早地理解对方团队背景，但不要把它变成临场套近乎。

【陷阱】最容易被低估的轮次不是算法

Pair Programming 是许多候选人事前不知道存在的一轮。公开复盘里相当一部分失败并不是“题不会”，而是环境配置、屏幕共享、VS Code Live Share、依赖安装或输入输出方式出了问题。把 Live Share 提前跑通，找朋友模拟一次共享编辑，比临时多刷两道题更有边际收益。

【设计思想】面试是在模拟“当一天 Duolingo 工程师”

多邻国的流程不是为了把候选人关在抽象题库里，而是故意让你经历真实工程动作：读别人写过的代码，评审别人提交的 diff，在已有代码库里扩展功能，解释一个模块为什么应该这样拆。贯穿所有技术轮的元能力，是在陌生代码中快速定位与表达，不是 puzzle-solving。你越早按这个方向训练，越不会在现场被题型变化吓住。

1.2 各轮在考什么 (Android 岗视角)

从 Android 岗看，多邻国每一轮的核心不是孤立知识点，而是不同颗粒度的工程判断。Technical Video Interview 看你能不能在有限时间内写出正确、清晰、可解释的代码；Pair Programming 看你能不能和另一位工程师一起推进一个小功能；Code Review 看你能不能识别别人代码里的风险；Design 看你能不能把移动端复杂度拆成可长期维护的架构。

²<https://interviewing.io/duolingo-interview-questions>

³<https://www.designgurus.io/answers/detail/what-is-the-duolingo-interview-process-like-round-by-round>

⁴<https://programhelp.net/vo/duolingo-sde-interview-recap/>

轮次	核心考点	Duolingo 侧重
Recruiter Screen	背景、动机、时间线、岗位匹配	是否理解教育产品、增长文化和移动端岗位范围
Technical Video Interview	基础编码、边界条件、复杂度、沟通	能否边写边解释, 并用 Kotlin/Java/Python 写出可读实现
Pair Programming	协作编码、已有代码理解、增量修改	能否在提示下快速进入陌生项目, 保持小步提交式思路
Code Review	查错、可维护性、Kotlin 惯用性、测试意识	能否指出真实风险, 而不是只挑格式和命名
Design Interview	Android 架构、状态管理、模块边界、离线与同步	能否把 MVVM、Repository、DI、SDUI、乐观更新讲成取舍
Behavioral	原则、冲突、主人翁意识、学习能力	是否能用具体故事体现 Learners First、Explain Why、Reduce Complexity

【要点】一句话区分

Technical Screen 考“能不能独立写对”；Pair Programming 考“能不能一起改好”；Code Review 考“能不能看出风险并说服别人”；Design 考“能不能让 Android 代码库继续长大”；Behavioral 考“你是不是多邻国愿意长期合作的人”。

对 Android 候选人来说, 最值得训练的是把语言从“我会某个框架”改成“我知道这个框架解决了什么代价”。比如不要只说“我会 Hilt”, 而要说“我们用依赖注入把全局单例和 Activity 生命周期解耦, 使测试可以替换 Repository, 也避免多个 feature 直接读写同一个可变对象”。这类表达会自然连接到多邻国自己的 Android Reboot 经验。

1.3 权重与准备时间的分配建议

如果你已经有基本刷题经验, 准备多邻国 Android 面试时不应该把 80% 时间继续投入 LeetCode。算法准备的迁移性很强: 你为其他公司练过的哈希表、双指针、BFS、堆、动态规划, 在这里仍然有用。Code Review 和 Android Design 的迁移性却弱得多, 因为它们要求你用多邻国关心的工程语言来表达。也正因为如此, 它们的边际收益最高。

准备模块	建议占比	原因
Code Review / 查错	25%–30%	独立轮次, 通用刷题几乎覆盖不到; 最能体现真实工程判断
Android 架构设计	20%–25%	行业 Android 岗核心差异点, 不能用后端系统设计模板硬套
Pair Programming	15%–20%	既考编码也考协作和环境稳定性, 需要模拟共享编辑
算法编码	20%–25%	仍是硬门槛, 但不应吞掉全部准备时间
Behavioral / Principles	10%	多邻国价值观明确, 故事要提前映射而不是临场发挥

【设计思想】把一半时间投给最不通用的两轮

Code Review 加 Design 约等于 50% 的准备时间，看似激进，实际很理性。它们最不像通用题库，也最接近多邻国真实工作方式。你在这两轮里形成的方法论，还会反哺 Pair Programming：读代码更快，表达取舍更清楚，遇到提示也更能顺着面试官的方向演进。

1.4 Duolingo Android 技术底座

多邻国的 Android 面试不是凭空设计的。它背后有一条很清楚的工程主线：一个高速增长、实验密集、业务状态复杂的教育 App，如何从历史包袱中演进成可维护的现代 Kotlin Android 代码库。下面这些官方文章，不只是公司宣传材料，而是理解题型的入口。

1.4.1 Android Reboot (2020)

[官方] Android Reboot 的公开材料来自 Google Android Developers 的案例⁵和 Android Developers Stories⁶。当时多邻国 Android 端有一个巨大的全局可变状态对象 `codeDuoState`，XP、streak、lesson 等多个业务域都共享它。随着团队和功能增长，这个对象让模块之间高度耦合，状态变更难以追踪，主线程工作也越来越难控制。官方披露的症状很严重：ANR rate 每月恶化 5–10%；某次发布导致 crash 增加 10%，frame rendering 慢 25%，入门设备上 lesson start 慢 70%。

修复方式不是“顺手重构”，而是一次非常重的工程选择：冻结所有 feature work 8 周，集中做 Android Reboot。团队引入 MVVM，把 UI 状态和业务逻辑分层；用 Dagger 加 Hilt 替代到处可见的全局

`codeDuoState` 访问；按 feature 做模块化；用 Repository pattern 把网络、数据库和耗时逻辑从主线程路径移走。结果同样明确：ANR 降低 41%，frames missing deadline 的时间降低 28%，scroll speed 提升 40%。

这段历史对面试的意义非常直接。任何 `codeDuoState` 式反模式，都是 Code Review 和 Design 的高频素材：全局可变状态、过度耦合、生命周期不清、主线程阻塞、测试困难、一个 feature 的改动影响另一个 feature。你在查错轮里看到“随手从 singleton 里读写用户状态”，不要只说“这不优雅”；要说它会怎样造成竞态、陈旧 UI、难以复现的实验差异，以及为什么 Repository 或单向状态流能降低风险。

1.4.2 100% Kotlin 迁移

[官方] 多邻国 Android 团队还公开写过迁移到 100% Kotlin 的过程⁷。这不是一次简单的语法替换，而是持续两年的工程迁移。官方数据包括：平均代码行数减少约 30%，个别文件最多减少 90%；Android developer NPS 提升 129；工具链上同时使用 detekt、ktlint、IntelliJ inspections、Android Lint，以及自研 Splinter。

⁵<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

⁶<https://developer.android.com/stories/apps/duolingo-excellence>

⁷<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

更值得面试准备借鉴的是迁移方法：一个 PR 只迁一个文件，并拆成三个提交，先由 IDE 自动转换，再修复编译错误，最后做 idiomatic refactor。这个顺序很像面试里应有的节奏：先让代码正确，再让它可读，最后再谈优雅。不要一开始就追求炫技式 Kotlin；但当功能正确后，要主动把 Java 味很重的写法改成 Kotlin 风格。

【要点】Kotlin 惯用性是评分信号

Android 候选人的代码要“看起来像 Kotlin，而不是 Java”。空安全、`data class`、不可变集合、`sealed` 状态、扩展函数、作用域函数、协程取消与结构化并发，这些不是装饰，而是多邻国明确重视的工程语言。

1.4.3 Server-Driven UI

[官方] 多邻国的 Server-Driven UI 文章⁸解释了他们如何把一些产品界面从客户端发版周期里解耦出来。响应由三部分组成：screens，也就是组件树；stylesheet，描述样式；以及 `codedata_model`，承载具体数据。组件库不是无限制的 HTML，而是约 12 种基础 component type，例如 labels、images、stacks、buttons、carousels。交互由 actions 描述，如 tap、show、dismiss；展示逻辑由 conditions 控制，可以依赖用户状态，例如 hearts count。

最有价值的是版本兼容策略。新客户端可以拿到完整的 `codeSDUIResponse` 并按服务端配置渲染；老客户端拿到 Partial Response，只接收 data model，再用本地缓存的 UI config 渲染。这样服务端不需要在每次响应前做复杂 client version check。多邻国已经把这套方案用于 Shop 和 Purchase Flow。

对 Android Design 轮来说，这篇文章提供了非常好的取舍材料。SDUI 的收益是实验速度、配置能力和跨端一致性；代价是客户端渲染器复杂、schema 版本管理困难、可测试性要求更高、错误兜底必须设计清楚。面试中如果题目涉及动态首页、商城、付费流程、活动页或实验平台，SDUI 都是值得主动讨论的方向。

1.4.4 Frontend Prediction / 乐观更新

[官方] 如果只能读一篇多邻国工程博客，我会选 Frontend Prediction⁹。这篇文章几乎把移动端面试最喜欢考的状态一致性问题讲透了，而且案例来自真实产品。

第一个案例是课程进度。用户答完题后，客户端先把本地 counter 加一，让界面立刻反馈，之后再和服务端同步。收益是体验顺滑；成本是客户端和服务端都要维护一份状态和逻辑，一旦规则变化就可能不一致。第二个案例是排行榜 XP。客户端在请求还没完成时先预测 XP，问题在于客户端和服务端可能对 correct-answer count 不一致，也可能对 XP boost 是否生效判断不同。文章讨论了三种回滚策略：never roll back，体验稳定但可能长期显示错误；immediate rollback，真值回来就改，但 session end card 用 prediction、leaderboard tab 用 truth 时会出现同一 App 内部不一致；deferred rollback，让两个界面都先用 prediction，等 app restart 后替换成真值，体验更顺但调试困难。第三个案例是 Follow 按钮，这类操作更接近纯 optimistic UI，失败时回滚或提示都比较容易理解。

⁸<https://blog.duolingo.com/server-driven-ui/>

⁹<https://blog.duolingo.com/frontend-prediction/>

关键是，多邻国明确说 prediction 不是到处都能用。购买 gems 这类 monetized resource 不适合预测，排行榜晋级这种涉及公平性的结果也不适合预测。这个判断比“会不会乐观更新”更重要：优秀候选人会先问数据域是否允许短暂不一致，再决定 UI 是否可以预测。

【设计思想】最高收益官方读物

Frontend Prediction 是本书建议反复读的一篇文章。它同时连接 Code Review、Android Design 和 Behavioral：代码层面有重复状态和回滚 bug，设计层面有一致性边界，原则层面有 Learners First 与公平性取舍。

1.4.5 技术栈速查

下面这张表把官方确认与高置信推断分开。准备时可以把官方项当作重点语境，把推断项当作合理候选方案；不要在面试中把推断说成“多邻国一定在用”。

领域	技术/实践	来源
Android 语言	Kotlin 100%	[官方]
Android 架构	MVVM, Repository, 模块化	[官方]
依赖注入	Dagger + Hilt	[官方]
UI	Jetpack Compose 迁移进行中, Server-Driven UI	[官方]
质量工具	detekt, ktlint, Splinter, Android Lint	[官方]
本地数据	Room	[高置信推断]
图片	Coil	[高置信推断]
网络	Retrofit + OkHttp	[高置信推断]
后台任务与推送	WorkManager, FCM	[高置信推断]
后端语言与框架	Python + Flask 为主, Kotlin JVM 微服务	[官方]
基础设施	AWS, DynamoDB, PostgreSQL, Redis, Kafka	[官方] / [高置信推断]
个性化学习	Birdbrain 个性化选题 ML 系统	[官方]
实验文化	全公司 A/B 实验文化, 300+ 并行实验	[官方]

后台背景对 Android 候选人也有帮助。排行榜常让人想到 Redis Sorted Set；学习进度和实验分组会牵涉 PostgreSQL、DynamoDB 或服务端配置；事件流和训练数据可能经过 Kafka；Birdbrain 说明多邻国的产品体验高度个性化。这些不要求你像后端候选人一样设计完整基础设施，但能帮助你在移动端设计里问对问题：哪些状态来自服务端，哪些可以缓存，哪些必须实时，哪些失败可以延迟补偿。

1.5 官方 12 条 Operating Principles

[官方] 多邻国官方 Operating Principles¹⁰是行为面和协作表达的底层语言。不要把它们当成口号；更好的用法是，把每条原则映射到一个真实故事或一次技术取舍。

¹⁰<https://blog.duolingo.com/operating-principles/>

Principle	中文理解
Learners First	先问什么对学习真正有利，而不是只优化团队局部指标。
Prioritize Ruthlessly	资源有限时敢于砍范围，把最重要的问题先做完。
Take the Long View	不只追短期发布，也考虑代码、产品和用户信任的长期成本。
Test It First	用实验和数据验证判断，尤其适合多邻国的 A/B 文化。
Reduce Complexity	主动降低系统、流程和产品复杂度，不为聪明而复杂。
Ship It	在质量可控的前提下尽快交付，让真实反馈进入循环。
Strive for Excellence	对性能、体验、代码质量和细节保持高标准。
Be Candid and Kind	反馈要直接，也要尊重人；Code Review 尤其需要这一点。
Explain Why	不只给结论，还要解释背景、约束和取舍。
Embrace Challenges	面对模糊、困难和失败时保持学习姿态。
Never Settle on Talent	对招聘和团队质量保持高标准。
All for One, and One for All	跨团队合作，愿意为整体目标调整局部计划。

【陷阱】不要引用不存在的价值观

网上不少准备材料会把“Show, Don't Tell”列为 Duolingo 官方价值观，但它并不在官方 12 条 Operating Principles 里。行为面引用不存在的原则不是小问题：它会暴露你没有读官方材料。第十章会把这 12 条逐一映射到 STAR 故事。

1.6 级别差异

不同级别看到的“同一轮”，评分尺度并不相同。SWE I 更强调基础和潜力，Senior 开始强调复杂代码库中的判断，Staff 则会被期待讨论跨团队架构、长期演进和组织影响。下面的表格把社区报告和高置信推断合并整理；具体安排仍以 recruiter 给你的日程为准。

级别	Pair Programming 期望	Code Review 期望	Design 期望	特别关注
SWE I (应届)	[面经报告] 能写出基础功能, 接受提示后修正方向	[面经报告] 通常无独立 Code Review	[面经报告] 通常无完整行业 Design, 可能有 Whiteboard	基础编码、学习速度、沟通清晰度
SWE II	[高置信推断] 实现完整 feature, 覆盖错误处理和简单测试	[高置信推断] 能找出主要 bug、边界条件和可读性问题	[高置信推断] 做单 feature 轻量设计, 讲清 ViewModel/Repository 边界	独立交付、Kotlin 风格、范围控制
Senior	[高置信推断] 能在复杂代码中重构并保持行为稳定	[高置信推断] 能识别架构性风险、并发/状态问题和测试缺口	[高置信推断] 端到端子系统设计, 考虑离线、同步、实验、性能	技术取舍、长期维护、带动他人
Staff	[高置信推断] 重点不在写多少代码, 而在快速建立方向和约束	[高置信推断] 能从 diff 看出跨模块、跨团队和平台层风险	[高置信推断] 架构级讨论, 跨团队边界、演进路线和迁移策略	组织影响、平台化、长期技术战略

这张表的用途不是给自己贴标签, 而是校准表达深度。SWE II 候选人把一个 feature 做完整, 比空谈平台化更有说服力; Senior 候选人如果只停留在“这里可以抽个 helper”, 就显得深度不足; Staff 候选人则需要主动把问题提升到边界、迁移、团队协作和长期成本。

1.7 官方给的准备建议

[官方] 多邻国官方面试文章给出的建议非常务实。第一, think out loud。不要等想出完美答案再开口; 面试官需要看到你的判断过程。第二, pragmatic, 先从时间内能完成的最简单版本开始, 再逐步改进。第三, 把面试官当协作者。多邻国明确说面试官会给提示, 他们不是敌人; 你能否接住提示, 本身就是评分信号。第四, 主动澄清问题, 不要擅自扩大范围。第五, 考虑 edge cases, 但不要让边界情况吞掉主路径。第六, 准备好反问他们的问题, 因为这也是你判断团队和岗位的机会。

这些建议听起来普通, 却和多邻国题型高度一致。Code Review 里, 你需要一边读一边说“我先找 correctness, 再看 state, 再看可维护性”; Pair Programming 里, 你需要说“我先让 happy path 跑通, 再补 error path”; Design 里, 你需要说“我先假设只做 Android 客户端, 不设计完整后端, 除非我们需要讨论同步协议”。这种开场能让面试官立刻知道你在控制范围。

【话术】开场可以这样说

“我先确认一下范围：这一轮我们更希望我优化 correctness，还是也要讨论架构和测试？”

“我会先实现一个最小可工作的版本，然后回头处理边界条件和可读性；如果你希望我优先走另一条路线，请打断我。”

“这里我先做一个假设：服务端返回的数据可能延迟或失败，所以客户端状态不能直接当作真值。后面如果这个假设不对，我会调整设计。”

“我会边读代码边说出我在验证的假设：先看状态来源，再看生命周期，再看异步回调是否可能乱序。”

Part II

编码轮：算法与结对编程

Chapter 2

算法轮：用 Kotlin 写出“像 Kotlin”的代码

如果说 FAANG 的算法轮常常在测试你能不能把一道抽象题推到最优，那么 Duolingo 的 Android 技术视频面更像是在看：你能不能在一小时里，用 Kotlin 写出未来同事愿意继续维护的代码。官方披露的技术视频面为 60 分钟，通常在 CodeSignal 中进行，Android 候选人使用 Kotlin，有时会有一位主面试官和一位观察者同时在场。[\[官方\]](#)¹ 社区面经也反复指向同一个结论：难度上限并不追求 LeetCode Hard，更多是 Medium 以内的数据结构、遍历、建模与 Big-O 讨论。[\[面经报告\]](#)² 但这不代表它容易。它只是把压力从“谜题难度”转移到了“代码质量下限”。

这点对 Android/Kotlin 候选人尤其重要。Duolingo 曾公开讲过它们花了约两年完成 100% Kotlin 迁移，并用 detekt、ktlint 与内部 linter Splinter 维护代码质量。[\[官方\]](#)³ 所以，一段能跑但充满 Java 味的 Kotlin，在这里不是小瑕疵，而是负信号。面试官当然关心你会不会栈、队列、图遍历；但他们也在观察你是否自然使用空安全、不可变建模、表达式式的集合操作、清楚的类边界。

本章的第二条主线是：Duolingo 高频题形很少是“给一个数组，返回一个数字”，而更像“给一个类，让它对一串操作做出正确判断或维护正确状态”。最典型的 `DataStream` 题就是这样：你不是一次性求答案，而是在 `add/remove` 的序列里维护一组候选假设。这种题考的 not 是背模板，而是状态建模、不变量、公共 API 与边界处理。把这一点吃透，比再刷十道冷门 DP 更有用。

2.1 Kotlin 编码心法

用类型表达状态，而不是用整数和注释解释状态

在 Kotlin 里，状态不是靠注释保护的，而应当被类型系统托住。面试中如果你写出 `status = 0`、`status = 1` 再靠注释说明含义，面试官会立刻看到 Java 旧习惯。更好的做法是用 `enum class` 表示有限枚举，用 `sealed class` 表示带数据的状态族；配合 `when` 的穷尽性检查，编译器会帮你发现漏分支。

```
1 class LessonState(var status: Int, var error: String?)
```

¹<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

²<https://interviewing.io/duolingo-interview-questions>

³<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

```

2
3 fun render(state: LessonState): String {
4     return when (state.status) {
5         0 -> "Loading"
6         1 -> "Done"
7         2 -> "Error: ${state.error}"
8         else -> "Unknown"
9     }
10 }

```

```

1 sealed class LessonState {
2     object Loading : LessonState()
3     data class Ready(val title: String) : LessonState()
4     data class Failed(val message: String) : LessonState()
5 }
6
7 fun render(state: LessonState): String = when (state) {
8     LessonState.Loading -> "Loading"
9     is LessonState.Ready -> state.title
10    is LessonState.Failed -> "Error: ${state.message}"
11 }

```

第一段代码的问题不是“不能工作”，而是把不变量藏在人的脑子里：`status=2` 时 `error` 理应非空，但类型系统不知道。第二段代码把状态拆成三种互斥形态，`Failed` 天然携带错误信息，`Ready` 天然携带标题。这就是 Kotlin 面试里的第一层信号：你写的不是 Java 的语法糖，而是 Kotlin 的类型建模。

空安全不是语法题，而是边界意识

Android 代码离不开网络、缓存、生命周期与 nullable 数据。不要把 `!!` 当快捷键；它在面试里几乎等价于“我还没想清楚这里为什么非空”。如果某个值来自外部输入，用 `?:` 给出兜底；如果它是业务不变量，用 `requireNotNull` 明确失败原因；如果只是存在才继续，用 `?.let` 收窄作用域。

```

1 fun avatarUrl(user: User?): String {
2     return user!!.profile!!.avatarUrl!!
3 }

```

```

1 fun avatarUrl(user: User?): String =
2     user?.profile?.avatarUrl ?: DEFAULT_AVATAR_URL
3
4 fun submit(answer: Answer?) {
5     val nonNullAnswer = requireNotNull(answer) { "Answer must exist before
6         submit" }
7 }

```

```
6     send(nonNullAnswer)
7 }
```

这里的关键不是“永远不抛异常”，而是区分两类情况：缺头像可恢复的 UI 兜底；提交答案为空则违反流程不变量，应当早失败、说清楚。面试官听到你这样解释，会知道你不是机械地躲避 !!，而是在设计错误边界。

集合操作要像 Kotlin，但别炫技

Kotlin 的标准库让许多数组和哈希表题写得更接近问题本身。统计、分组、索引、求和、取最大值，优先想到 `filter`、`map`、`groupBy`、`associateBy`、`sumOf`、`maxByOrNull`。不过面试不是函数式编程秀场。链式调用超过三四步、每一步还带复杂 lambda 时，可读性和可调试性会下降。好的标准是：代码读起来像题意，而不是像你在证明自己会 API。

```
1 data class Skill(val id: String, val topic: String, val xp: Int, val mastered:
   Boolean)
2
3 fun xpByTopic(skills: List<Skill>): Map<String, Int> =
4     skills
5         .filter { it.mastered }
6         .groupBy { it.topic }
7         .mapValues { (_, topicSkills) -> topicSkills.sumOf { it.xp } }
8
9 fun skillIndex(skills: List<Skill>): Map<String, Skill> =
10    skills.associateBy { it.id }
```

如果你担心链条过长，可以主动说：“这里我先用标准库表达意图；如果后面要加日志或调试中间状态，我会拆成局部变量。”这句话本身就是成熟信号。

不可变优先，让更新点变少

算法题里当然可以用可变数组和队列，但对象状态不要随手暴露为 `MutableList`。能用 `val` 就不用 `var`；返回 `List` 而不是 `MutableList`；数据对象更新优先用 `copy()`。不可变不是洁癖，而是降低面试中出错面的办法。

```
1 data class UiState(
2     val isLoading: Boolean = false,
3     val words: List<String> = emptyList(),
4     val error: String? = null,
5 )
6
7 fun UiState.withWords(newWords: List<String>): UiState =
8     copy(isLoading = false, words = newWords, error = null)
```


用 object、伴生对象与扩展函数收敛小工具

Kotlin 里单例配置适合 `object`，命名构造适合 `companion object`，不拥有状态的小转换适合扩展函数。它们的共同目的都是让调用处更像业务语言，而不是到处散落工具函数。

```
1 object CachePolicy {
2     const val MAX_ITEMS = 200
3     const val TTL_MILLIS = 10 * 60 * 1000L
4 }
5
6 data class WordId(val raw: String) {
7     companion object {
8         fun fromApi(value: String): WordId = WordId(value.trim().lowercase())
9     }
10 }
11
12 fun String.normalizedWord(): String = trim().lowercase()
```

【设计思想】面试官读你的代码时，第一眼看到的是命名与状态建模，第二眼才是复杂度

复杂度当然要对，但多数 Duolingo 编码题的复杂度并不离奇。真正拉开差距的是：类名是否说明职责，状态是否互斥，方法是否只做一件事，边界是否被类型表达。一个 $O(n)$ 的乱糟糟实现，不如一个同样 $O(n)$ 但可读、可扩展、容易自测的实现。

【陷阱】最容易泄露的 Java 味

面试 Kotlin 里常见的负信号包括：大量 `!!`；用 `ArrayList` 和 `HashMap` 作为返回类型而不是接口 `List/Map`；可用 `data class` 却手写 `getter`；用整数 `flag` 表示状态；把所有函数塞进静态工具类思维；循环里手动维护一堆临时变量却不命名中间概念；忘记主动说明 Big-O。它们单独看都不致命，连在一起就会让人怀疑你只是“会写 Kotlin 语法的 Java 工程师”。

最后，Big-O 要主动开口。官方建议候选人准备讨论复杂度与边界。[\[官方\]](#)⁴ 你不需要等面试官才说：“这里每个操作摊还 $O(1)$ ，空间和候选结构数量成正比，在本题是常数。”这类报价会让对话变得稳定。

2.2 答题节奏：60 分钟怎么用

60 分钟看似很长，真正写代码的时间并不多。一个可靠节奏是：前 5 分钟澄清题意与输入输出；接着 5 分钟口述方案、不变量与复杂度；用 20 分钟写最小可用版本；再用 10 分钟拿样例和自造边界手动跑；最后 15 分钟处理边界、优化、追问与重构。官方明确建议候选人务实，从能在时间内完成的最简单版本开始，并且边想边说、把面试官当合作者。[\[官方\]](#)⁵

这个节奏的重点不是机械计时，而是防止两种极端：一种是没想清楚就写，30 分钟后发现接

⁴<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

⁵<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

口错了；另一种是一直追求完美抽象，20 分钟过去还没有可运行代码。Duolingo 的题通常允许你先写朴素版本，再在追问里展示优化意识。你应该把“先跑通”当成策略，而不是妥协。

【话术】每个阶段可以直接说出口的话

澄清时说：“Let me restate the problem first. We receive a sequence of operations, and the class should maintain enough state to answer after each operation. I want to clarify empty input, duplicate values, and what to return when multiple answers are possible.”

动手前说：“I will start with the simplest complete version: maintain the candidate hypotheses explicitly, then we can discuss whether any state can be compressed.”

写完第一版后说：“Before optimizing, I want to walk through a small example and one edge case, because hidden tests usually punish state bugs more than asymptotic tricks.”

追问时说：“That changes the contract slightly. I would first name the new output semantics, then adjust the internal representation. I do not want to silently widen the scope without confirming it.”

2.3 题目精解

2.4 数据流结构判定 (DataStream)

面经报告 · 高频 | 难度：Medium

题意

社区面经中最详细的一题是 DataStream：给定一串 `add(x)` 与 `remove(x)` 操作，判断底层结构可能是 Stack、Queue 还是 PriorityQueue。[\[面经报告\]⁶ interviewing.io](https://interviewing.io/duolingo-interview-questions) 也报告过同族题：定义一个类，根据“put this value, get this value”的流式操作推断底层数据结构。[\[面经报告\]⁷](https://interviewing.io/duolingo-interview-questions) 如果多种结构都能解释当前序列，就返回多个候选；如果都不能解释，就返回空集合。

思路推导

这题的核心不是某个数据结构，而是“不变量筛选”。你同时维护三个模拟器：一个栈、一个队列、一个优先队列；再维护三个候选是否仍成立的标记。每次 `add(x)`，所有仍可能的模拟器都加入 `x`。每次 `remove(x)`，分别从三个模拟器取出它们认为应该被移除的值：栈取最后进入的，队列取最早进入的，优先队列取最大值。如果模拟器为空，或者取出的值不等于 `x`，该候选被淘汰。`guess()` 返回尚未被淘汰的候选集合。

注意优先队列的 tie。若题目没有说重复值如何比较，最安全的解释是 PriorityQueue 只按值比较；两个相等最大值取哪个都不影响 `remove(x)` 的可观察结果。Kotlin 的 PriorityQueue 默认是小根堆，所以最大堆要传反向比较器。

```
1 import java.util.ArrayDeque
```

⁶<https://programhelp.net/vo/duolingo-sde-interview-recap/>

⁷<https://interviewing.io/duolingo-interview-questions>

```
2 import java.util.PriorityQueue
3
4 enum class Kind { STACK, QUEUE, PRIORITY_QUEUE }
5
6 class DataStream {
7     private val stack = ArrayDeque<Int>()
8     private val queue = ArrayDeque<Int>()
9     private val maxHeap = PriorityQueue<Int>(compareByDescending { it })
10
11     private val possible = Kind.entries.associateWith { true }.toMutableMap()
12
13     fun add(value: Int) {
14         if (possible[Kind.STACK] == true) stack.addLast(value)
15         if (possible[Kind.QUEUE] == true) queue.addLast(value)
16         if (possible[Kind.PRIORITY_QUEUE] == true) maxHeap.add(value)
17     }
18
19     fun remove(value: Int) {
20         checkStack(value)
21         checkQueue(value)
22         checkPriorityQueue(value)
23     }
24
25     fun guess(): Set<Kind> = possible
26         .filterValues { it }
27         .keys
28         .toSet()
29
30     private fun checkStack(expected: Int) {
31         if (possible[Kind.STACK] != true) return
32         val actual = stack.pollLast()
33         if (actual != expected) possible[Kind.STACK] = false
34     }
35
36     private fun checkQueue(expected: Int) {
37         if (possible[Kind.QUEUE] != true) return
38         val actual = queue.pollFirst()
39         if (actual != expected) possible[Kind.QUEUE] = false
40     }
41
42     private fun checkPriorityQueue(expected: Int) {
43         if (possible[Kind.PRIORITY_QUEUE] != true) return
44         val actual = maxHeap.poll()
45         if (actual != expected) possible[Kind.PRIORITY_QUEUE] = false
46     }
47 }
```

47 }

这版代码刻意保留三个模拟器，而不是过早压缩状态。因为面试里最重要的是让不变量清楚：每个候选模拟器的内容，代表“如果真实结构是它，那么到目前为止内部应当长这样”。一旦某个输出与预期不一致，候选永久淘汰。

【要点】复杂度

add 对栈和队列是 $O(1)$ ，对优先队列是 $O(\log n)$ ；**remove** 同理被优先队列主导，为 $O(\log n)$ 。空间 $O(n)$ ，因为最坏情况下三个模拟器都保留全部元素；候选种类固定为 3，所以候选集合本身是常数空间。

【设计思想】用不变量筛选候选假设

DataStream 的可迁移心法是：当你不知道真实模型是哪一个，就维护一组“仍能解释观测”的候选假设；每来一个新观测，就用不变量淘汰矛盾者。这不仅适用于数据结构判定，也适用于类型推断、协议探测、A/B 归因、日志事件归类。面试官真正想看的，是你能不能把“猜”变成可维护的状态机。

【陷阱】三个边界最容易漏

第一，空结构上 **remove** 不能崩溃，而应淘汰对应候选；**pollFirst/pollLast** 返回 nullable，正好表达这一点。第二，所有候选都被淘汰时，**guess()** 返回空集合，不要强行选一个。第三，优先队列有重复最大值时，只比较值，不要引入不存在的稳定顺序假设。

【追问】“如果你不能保证这个数据流会按某种模式来，怎么判断应该用哪种数据结构？”

这会把确定性判定改成评分或概率问题。公共 API 也要随之改变：**guess()** 不再返回 `Set<Kind>`，而返回按置信度排序的结果，例如每个候选记录 `matchedRemoves / totalRemoves`、最近窗口的一致率，或者一个带权分数。这样即使三种结构都偶尔不一致，也能说“Stack 的解释力最高，但置信度只有 0.62”。关键是主动指出：一旦输入不保证来自某个完美模型，原来的布尔候选不变量就不够了，必须把“是否可能”升级为“多大程度上像”。

2.5 二叉树前序遍历与迭代优化

面经报告 | 难度：Easy

题意

Glassdoor 汇总的 Duolingo 面经中出现过二叉树前序遍历，以及“如何优化”的追问。[\[面经报告\]](#)⁸ 前序遍历顺序是 root、left、right。基础版可以递归；优化版通常讨论显式栈；如果面试官继续追问空间，则可以提 Morris traversal。

⁸<https://www.glassdoor.com/Interview/Duolingo-Interview-Questions-E629348.htm>

思路推导

递归是最自然的第一版：访问当前节点，再递归左、右子树。它短、正确、适合先跑通。问题是递归深度等于树高，极端链状树可能触发 `StackOverflowError`，这在 Android 上不是理论问题。第二版用显式栈模拟调用栈：先压右孩子，再压左孩子，保证左孩子先弹出。第三版 Morris traversal 利用临时线索指针做到 $O(1)$ 额外空间，但会短暂修改树结构，代码复杂度更高。

```
1  data class TreeNode(  
2      val value: Int,  
3      var left: TreeNode? = null,  
4      var right: TreeNode? = null,  
5  )  
6  
7  fun preorderRecursive(root: TreeNode?): List<Int> {  
8      val result = mutableListOf<Int>()  
9  
10     fun dfs(node: TreeNode?) {  
11         if (node == null) return  
12         result += node.value  
13         dfs(node.left)  
14         dfs(node.right)  
15     }  
16  
17     dfs(root)  
18     return result  
19 }  
20  
21 fun preorderIterative(root: TreeNode?): List<Int> {  
22     if (root == null) return emptyList()  
23  
24     val result = mutableListOf<Int>()  
25     val stack = ArrayDeque<TreeNode>()  
26     stack.addLast(root)  
27  
28     while (stack.isNotEmpty()) {  
29         val node = stack.removeLast()  
30         result += node.value  
31         node.right?.let { stack.addLast(it) }  
32         node.left?.let { stack.addLast(it) }  
33     }  
34     return result  
35 }  
36  
37 fun preorderMorris(root: TreeNode?): List<Int> {  
38     val result = mutableListOf<Int>()  
39     var current = root
```

```
40
41 while (current != null) {
42     val left = current.left
43     if (left == null) {
44         result += current.value
45         current = current.right
46     } else {
47         var predecessor = left
48         while (predecessor?.right != null && predecessor.right != current)
49         {
50             predecessor = predecessor.right
51         }
52         if (predecessor?.right == null) {
53             result += current.value
54             predecessor?.right = current
55             current = current.left
56         } else {
57             predecessor.right = null
58             current = current.right
59         }
60     }
61     return result
62 }
```

【要点】复杂度

三种写法时间都是 $O(n)$ 。递归与显式栈空间为 $O(h)$ ， h 是树高；Morris traversal 额外空间 $O(1)$ ，但会临时改指针，代码风险更高。

【设计思想】优化不是永远追求最省空间

在移动端， $O(1)$ 空间听起来诱人，但 Morris traversal 需要修改树结构。如果这棵树来自共享缓存、UI 状态或并发读取路径，临时改指针可能比 $O(h)$ 栈更危险。成熟回答应当是：“如果树可能很深，我优先改成显式栈避免系统栈溢出；只有在内存极紧且树结构可安全修改时，才考虑 Morris。”

【陷阱】Kotlin 可空节点不要用 !!

链式树结构最容易诱惑你写 `node.left!!`。但前序遍历的边界本来就是空子树，正确写法应当让 nullable 流过控制结构：`node.left?.let ...` 或函数入口直接接收 `TreeNode?`。这比在代码里撒 !! 更能体现 Kotlin 基本功。

【追问】“如果树非常深，递归为什么不好？”

递归消耗的是线程调用栈，不受你业务层集合容量控制。Android 主线程栈空间有限，极深输入会导致 `StackOverflowError`。显式栈把风险转移到堆上，并且更容易做分批、取消或协程调度；这是移动端语境下比单纯背复杂度更好的解释。

2.6 图遍历与边界压力测试

面经报告 | 难度: **Medium**

题意

图遍历和优化题也出现在 Duolingo 面经汇总中，且新毕业生 OA 被报告为 CodeSignal 四题，包含图与数据结构遍历，隐藏测试会惩罚边界处理不足。[\[面经报告\]](#)⁹ 我们把它改写成更贴近 Duolingo 的 Android 场景：课程技能树中，每个 skill 可能依赖若干前置 skill。给定已掌握集合与依赖边，计算当前可解锁的 skill；如果依赖图有环，要能检测出来；若问到目标 skill，还能用 BFS 说明最短解锁路径的层数。

思路推导

解锁判断是拓扑思想：一个 skill 可解锁，当它尚未 mastered 且所有前置依赖都已 mastered。若要批量模拟“接下来不断学习可解锁技能”，可以维护入度和邻接表，先把已满足依赖的节点入队，逐步削减后继入度。环检测同样来自拓扑排序：处理完后仍有未处理节点，说明存在环或无法满足的依赖。

边界比算法更重要：空图返回空；自环应被识别为环；重复边要去重，否则入度会被多加；不连通分量不能漏；超深 DFS 可能爆栈，面试里可以主动选择迭代 BFS/Kahn。

```
1 data class SkillGraph(  
2     val skills: Set<String>,  
3     val prerequisites: Map<String, Set<String>>,  
4 )  
5  
6 data class UnlockResult(  
7     val unlockableNow: Set<String>,  
8     val hasCycle: Boolean,  
9 )  
10  
11 fun analyzeUnlocks(graph: SkillGraph, mastered: Set<String>): UnlockResult {  
12     val normalizedPrereqs = graph.skills.associateWith { skill ->  
13         graph.prerequisites[skill].orEmpty().filter { it in graph.skills }.toSet  
14     }  
15 }
```

⁹<https://www.glassdoor.com/Interview/Duolingo-Interview-Questions-E629348.htm>


```

16     val unlockableNow = normalizedPrereqs
17         .filter { (skill, prereqs) -> skill !in mastered && prereqs.all { it in
18 mastered } }
19         .keys
20         .toSet()
21
22     val outgoing = graph.skills.associateWith { mutableSetOf<String>() }.
23 toMutableMap()
24     val indegree = graph.skills.associateWith { 0 }.toMutableMap()
25
26     for ((skill, prereqs) in normalizedPrereqs) {
27         for (pre in prereqs) {
28             if (outgoing.getValue(pre).add(skill)) {
29                 indegree[skill] = indegree.getValue(skill) + 1
30             }
31         }
32     }
33
34     val queue = ArrayDeque<String>()
35     indegree.filterValues { it == 0 }.keys.forEach { queue.addLast(it) }
36
37     var visited = 0
38     while (queue.isNotEmpty()) {
39         val current = queue.removeFirst()
40         visited++
41         for (next in outgoing.getValue(current)) {
42             val nextDegree = indegree.getValue(next) - 1
43             indegree[next] = nextDegree
44             if (nextDegree == 0) queue.addLast(next)
45         }
46     }
47
48     return UnlockResult(
49         unlockableNow = unlockableNow,
50         hasCycle = visited != graph.skills.size,
51     )
52 }
53
54 fun shortestPrerequisiteDistance(
55     graph: SkillGraph,
56     start: String,
57     target: String,
58 ): Int? {
59     if (start == target) return 0
60 }

```



```
59     val outgoing = graph.skills.associateWith { mutableSetOf<String>() }.  
        toMutableMap()  
60     for ((skill, prereqs) in graph.prerequisites) {  
61         for (pre in prereqs) outgoing.getOrPut(pre) { mutableSetOf() }.add(skill  
        )  
62     }  
63  
64     val seen = mutableSetOf(start)  
65     val queue = ArrayDeque<Pair<String, Int>>()  
66     queue.addLast(start to 0)  
67  
68     while (queue.isNotEmpty()) {  
69         val (current, distance) = queue.removeFirst()  
70         for (next in outgoing[current].orEmpty()) {  
71             if (!seen.add(next)) continue  
72             if (next == target) return distance + 1  
73             queue.addLast(next to distance + 1)  
74         }  
75     }  
76     return null  
77 }
```

【要点】复杂度

设技能数为 V 、去重后的依赖边为 E 。构图与 Kahn 拓扑排序时间 $O(V + E)$ ，空间 $O(V + E)$ 。BFS 最短层数同样是 $O(V + E)$ 。

【陷阱】隐藏测试常打这些边界

空图、只有一个节点、自环、重复依赖边、不在 `skills` 集合里的脏依赖、不连通分量、所有节点都已掌握、没有任何节点可解锁、链很深导致递归 DFS 爆栈。你不一定要把每个边界都写成测试，但面试中至少要口头走两三个，证明你知道 hidden cases 会从哪里来。

【追问】“如果要实时更新 mastered 集合呢？”

如果用户每完成一课就更新一次，不必每次从零扫描全图。可以维护每个 skill 剩余未满足依赖数；当一个 skill mastered 后，只更新它的后继节点，把剩余数减一，变成零时加入可解锁集合。这样单次更新成本接近该 skill 的出边数，适合客户端本地状态增量维护。

2.7 链表操作

题意

链表操作也出现在 Duolingo 面经汇总中。[\[面经报告\]¹⁰](https://www.glassdoor.com/Interview/Duolingo-Interview-Questions-E629348.htm) 这里选一个常见 Medium 变体：给定单链表 $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ ，重排为 $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ 。要求原地修改节点连接，不额外创建新节点。

思路推导

标准三步：第一，用快慢指针找到中点；第二，反转后半段；第三，把前半段与反转后的后半段交替合并。这题真正适合 Kotlin 的教学点是 nullable discipline。链表节点的 `next` 天然可空，写法要围绕 `?.`、局部变量和早返回组织，而不是用一串 `!!` 硬闯。

```
1 class ListNode(  
2     val value: Int,  
3     var next: ListNode? = null,  
4 )  
5  
6 fun reorderList(head: ListNode?) {  
7     if (head?.next == null) return  
8  
9     val secondHalf = splitSecondHalf(head)  
10    val reversed = reverse(secondHalf)  
11    mergeAlternating(head, reversed)  
12 }  
13  
14 private fun splitSecondHalf(head: ListNode): ListNode? {  
15     var slow: ListNode = head  
16     var fast: ListNode? = head.next  
17  
18     while (fast?.next != null) {  
19         slow = slow.next ?: break  
20         fast = fast.next?.next  
21     }  
22  
23     val second = slow.next  
24     slow.next = null  
25     return second  
26 }  
27  
28 private fun reverse(head: ListNode?): ListNode? {  
29     var previous: ListNode? = null  
30     var current = head  
31  
32     while (current != null) {
```

¹⁰<https://www.glassdoor.com/Interview/Duolingo-Interview-Questions-E629348.htm>

```
33     val next = current.next
34     current.next = previous
35     previous = current
36     current = next
37 }
38 return previous
39 }
40
41 private fun mergeAlternating(firstHead: ListNode, secondHead: ListNode?) {
42     var first: ListNode? = firstHead
43     var second = secondHead
44
45     while (first != null && second != null) {
46         val firstNext = first.next
47         val secondNext = second.next
48
49         first.next = second
50         second.next = firstNext
51
52         first = firstNext
53         second = secondNext
54     }
55 }
```

【要点】复杂度

快慢指针、中段反转、交替合并各扫描一次链表，时间 $O(n)$ ；除少量指针变量外不分配额外节点，空间 $O(1)$ 。

【设计思想】把指针题拆成阶段，不要在一个循环里做完

链表题最怕“聪明地一遍完成”。面试里更稳的做法是把中点、反转、合并拆成三个私有函数，每个函数都有单一不变量。这样不仅更容易写对，也方便你在追问里替换某一步，例如把反转改成用栈，或把合并规则改成每次取两个节点。

【陷阱】快慢指针的偶数长度边界

偶数长度时中点切在哪里会影响合并是否形成环。这里用 `fast = head.next`，让前半段长度不小于后半段；切断 `slow.next = null` 是必须的，否则合并时可能把旧链路带回来形成环。

【追问】“如果链表很长且来自 UI 列表数据，是否应该原地改？”

如果节点对象被其他层共享，原地重排会产生副作用。Android 实际 UI 数据更常见是不可变 `List<Item>` 加 `DiffUtil`，而不是可变链表节点。可以回答：算法题按原地链表写；生产代码里我会优先返回新的不可变列表，除非性能分析证明必须复用节点。

2.8 设计一个带过期与容量上限的本地缓存

面经报告 · 变体 | 难度：Medium

题意

HashMap 操作题在面经汇总中出现过。[\[面经报告\]](#)¹¹ 我们把它包装成 Android 语境：客户端要缓存发音音频或课程数据，缓存有容量上限，并且每个条目有 TTL。实现 `get(key)` 与 `put(key, value)`：读取过期条目应返回空并删除；超过容量时淘汰最近最少使用的条目。

思路推导

JVM 已经给了一个很适合面试的工具：`LinkedHashMap` 支持 `access-order`，也就是每次 `get` 或 `put` 会把条目移动到末尾。这样最旧的条目就在迭代器开头。TTL 则存在 `entry` 里，每次 `get` 检查当前时间。为了可测试性，不要在代码里直接调用 `System.currentTimeMillis()`，而是注入 `Clock` 函数。

```
1 class TtlLruCache<K, V>(  
2     private val maxSize: Int,  
3     private val ttlMillis: Long,  
4     private val nowMillis: () -> Long = { System.currentTimeMillis() },  
5 ) {  
6     init {  
7         require(maxSize > 0) { "maxSize must be positive" }  
8         require(ttlMillis > 0) { "ttlMillis must be positive" }  
9     }  
10  
11     private data class Entry<V>(val value: V, val expiresAt: Long)  
12  
13     private val map = object : LinkedHashMap<K, Entry<V>>(16, 0.75f, true) {  
14         override fun removeEldestEntry(eldest: MutableMap.MutableEntry<K, Entry<  
15             V>>): Boolean {  
16             return size > maxSize  
17         }  
18     }  
19  
20     @Synchronized  
21     fun get(key: K): V? {
```

¹¹<https://www.glassdoor.com/Interview/Duolingo-Interview-Questions-E629348.htm>

```
21     val entry = map[key] ?: return null
22     if (entry.expiresAt <= nowMillis()) {
23         map.remove(key)
24         return null
25     }
26     return entry.value
27 }
28
29 @Synchronized
30 fun put(key: K, value: V) {
31     map[key] = Entry(value = value, expiresAt = nowMillis() + ttlMillis)
32 }
33
34 @Synchronized
35 fun size(): Int = map.size
36 }
```

【要点】复杂度

get 与 put 在哈希良好的情况下为 $O(1)$ ；空间为 $O(capacity)$ 。TTL 采用懒删除，所以过期项可能在未访问前暂时占位，但容量淘汰仍由 `LinkedHashMap` 控制。

【设计思想】把时间作为依赖注入

缓存题的隐藏难点是测试。直接读系统时间会让单元测试又慢又不稳定；把 `nowMillis` 作为函数注入，就可以在测试里手动推进时间。这是 Android 代码里非常可迁移的设计心法：凡是不可控外部环境，先变成依赖。

【陷阱】线程安全别轻描淡写

Android 上 Repository、预加载任务、播放器回调可能从不同线程访问缓存。面试里可以先用 `@Synchronized` 说明最小线程安全版本；如果吞吐量更高，再讨论 `Mutex`、单线程 dispatcher 或分段锁。不要一边说“本地缓存”，一边完全忽略并发访问。

【追问】“如果这是 Android 上的磁盘缓存呢？”

磁盘缓存会多出文件命名、原子写入、崩溃恢复、总字节数限制、后台清理、加密与隐私策略。内存里的 `LinkedHashMap` 只能保留索引思路，不能直接照搬。完整答案会转向 `asset-cache` 设计：元数据表记录 key、路径、大小、最后访问时间和过期时间；写入先落临时文件，成功后原子 rename；淘汰按 LRU 和总字节数双条件执行。这个问题会在后面的设计章节继续展开。

Chapter 3

结对编程轮：75 分钟在别人的代码里干活

很多候选人知道 Duolingo 有算法轮，却不知道还有一轮更不像 LeetCode: Pair Programming。官方把这一轮描述为在 production-like codebase 里工作，目标是让候选人感受成为 Duolingo 工程师是什么样子，同时评估你能否读懂别人的代码、快速学习并协作推进任务。Android 候选人仍然使用 Kotlin，Duolingo 会提前发送环境设置说明；这一轮时长 75 分钟，是整个 loop 中最长的一轮。[\[官方\]](#)¹

这轮最难的地方不是某个 API，而是你必须在陌生代码里做真实改动。算法轮可以靠你自己的节奏完成；结对编程轮则会暴露工作习惯：会不会先找既有模式，会不会尊重代码风格，会不会边读边说，会不会在卡住时接受提示，会不会为了完美方案迟迟不交付。Duolingo 官方建议候选人务实、先做能在时间内完成的版本、把面试官当合作者；这在结对轮里不是建议，而是生存策略。[\[官方\]](#)²

3.1 赛前准备：环境是最被低估的失分点

社区报告里，一个反复出现的非技术风险是开发环境与屏幕共享：VS Code、Live Share、权限、网络、终端共享，任何一个环节出问题，都会把 75 分钟切掉一大块。[\[面经报告\]](#)³ 对 Android 候选人尤其要注意：这轮很可能不是 Android Studio 里让你舒舒服服地用熟悉快捷键重构，而是在 VS Code 或远程协作环境里改 Kotlin。你的 Android Studio 肌肉记忆、插件、AI 补全、Gradle 面板，都可能不在场。

赛前预检应当具体到操作，而不是一句“我会准备环境”。至少做一次和朋友的 Live Share 演练；确认你会分享终端、切换文件、全局搜索、运行测试；准备备用网络或热点；关闭通知和勿扰；把仓库按说明提前 clone、build、跑最小测试；确认 JDK、Android SDK、Gradle cache 与 Kotlin 插件状态；问 recruiter 哪些工具会预装，是否允许本地 IDE，是否会给 mock backend；最后，练习不用 AI 插件时的编辑器快捷键。

¹<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

²<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

³<https://interviewing.io/duolingo-interview-questions>

【陷阱】环境失败会被误读成协作失败

如果你前 20 分钟都在找权限按钮，面试官未必会把它完全归因于工具。它会影响他们对“能否快速上手生产环境”的判断。真正成熟的做法是在一开始就说：“I tested Live Share and terminal sharing yesterday, but if this environment has issues I can switch to screen share and narrate each change.”你不是保证永不出错，而是证明你有降级方案。

3.2 读陌生代码的 20 分钟法则

拿到任务后，第一反应不要是新建文件。你应该用前 20 分钟建立地图：先读目录结构和依赖注入入口，知道 app/module/feature/data/ui 大致分层；再找到一个与任务最相似的既有功能，把它端到端读一遍；接着定位数据流方向：UI ← ViewModel ← Repository ← Local/Remote；最后才决定在哪些文件做最小改动。

这套方法的核心是“找先例”。如果任务是展示 Word of the Day，就去找已经展示 Daily Quest、Streak、Lesson Tip 或类似卡片的代码；如果任务是新增 API，就找已有 service 方法、repository 方法和 ViewModel state 的组合；如果任务是 adapter 增删，就找项目里已有 ListAdapter 的 DiffUtil 写法。你不需要从零设计一个干净架构，面试官恰恰在看你能不能融入既有架构。

【设计思想】不要从零设计，要找到代码库里的先例并模仿它

结对轮考的不是“你能不能发明一个更好的架构”，而是“你能不能在别人的约定里安全交付”。先例是最快的隐性文档：命名、错误处理、协程作用域、状态流、测试风格、日志方式，全都藏在相似功能里。照着先例做，通常比自作主张更像真实同事。

【话术】把读代码方法说出来

“I am going to spend a few minutes finding an existing feature that looks similar, because I want to follow the codebase conventions rather than invent a new pattern.”

“I see the data flow is UI to ViewModel to Repository to RemoteDataSource. I will first wire a hardcoded value through that path, then replace the hardcoded part with the real logic.”

“I am not changing this abstraction yet because it is used elsewhere. I will keep the first pass local and ask before widening the scope.”

3.3 通用打法：先跑通，再做对，最后做好

结对编程的最佳节奏是三段式：先跑通，再做对，最后做好。先跑通，意思是用 hardcoded response 或 fake data 把 UI 到数据层的路径接通，让大家看到端到端行为。再做对，才替换成真实 API、真实过滤逻辑、真实错误状态。最后做好，则是补边界、命名、测试、空状态、loading、重试、DiffUtil、线程与生命周期。

这并不是偷懒，而是符合官方 “be pragmatic, start with the simplest version you can finish in time” 的建议。[\[官方\]⁴](https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/) 在 75 分钟里，一段端到端跑通的朴素实现，比一个局部很优雅但没有

⁴<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

接到 UI 的抽象更有说服力。你应该在一开始就宣布计划：“我会先 hardcode 一条数据确认路径，再替换成真实逻辑。”这样面试官不会误以为你只会写假代码，而会看到你在控制风险。

问与不问也要拿捏。会影响产品语义的地方要问：空数据展示什么？失败是否重试？是否需要缓存？是否影响实验？不影响第一版的小实现细节可以先假设并说出口：我先沿用现有 repository 的错误包装；我先用已有 dispatcher；我先不引入分页。接受提示时不要防御，直接复述并应用：“That makes sense. I was about to create a new state field, but reusing the existing content state is more consistent. I will adjust.”这句话比沉默硬扛更像同事。

3.4 已报告与高置信的题型

3.4.1 新增一个 API 调用并展示数据

题面

社区面经报告过一个 backend-flavored 任务：给首页增加 Word of the Day API。候选人先浏览 models 和 routes，站起一个返回 hardcoded value 的 endpoint，再逐步加入推荐逻辑：简单版随机选一个用户正在学但未掌握的词，更好版按最近学习主题推荐。[\[面经报告\]](#)⁵ Android analogue 则是：在既有 MVVM 功能中新增一个 API 调用，并把返回的 Word of the Day 展示到首页卡片。

前 5 分钟做什么

先找一个首页已有卡片，从 UI 读到 ViewModel，再读 Repository 和 service interface。确认项目用的是 Retrofit、Ktor 还是内部 client；确认 UI 用 XML、RecyclerView、Compose 或混合；确认 state 是 `StateFlow`、LiveData 还是自定义 store。然后说清楚第一版路径：Remote 返回 hardcoded/fake word，Repository 暴露 domain model，ViewModel 更新 state，UI 展示 title 和 pronunciation。

分步实现计划

第一步新增 API DTO 与 service 方法，但可以先让 fake implementation 返回固定值。第二步在 Repository 中映射到 domain model，并处理空响应。第三步 ViewModel 在 `init` 或显式 `load()` 中调用 repository，写入 `StateFlow`。第四步 UI 订阅 state，展示 loading、content、error。第五步把 hardcoded 推荐换成真实逻辑：随机取正在学但未掌握的词；若时间允许，再按最近学习 topic 排序。

关键代码骨架

```
1 data class WordOfDay(  
2     val id: String,  
3     val text: String,  
4     val translation: String,  
5     val topic: String,
```

⁵<https://programhelp.net/vo/duolingo-sde-interview-recap/>


```
6 )
7
8 interface WordService {
9     suspend fun fetchWordOfDay(): WordOfDayDto
10 }
11
12 class WordRepository(
13     private val service: WordService,
14 ) {
15     suspend fun wordOfDay(): Result<WordOfDay> = runCatching {
16         service.fetchWordOfDay().toDomain()
17     }
18 }
19
20 data class HomeUiState(
21     val isLoadingWord: Boolean = false,
22     val wordOfDay: WordOfDay? = null,
23     val wordError: String? = null,
24 )
25
26 class HomeViewModel(
27     private val repository: WordRepository,
28 ) : ViewModel() {
29     private val _state = MutableStateFlow(HomeUiState())
30     val state: StateFlow<HomeUiState> = _state
31
32     fun loadWordOfDay() {
33         viewModelScope.launch {
34             _state.update { it.copy(isLoadingWord = true, wordError = null) }
35             repository.wordOfDay()
36                 .onSuccess { word ->
37                 _state.update { it.copy(isLoadingWord = false, wordOfDay =
word) }
38             }
39             .onFailure { error ->
40                 _state.update {
41                     it.copy(isLoadingWord = false, wordError = error.message
?: "Unable to load word")
42                 }
43             }
44         }
45     }
46 }
```

容易翻车的点

最常见的错误是只写 service，不把数据接到 UI；或者只写 UI mock，不说明如何替换真实 API。其次是把网络异常直接抛到 ViewModel 外，让 loading 永远不结束。再者是忘记生命周期：UI 订阅 `StateFlow` 应该遵循项目已有的 lifecycle-aware 方式。最后，推荐逻辑不要自行扩大范围到复杂 ML 排序；先交付随机未掌握词，再讲 topic-based ranking 是下一步。

【追问】“如果要按最近学习主题推荐呢？”

Repository 可以拿到 `recentTopics` 与 `learningWords` 后做一个简单排序：过滤未掌握词，再按 topic 是否出现在最近主题中加权，最后稳定地取分数最高或在同分中随机。关键是说明这会改变数据依赖，可能需要新增 endpoint 或把本地学习记录传入推荐函数；不要在 ViewModel 里堆业务排序。

3.4.2 补全 RecyclerView Adapter 的增删与自动刷新

题面

Android-flavored 社区报告提到过不完整的 RecyclerView Adapter：需要支持 add/remove，并在数据变化时自动刷新。[\[面经报告\]](#) 这题表面是 Adapter，实际在看你是否知道现代 Android 列表更新不应粗暴 `notifyDataSetChanged()`，而应优先使用 `ListAdapter` 与 `DiffUtil`，必要时讨论 stable IDs 和主线程安全。

前 5 分钟做什么

先确认项目已有列表组件怎么写：是否用 `ListAdapter`、`RecyclerView.Adapter`、`PagingDataAdapter` 或 `Compose LazyColumn`。找一个现成 Adapter 复制风格。然后确认 item 是否有稳定 id，remove 是按位置还是按 id，add 是否插到头部、尾部或排序位置。

分步实现计划

第一版把 Adapter 改为持有不可变列表快照，每次增删生成新列表并 submit；第二版引入 `DiffUtil.ItemCallback`；第三版如果项目需要动画和状态恢复，开启 stable IDs；最后补空列表和越界 remove 的行为。

关键代码骨架

```
1 data class WordItem(  
2     val id: String,  
3     val text: String,  
4     val mastery: Int,  
5 )  
6  
7 class WordAdapter(  
8     private val onClick: (WordItem) -> Unit,  
9 ) : ListAdapter<WordItem, ViewHolder>(Diff) {
```

```
10
11     init {
12         setHasStableIds(true)
13     }
14
15     override fun getItemId(position: Int): Long = getItem(position).id.hashCode
16         ().toLong()
17
18     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
19     ViewHolder {
20         return ViewHolder.create(parent, onClick)
21     }
22
23     override fun onBindViewHolder(holder: ViewHolder, position: Int) {
24         holder.bind(getItem(position))
25     }
26
27     fun addItem(item: WordItem) {
28         submitList(currentList + item)
29     }
30
31     fun removeItem(id: String) {
32         submitList(currentList.filterNot { it.id == id })
33     }
34
35     private object Diff : DiffUtil.ItemCallback<WordItem>() {
36         override fun areItemsTheSame(oldItem: WordItem, newItem: WordItem):
37         Boolean =
38             oldItem.id == newItem.id
39
40         override fun areContentsTheSame(oldItem: WordItem, newItem: WordItem):
41         Boolean =
42             oldItem == newItem
43     }
44 }
```

容易翻车的点

`notifyDataSetChanged()` 不是不能用, 但你应该主动说它会牺牲动画、局部刷新和性能, 只适合临时跑通或数据很小的场景。`submitList(currentList)` 如果传同一个可变列表实例, `DiffUtil` 可能看不到变化; 所以要提交新列表。Adapter 更新必须在主线程。stable IDs 也不是万能药, id 必须稳定且冲突概率可接受; 如果 `hashCode()` 风险不可接受, 项目应提供 Long id。

【追问】“如果列表来自 ViewModel 的 StateFlow 呢？”

Adapter 不应该自己拥有业务真相。更好的结构是 ViewModel 暴露 StateFlow<List<WordItem>>, Fragment/Activity 收集后调用 submitList。Adapter 的 add/remove 可以只作为局部练习；生产代码中，点击删除应回调 ViewModel，由状态源更新列表，再单向流回 UI。

3.4.3 在既有 SDUI 解析框架中新增组件类型**题面**

Duolingo 公开写过 Server-Driven UI：服务端下发组件树，客户端解析组件并递归渲染。[\[高置信推断\]](#)⁶ 高置信的 Android 结对变体是：已有 Component sealed class、JSON parser 与 renderer，现在要新增一种组件，比如 PromoBanner 或 WordCard。重点不是写一个 view，而是理解未知组件、版本兼容和递归渲染。

前 5 分钟做什么

先找现有组件从 JSON 到 UI 的全链路：schema/DTO、parser、domain sealed class、renderer、测试 fixture。确认未知 type 当前怎么处理：忽略、渲染占位，还是抛异常。然后照一个最相似的组件复制路径，不改变整体 parser 架构。

分步实现计划

第一步给 sealed class 增加新类型。第二步给 DTO/parser 增加 type 分支，并把缺字段情况降级为 Unknown 或返回 null。第三步 renderer 增加对应 UI。第四步加一个 JSON fixture 或单元测试，覆盖新组件和未知组件。

关键代码骨架

```

1 sealed class Component {
2     data class TextBlock(val text: String) : Component()
3     data class WordCard(val word: String, val translation: String) : Component()
4     data class Column(val children: List<Component>) : Component()
5     data class Unknown(val type: String) : Component()
6 }
7
8 fun parseComponent(json: JsonObject): Component {
9     return when (val type = json["type"]?.jsonPrimitive?.contentOrNull) {
10         "text" -> Component.TextBlock(
11             text = json["text"]?.jsonPrimitive?.contentOrNull.orEmpty(),
12         )
13         "word_card" -> {
14             val word = json["word"]?.jsonPrimitive?.contentOrNull

```

⁶<https://blog.duolingo.com/server-driven-ui/>

```

15         val translation = json["translation"]?.jsonPrimitive?.contentOrNull
16         if (word == null || translation == null) {
17             Component.Unknown(type)
18         } else {
19             Component.WordCard(word = word, translation = translation)
20         }
21     }
22     "column" -> Component.Column(
23         children = json["children"]
24             ?.jsonArray
25             .orEmpty()
26             .mapNotNull { it.jsonObjectOrNull()?.let(::parseComponent) },
27     )
28     else -> Component.Unknown(type = type ?: "missing")
29 }
30 }

```

容易翻车的点

未知组件类型不能让旧客户端崩溃。SDUI 的本质是服务端可能比客户端更新，如果 Android 遇到未来 type 就 crash，发布节奏会被锁死。另一个坑是递归 children 中一个坏组件拖垮整棵树；更稳的策略是局部降级。最后，新增组件要遵守版本策略：服务端何时可以下发，旧客户端如何忽略，是否需要 feature flag。

【追问】“为什么不用 else throw?”

因为 SDUI 的契约不是“输入永远来自同版本代码”，而是“客户端要面对服务端可演进 schema”。throw 适合开发期发现本地 bug；生产解析外部 payload 时，未知 type 应当优雅降级并记录日志。这样服务端才能渐进发布新组件。

3.4.4 在课程完成流程中实现乐观更新

题面

Duolingo 公开讨论过 frontend prediction：为了让体验更快，客户端可以先预测操作成功并立即更新界面，再与服务端结果对账。[\[高置信推断\]](https://blog.duolingo.com/frontend-prediction/)⁷ 高置信 Android 变体是：用户完成 lesson 后，立刻更新本地 XP、streak 或 skill progress，同时发请求；请求失败时回滚。需要注意，Duolingo 明确不把预测用于货币化购买等高风险操作。[\[高置信推断\]](https://blog.duolingo.com/frontend-prediction/)⁸

前 5 分钟做什么

先找到现有 lesson completion 的状态源：ViewModel、Repository、本地 store、remote endpoint。确认哪些字段可以乐观更新，哪些必须等服务端。问清楚失败 UX：toast、snackbar、静默重试还

⁷<https://blog.duolingo.com/frontend-prediction/>

⁸<https://blog.duolingo.com/frontend-prediction/>

是回滚动画。明确第一版只处理非货币化进度，不碰购买、订阅、宝石扣费。

分步实现计划

先保存更新前快照；立即把 `StateFlow` 更新到 optimistic state；发起网络请求；成功时用服务端 canonical response reconcile；失败时恢复快照并暴露错误事件。若有多个并发完成事件，要给 mutation 编号或串行化，避免旧失败回滚新状态。

关键代码骨架

```
1 data class ProgressState(  
2     val xp: Int,  
3     val streak: Int,  
4     val completedLessons: Set<String>,  
5     val errorMessage: String? = null,  
6 )  
7  
8 class LessonViewModel(  
9     private val repository: LessonRepository,  
10 ) : ViewModel() {  
11     private val _progress = MutableStateFlow(ProgressState(0, 0, emptySet()))  
12     val progress: StateFlow<ProgressState> = _progress  
13  
14     fun completeLesson(lessonId: String, earnedXp: Int) {  
15         val before = _progress.value  
16         val optimistic = before.copy(  
17             xp = before.xp + earnedXp,  
18             completedLessons = before.completedLessons + lessonId,  
19             errorMessage = null,  
20         )  
21         _progress.value = optimistic  
22  
23         viewModelScope.launch {  
24             repository.completeLesson(lessonId)  
25                 .onSuccess { serverState -> _progress.value = serverState.  
26 toProgressState() }  
27                 .onFailure { error ->  
28                     _progress.value = before.copy(  
29                         errorMessage = error.message ?: "Could not save progress  
30  
31  
32  
33     },  
34     )  
35     }  
36 }  
37 }
```

容易翻车的点

乐观更新最大的坑是并发：用户连续完成两个操作，第一个失败时不能简单回滚到最早快照，否则会抹掉第二个成功。面试时间不够时，可以先声明 single flight 假设；若要扩展，用 mutation queue 或让 Repository 串行化。第二个坑是边界选择：购买、扣费、订阅这类 monetized 操作不应预测成功。完整设计会在后续章节继续讨论。

【追问】“如果失败后用户已经看到了新 XP，回滚会不会很差？”

这取决于产品语义。低风险进度可以展示轻量错误并重新同步；高风险资产必须以服务端为准。技术上可以把 optimistic state 标记为 pending，让 UI 用微弱状态提示“正在保存”，失败时解释原因，而不是突然无声回退。

3.5 评分维度与自查表

下表把社区报告中的评价点整理成结对轮自查表。[\[面经报告\]](#)⁹

维度	强信号	弱信号
读代码	先找相似功能，沿数据流端到端阅读	随手新建文件，忽略既有模式
Kotlin 风格	使用 sealed class、StateFlow、不可变 state、空安全	Java-flavored Kotlin，大量 !! 与可变共享状态
协作沟通	持续 narrate，澄清范围，接受提示后快速调整	闷头写，卡住不说，提示后辩解
交付策略	hardcode 跑通，再替换真实逻辑，保留可演示路径	追求完美抽象，75 分钟没有端到端结果
边界意识	主动处理空数据、网络失败、生命周期、主线程刷新	只处理 happy path，loading/error 卡死
代码风格	模仿项目命名、错误包装、测试方式	重写架构或引入个人偏好

【要点】75 分钟的目标

你不是来证明自己一个人能写完整功能，而是证明自己加入一个陌生团队后，能快速读懂约定、做出小而正确的改动、在不定时沟通、在时间限制内交付可运行路径。面试官给提示不是失败信号；你如何接住提示，才是评分的一部分。

【陷阱】经典失败模式

最常见的失败是闷头写不说话；第二是看不惯既有代码风格，顺手重构半个模块；第三是纠结完美方案，最后没有任何东西跑通；第四是不问就扩大范围，把“展示一个词”做成复杂推荐系统；第五是忽略环境与屏幕共享，真实编码时间被工具吃掉。结对轮的关键词不是 hero，而是 teammate。

⁹<https://interviewing.io/duolingo-interview-questions>

Part III

查错轮：Code Review

Chapter 4

查错轮方法论：60 分钟怎么评审

如果说算法轮考的是“你能不能在压力下把一个问题解出来”，那么 Duolingo 的 Code Review 轮考的是另一种更接近日常工作的能力：**你能不能像一个资深同事那样组织表达**。这不是一句场面话。真实团队里，最有价值的评审者不是那个能在五分钟里挑出十五个命名问题的人，而是那个会先指出“这里可能让用户崩溃”“这里会在页面退出后继续写状态”“这里把失败域扩大到了整节课”的人。风格意见可以有，但它们必须排在用户损失、数据损坏、ANR、泄漏之后。

这也是为什么 FAANG 式刷题准备在这一轮迁移最差。刷题训练你在一个干净输入上追求最优复杂度；查错轮要求你在一段不完美、带业务语义、带生命周期、带线程边界的 Kotlin 代码里判断风险。它最接近 Android 工程师每天真正做的事：读别人刚改的代码，判断是否能安全进主干，然后用让人愿意接受的方式提出修改建议。

Duolingo 把这一轮单独拿出来，并不奇怪。[\[官方\]¹](#) 官方说明里，Code Review 是工业界岗位都有的 60 分钟独立环节；而 Android 岗还有专门版本。换句话说，你不是在做一套泛泛的“找 bug”题，而是在展示你是否像一个会维护大型 Kotlin Android 应用的人。

4.1 这一轮的形式与官方说明

[\[官方\]²](#) Duolingo 官方把 Code Review 描述成一个 60 分钟、在 CodeSignal 上进行的独立面试。所有 industry role 都会遇到它；Backend、iOS、Android 有各自版本；任务是阅读一段代码改动，识别问题并提出改进。官方还特别提醒：面试官会帮助你导航，不要被这一轮吓到。新毕业生流程通常用 Whiteboard 轮替代它，这也说明 Code Review 更偏向对真实工程经验的评估。

对 Android/Kotlin 候选人来说，这一点尤其重要。你看到的不是抽象伪代码，而往往是接近真实移动端工程的文件组：ViewModel、Repository、Fragment、Flow、Room、RecyclerView、Compose 或协程。[\[面经报告\]](#) 候选人报告过的题型包括：带空指针风险的数据处理管线、在 GlobalScope 中发起协程、Fragment 中没有绑定生命周期的网络响应处理，以及“评审 RecyclerView 列表更新代码，有什么性能或崩溃风险”。这些不是随机知识点，而是 Android 代码库里最容易把用户体验拖垮的边界。

还有一层背景解释了为什么 Kotlin 惯用性本身也是信号。[\[官方\]³](#) Duolingo 曾在两年内迁移到

¹<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

²<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

³<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

100% Kotlin, 并同时使用 detekt、ktlint、Android Lint、IntelliJ inspections 和自研 linter Splinter。一个候选人的代码评审如果只会说“这里可能 NPE”，却看不出 Java 味 Kotlin、可空类型契约、协程 scope、Flow 生命周期，那就不像一个会融入这套工程文化的人。

最后，Duolingo Android Reboot 的故事给这一轮定了调。[\[官方\]⁴](#) 公开案例提到，早期 Android 应用里一个被 XP、连胜、课程等模块共享的巨大可变状态对象 DuoState，让 ANR 每月恶化 5–10%；某次发布还带来 10% 崩溃增加、低端设备首帧与课程启动明显变慢。团队最终冻结功能 8 周，重建为 MVVM、Dagger/Hilt、Repository 与模块化架构，获得 ANR 下降 41%、错过帧 deadline 下降 28%、滚动速度提升 40% 的结果。这意味着：全局可变状态、主线程重活、生命周期越界，不是“理论上的坏味道”，而是 Duolingo 真正付过代价的故障类型。

4.2 评审的优先级阶梯

查错轮最核心的能力，是排序。你当然要能看见问题，但更要知道先说哪个、后说哪个、哪些只值得一句带过。下面这条阶梯请背下来；它比任何单个 Kotlin API 都重要。

1. **正确性与崩溃**：crash、ANR、数据损坏、安全问题、用户进度丢失。这一层直接伤害用户或业务指标。
2. **资源与泄漏**：内存泄漏、协程未取消、监听器未移除、Fragment view 销毁后仍被引用。这一层会把短期小 bug 拖成长期稳定性问题。
3. **并发与竞态**：线程可见性、原子性、主线程约束、读改写丢失、取消传播。这一层通常不稳定复现，但一旦线上发生很难排查。
4. **契约与可测试性**：API 暴露过宽、依赖不可注入、时间与 dispatcher 写死、副作用藏在 getter 或 mapper 里。这一层决定代码未来能不能安全演进。
5. **性能**：主线程 IO、列表全量刷新、Compose 过度重组、重复网络请求、对象分配过多。性能问题要和用户路径绑定讲，不要泛泛说“会慢”。
6. **可读性与惯用性**：命名、重复代码、Java 味 Kotlin、没用 `?.`、`let` 滥用、过长函数。这一层不是不重要，而是不能抢在前面。

【设计思想】先说会让用户损失的，再说会让同事痛苦的

一个资深 reviewer 的默认排序是：先保护用户，再保护系统，最后保护审美。用户会崩溃、课程进度会丢、低端机会上 ANR 的问题，必须排在“这个变量名不够 Kotlin”之前。你可以在最后补一句“还有几个 nits”，但不能用 nits 开场。

【陷阱】以风格问题开场

经典失败样子是：候选人读到第一行就说“这个类名不够好”“这里可以用 `also`”，然后五分钟后才发现代码会在 Fragment view 销毁后更新 UI。面试官会认为你缺乏工程风险排序。查错轮不是 lint 竞赛；Duolingo 已经有 detekt、ktlint、Android Lint 和 Splinter [\[官方\]^a](#)，他

⁴<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

们需要的是能补足机器规则之外判断的人。

^a<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

4.3 一遍读、两遍读、三遍读

不要从第一行开始逐字抠。60 分钟看似很长，但真实代码一旦有两个文件、三层调用和几个异步边界，时间很快被细节吞掉。推荐用三遍读法。

第一遍，只看数据流与线程边界。问三个问题：数据从哪里来？在哪个线程或 dispatcher 上被改？最后写到哪里？比如课程提交结果从网络返回，进入 Repository，更新本地 Room，再通过 StateFlow 驱动 ViewModel。你先不要管变量名，也不要管 mapper 是否漂亮；先画出“谁在什么线程上改什么状态”。这一遍最容易抓到 crash、ANR、数据损坏、主线程 IO 和竞态。

第二遍，看生命周期与所有权。谁创建了这个协程？谁负责取消？谁持有 Fragment view、adapter、listener、callback、context？这个对象活得是否比它引用的 UI 更久？一个网络请求在用户退出课程页后继续回来，是否还会写 binding？一个 Flow 在 onCreateView 里收集，view 重建后是否重复订阅？这一遍找的是泄漏、幽灵更新、重复事件。

第三遍，才看语法、惯用法与局部可读性。这时再看可空类型、!!、mapNotNull、copy、命名、重复逻辑、sealed state 是否更好。不是因为这些不重要，而是因为它们通常不会比前两遍的问题更贵。

【要点】为什么这个顺序有效

昂贵 bug 往往跨行、跨函数、跨生命周期：它们不会暴露在某个单独的 if 里。三遍读法先建立系统图，再看所有权，最后做局部清理，正好匹配 Android 线上故障的形态。[\[高置信推断\]](#)

4.4 说出口的模板

查错轮不是你在纸上写审阅意见，而是你边读边说。话术会影响评分：太武断像没经验，太含糊像不敢判断。好的表达结构是“影响先行、条件清楚、修复具体”。

【话术】提出问题

中文：我会把这里标成一个崩溃风险，因为 `user.profile!!` 来自网络响应，而前面没有保证 profile 一定存在。用户如果是新账号或响应降级，就会直接 crash。

English: I would flag this as a crash risk because `user.profile!!` depends on a network field that is not guaranteed to be present. A safer fix is to model the missing profile state explicitly instead of asserting non-null.

【话术】给出修复

中文：我建议把线程约束放到 Repository 内部，用注入的 IO dispatcher 包住 Room 查询。这样调用方不需要记住“必须在后台线程调用”，测试里也能替换 dispatcher。

English: I would move the dispatcher boundary into the repository. The suspend function should protect its own threading contract, and the dispatcher should be injected so the View-Model and tests do not rely on global dispatchers.

【话术】表达不确定

中文：我不确定这个方法一定从哪个线程调用。如果它可能在主线程上跑，那么这段同步数据库查询会有 ANR 风险；我会问调用链，或者把它改成内部 `withContext(ioDispatcher)` 来消除假设。

English: I am not sure what the calling context is. If this can run on the main thread, the synchronous database read is an ANR risk. I would either confirm the caller contract or make the function enforce the IO dispatcher itself.

【话术】软化风格意见

中文：这是一个 nit，可以最后再处理：这里的 `if (x != null)` 可以写成可空链或 `mapNotNull`，会更 Kotlin 一点，但它不是我最担心的问题。

English: This is a nit and I would not block on it: we could make this more idiomatic Kotlin with a null-safe chain, but the lifecycle issue above is the one I would prioritize.

【设计思想】会提问也是 senior signal

不要把未知条件硬猜成事实。问“这个方法是从哪个线程调用的？”“这个 scope 谁取消？”“这个 Flow 是 cold 还是 hot？”比直接断言“这里一定错了”更像真实 reviewer。正确的 hedge 不是怂，而是在约束不完整时保护判断质量。

4.5 七类 Checklist

下面的 checklist 不是让你机械读完，而是让你在沉默时有抓手。读到某类代码，就快速过对应问题。

类别	评审时间自己什么
空安全	网络字段、Room 字段、Intent extra 是否可空？有没有 <code>!!</code> 、 <code>lateinit</code> 未初始化、 <code>first()</code> 空集合崩溃？错误态是否被建模成 sealed state？详见第六章。
协程 Scope 与 Dispatcher	是否用了 <code>GlobalScope</code> ？协程是否跟 <code>ViewModel</code> 、 <code>viewLifecycleOwner</code> 或 <code>WorkManager</code> 绑定？IO 是否跑在主线程？dispatcher 是否可注入？详见第四章。
Flow / StateFlow / LiveData	Fragment 收集是否用 <code>repeatOnLifecycle</code> ？ <code>StateFlow</code> 是否被用来发一次性事件？ <code>stateIn</code> 的 <code>SharingStarted</code> 是否合理？上游是否重复订阅？详见第四章。
Compose 副作用与重组	网络请求是否直接写在 Composable body？ <code>LaunchedEffect</code> key 是否稳定？ <code>remember</code> 是否遗漏状态键？派生状态是否导致过度重组？详见第六章。
生命周期与内存泄漏	Fragment view 是否在 <code>onDestroyView</code> 后仍被持有？listener、callback、adapter、binding 是否释放？是否把 Activity context 存进单例？详见第五章。
RecyclerView / 列表	是否 <code>notifyDataSetChanged()</code> 全量刷新？ <code>DiffUtil</code> item identity 是否稳定？异步 submit 与旧列表引用是否竞态？点击 position 是否过期？详见第五章。
Kotlin 惯用性	是否 Java 味空判断、可变集合外泄、data class 被错误复用、 <code>Result</code> 被吞、 <code>runCatching</code> 吃掉取消？是否能用不可变快照表达状态？详见第六章。

再把它拆成可背的问题：

- **空安全**：这个值来自不可信边界吗？如果为 `null`，用户看到的是空态、错误态还是 crash？`?: return` 是否偷偷吞掉了业务失败？
- **协程**：这个 job 谁拥有？屏幕走了会不会取消？异常会不会击穿父 scope？是否在 `catch (Exception)` 中吞了 `CancellationException`？
- **Flow**：这是 state 还是 event？收集是否随生命周期停止？`collectLatest` 会不会取消一半写库操作？`flowOn` 是否改变了预期线程？
- **Compose**：副作用是否放进 `LaunchedEffect` / `DisposableEffect`？key 改变时是否会重启？重组是否导致重复请求？
- **生命周期**：谁注册，谁反注册？谁持有 view？长生命周期对象是否引用短生命周期对象？
- **列表**：item identity 与 content equality 是否分开？列表 mutation 是否在 submit 后继续发生？点击回调是否用 `bindingAdapterPosition` 并检查 `NO_POSITION`？
- **惯用性**：可变状态是否封装？public API 是否暴露 `MutableStateFlow`？Java getter/setter 风格是否掩盖副作用？

4.6 一个完整的评审演练

假设你在 CodeSignal 里看到一个新改动：课程结束页要加载总结卡片，展示 XP、连胜文案和推荐下一课。代码很短，但混了 Repository、ViewModel、Room、网络和 UI state。

```

1  data class LessonSummaryUi(
2      val title: String,
3      val xpText: String,
4      val streakText: String,
5      val nextLessonId: String?
6  )
7
8  class LessonSummaryViewModel(
9      private val sessionRepository: SessionRepository,
10     private val userRepository: UserRepository
11 ) : ViewModel() {
12
13     private val _summary = MutableStateFlow<LessonSummaryUi?>(null)
14     val summary: StateFlow<LessonSummaryUi?> = _summary
15
16     fun load(sessionId: String) {
17         viewModelScope.launch {
18             val session = sessionRepository.getSession(sessionId)
19             val user = userRepository.cachedUser!!
20             _summary.value = LessonSummaryUi(
21                 title = "Great job, ${user.displayName}!",
22                 xpText = "+${session.xpEarned} XP",
23                 streakText = "${user.streakDays + 1} day streak",
24                 nextLessonId = session.recommendations.first().lessonId
25             )
26             userRepository.saveLastSeenLesson(session.recommendations.first().
27                 lessonId)
28         }
29     }
30
31     class SessionRepository(
32         private val api: SessionApi,
33         private val dao: SessionDao
34     ) {
35         suspend fun getSession(sessionId: String): SessionSummary {
36             val cached = dao.findSummary(sessionId)
37             return cached ?: api.fetchSessionSummary(sessionId).also {
38                 dao.insert(it)
39             }
40         }
41     }

```

第一遍看数据流与线程边界:load 在 viewModelScope 中启动,默认在主线程;SessionRepository.getSe

内部没有切 dispatcher，却做了 Room 查询、网络请求和写库。即使 Room suspend DAO 通常会处理线程，代码片段本身没有表达契约；如果 `dao.findSummary` 是同步 DAO 或封装层阻塞调用，就会把主线程卡住。更稳的评审说法是：线程契约没有由被调用方保证。

第二遍看生命周期与所有权：这里用的是 `viewModelScope`，比 `GlobalScope` 好；View-Model 清掉后 `job` 会取消。主要生命周期问题不在这里。继续看状态所有权：`summary` 暴露成只读 `StateFlow`，这个方向是对的。

第三遍看局部正确性与惯用性：`cachedUser!!` 是明显崩溃风险；`recommendations.first()` 在空推荐时也会崩溃，而且调用了两次；UI 文案把 `streakDays + 1` 写死，可能与服务端结算口径不一致；`saveLastSeenLesson` 是副作用，和 UI state 组装混在一起，测试不够清楚。

按优先级，你可以这样输出：

1. **崩溃风险**： `cachedUser!!` 与 `recommendations.first()` 都假设远端和缓存一定完整；新用户、离线、推荐为空时会 crash。
2. **ANR / 线程契约风险**： `Repository suspend` 函数没有自保 dispatcher，调用方默认主线程时可能把 Room 或磁盘 IO 放到 UI 线程。
3. **数据一致性**：连胜文案在客户端 + 1，可能和后端最终结果不一致；最好使用 session summary 返回的已结算 streak。
4. **可测试性**：保存 last seen lesson 的副作用和 state 组装耦合，建议明确错误态、空推荐态，并注入 dispatcher。
5. **nit**： `first()` 结果应保存为局部变量，避免重复取值，也让意图更清楚。

```

1 data class LessonSummaryUi(
2     val title: String,
3     val xpText: String,
4     val streakText: String,
5     val nextLessonId: String?
6 )
7
8 sealed interface LessonSummaryState {
9     data object Loading : LessonSummaryState
10    data class Ready(val ui: LessonSummaryUi) : LessonSummaryState
11    data class Error(val message: String) : LessonSummaryState
12 }
13
14 class LessonSummaryViewModel(
15     private val sessionRepository: SessionRepository,
16     private val userRepository: UserRepository
17 ) : ViewModel() {
18
19     private val _summary = MutableStateFlow<LessonSummaryState>(
20         LessonSummaryState.Loading)

```

```

20  val summary: StateFlow<LessonSummaryState> = _summary
21
22  fun load(sessionId: String) {
23      viewModelScope.launch {
24          _summary.value = LessonSummaryState.Loading
25          val result = runCatching {
26              val session = sessionRepository.getSession(sessionId)
27              val user = userRepository.cachedUser()
28              ?: error("User cache is not ready")
29              val nextLesson = session.recommendations.firstOrNull()
30
31              nextLesson?.let { userRepository.saveLastSeenLesson(it.lessonId)
32          }
33
34          LessonSummaryUi(
35              title = "Great job, ${user.displayName}!",
36              xpText = "+${session.xpEarned} XP",
37              streakText = "${session.streakDaysAfterSession} day streak",
38              nextLessonId = nextLesson?.lessonId
39          )
40      }
41      _summary.value = result.fold(
42          onSuccess = { LessonSummaryState.Ready(it) },
43          onFailure = { LessonSummaryState.Error("We could not load your
44              summary.") }
45      )
46  }
47
48  class SessionRepository(
49      private val api: SessionApi,
50      private val dao: SessionDao,
51      private val ioDispatcher: CoroutineDispatcher
52  ) {
53      suspend fun getSession(sessionId: String): SessionSummary = withContext(
54          ioDispatcher) {
55          val cached = dao.findSummary(sessionId)
56          cached ?: api.fetchSessionSummary(sessionId).also { dao.insert(it) }
57      }
58  }

```


【陷阱】修复也要分层

不要把所有问题都塞进一个“更 Kotlin”的重写里。真正的修复顺序是：先把崩溃路径变成状态，再把线程契约收进 Repository，最后再清理重复表达。这样面试官能看到你是在按风险推进，而不是在炫技。

【要点】五行速记

Code Review 轮考的是风险排序，不是 nit 数量。
先说正确性、崩溃、ANR、数据损坏，再说风格。
三遍读：数据流与线程，生命周期与所有权，语法与惯用法。
不确定时提问并说明条件，不要把猜测说成事实。
每个问题都要带 impact、触发条件和具体修复。

Chapter 5

查错题集 A：协程、Flow 与并发

在 Duolingo Android 面试的查错轮里，协程、Flow 与并发几乎是绕不开的核心区。原因很直接：[\[官方\]](#)¹ Duolingo Android 已经迁移到 100% Kotlin；[\[官方\]](#)² Android Reboot 之后的架构强调 MVVM、Repository、Dagger/Hilt 与模块化；而[\[面经报告\]](#)面经里也反复出现 GlobalScope、Fragment 中启动协程、Flow 收集和列表更新这类题。换句话说，这一章不是“并发语法大全”，而是面试官最可能用来判断你是否真写过大型 Android Kotlin 代码的故障样本。

5.1 在 GlobalScope 里发起网络请求

查错 #1 | 类别：协程 Scope

场景：用户完成一节课后，客户端要上报 session 结果、刷新 XP 和连胜，再把结果展示在完成页。[\[面经报告\]](#)候选人报告过类似模式：代码里用 GlobalScope 发起网络请求，然后在响应后写回 UI 状态。

```
1 data class LessonResultUi(  
2     val xp: Int,  
3     val streakDays: Int,  
4     val isSynced: Boolean  
5 )  
6  
7 class LessonViewModel(  
8     private val sessionRepository: SessionRepository,  
9     private val streakRepository: StreakRepository  
10 ) : ViewModel() {  
11  
12     private val _result = MutableStateFlow<LessonResultUi?>(null)  
13     val result: StateFlow<LessonResultUi?> = _result  
14  
15     fun onLessonFinished(sessionId: String, answers: List<Answer>) {  
16         _result.value = LessonResultUi(xp = 0, streakDays = 0, isSynced = false)
```

¹<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

²<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

```

17
18     GlobalScope.launch(Dispatchers.IO) {
19         val submission = sessionRepository.submitSession(sessionId, answers)
20         val streak = streakRepository.refreshStreak()
21
22         _result.value = LessonResultUi(
23             xp = submission.xpEarned,
24             streakDays = streak.days,
25             isSynced = true
26         )
27     }
28 }
29 }

```

你在面试中该怎么说

【话术】影响先行

I would flag this as a lifecycle and crash risk. The work is launched in `GlobalScope`, so it can outlive the `LessonViewModel`; after the screen is gone it may still write UI state, and any uncaught exception is no longer tied to the screen's structured concurrency. I would use `viewModelScope` for screen-owned work, or move must-finish uploads to `WorkManager` or an injected application scope.

根因分析：`GlobalScope` 的问题不是“看起来不优雅”，而是它绕开了结构化并发。用户提交课程后立刻退出、旋屏或系统回收页面时，`viewModelScope` 会随 `ViewModel` 清理而取消；`GlobalScope` 不会。它还让异常脱离父作用域：网络失败、解析失败、服务器返回异常状态，如果没有局部捕获，就可能走到全局异常处理，表现为进程崩溃或不可预测日志。更隐蔽的是幽灵更新：旧页面发出的请求晚于新页面返回，把旧 session 的 XP 写进新状态。[\[高置信推断\]](#)

```

1  data class LessonResultUi(
2      val xp: Int,
3      val streakDays: Int,
4      val isSynced: Boolean
5  )
6
7  class LessonViewModel(
8      private val sessionRepository: SessionRepository,
9      private val streakRepository: StreakRepository
10 ) : ViewModel() {
11
12     private val _result = MutableStateFlow<LessonResultUi?>(null)
13     val result: StateFlow<LessonResultUi?> = _result
14
15     fun onLessonFinished(sessionId: String, answers: List<Answer>) {
16         _result.value = LessonResultUi(xp = 0, streakDays = 0, isSynced = false)

```

```

17
18     viewModelScope.launch {
19         val ui = runCatching {
20             val submission = sessionRepository.submitSession(sessionId,
21                 answers)
22             val streak = streakRepository.refreshStreak()
23             LessonResultUi(
24                 xp = submission.xpEarned,
25                 streakDays = streak.days,
26                 isSynced = true
27             )
28         }.getOrElse {
29             LessonResultUi(xp = 0, streakDays = 0, isSynced = false)
30         }
31         _result.value = ui
32     }
33 }

```

【设计思想】Scope 表达所有权

协程 scope 的第一含义不是“在哪个线程跑”，而是“谁拥有这段工作”。页面拥有的工作用 `viewModelScope` 或 `viewLifecycleOwner.lifecycleScope`；应用级工作要由应用级组件拥有；必须可靠完成的延迟工作交给 `WorkManager`。把所有权说清楚，dispatcher 才有意义。

【追问】“但这个上报就是希望用户退出后也要完成，怎么办？”

这正是区分中级和高级候选人的追问。答案不是“那就 `GlobalScope`”，而是重新定义工作类型。如果上报必须在用户离开后继续、需要重试、需要网络约束、需要进程死亡后恢复，应使用 `WorkManager`，把 `sessionId` 与幂等 key 写入任务输入，服务端也要能安全处理重复提交。如果它只需要在应用进程内尽量完成，可以通过 DI 注入一个 application-scoped `CoroutineScope(SupervisorJob() + Dispatchers.IO)`，由 `Application` 或单例组件持有，并且仍要有错误记录、重试策略和幂等设计。`GlobalScope` 最大的问题是没有可测试、可取消、可观察的所有者。

【陷阱】把 `Dispatchers.IO` 当成所有权

`GlobalScope.launch(Dispatchers.IO)` 只说明线程池，不说明生命周期。IO dispatcher 不能帮你取消旧页面请求，也不能帮你把异常归属到正确的业务失败域。

5.2 在主线程上做 Room 查询与磁盘 IO

查错 #2 | 类别: **Dispatcher**

场景：用户打开排行榜页时，产品希望先展示本地缓存，再异步刷新远端排名。为了让首屏快，

开发者写了一个“同步读缓存”的路径。

```
1  data class LeaderboardUi(  
2      val rows: List<LeaderboardRow>,  
3      val fromCache: Boolean  
4  )  
5  
6  class LeaderboardViewModel(  
7      private val repository: LeaderboardRepository  
8  ) : ViewModel() {  
9  
10     private val _ui = MutableStateFlow(LeaderboardUi(emptyList(), fromCache =  
11         true))  
12     val ui: StateFlow<LeaderboardUi> = _ui  
13  
14     fun load(courseId: String) {  
15         viewModelScope.launch(Dispatchers.Main) {  
16             val cached = repository.loadCachedLeaderboard(courseId)  
17             _ui.value = LeaderboardUi(cached, fromCache = true)  
18  
19             val fresh = repository.refreshLeaderboard(courseId)  
20             _ui.value = LeaderboardUi(fresh, fromCache = false)  
21         }  
22     }  
23  
24     class LeaderboardRepository(  
25         private val dao: LeaderboardDao,  
26         private val api: LeaderboardApi  
27     ) {  
28         suspend fun loadCachedLeaderboard(courseId: String): List<LeaderboardRow> {  
29             return dao.selectRowsForCourse(courseId)  
30         }  
31  
32         suspend fun refreshLeaderboard(courseId: String): List<LeaderboardRow> {  
33             val rows = api.fetchLeaderboard(courseId)  
34             dao.replaceRows(courseId, rows)  
35             return rows  
36         }  
37     }  
}
```

你在面试中该怎么说

【话术】线程契约

I would flag this as an ANR risk. The ViewModel launches on Main and the repository does not enforce an IO boundary around the database read or write. Even if some DAO methods are suspend, I do not want every caller to remember the threading rule; the repository should own its dispatcher contract and inject the dispatcher for tests.

根因分析：Android 主线程负责输入、绘制和生命周期分发。排行榜首屏看似只是“读一下缓存”，但磁盘 IO、Cursor 读取、反序列化、批量写库都可能在低端设备上卡住 UI。**[官方]**³ Duolingo Android Reboot 的官方案例里，ANR 曾经每月恶化 5–10%，重构后 ANR 下降 41%；这类数字说明主线程重活不是小洁癖，而是会被用户真实感知的稳定性问题。**[高置信推断]** 在评审里，最好的原则是：**由被调用方保证自己的线程约束**。调用方可以在主线程启动协程，但 Repository 的 suspend 函数应该自己把阻塞或重 IO 部分切到 Dispatchers.IO。

```
1 class LeaderboardViewModel(  
2     private val repository: LeaderboardRepository  
3 ) : ViewModel() {  
4  
5     private val _ui = MutableStateFlow(LeaderboardUi(emptyList(), fromCache =  
6         true))  
7     val ui: StateFlow<LeaderboardUi> = _ui  
8  
9     fun load(courseId: String) {  
10         viewModelScope.launch {  
11             val cached = repository.loadCachedLeaderboard(courseId)  
12             _ui.value = LeaderboardUi(cached, fromCache = true)  
13  
14             val fresh = repository.refreshLeaderboard(courseId)  
15             _ui.value = LeaderboardUi(fresh, fromCache = false)  
16         }  
17     }  
18  
19 class LeaderboardRepository(  
20     private val dao: LeaderboardDao,  
21     private val api: LeaderboardApi,  
22     private val ioDispatcher: CoroutineDispatcher  
23 ) {  
24     suspend fun loadCachedLeaderboard(courseId: String): List<LeaderboardRow> =  
25         withContext(ioDispatcher) {  
26             dao.selectRowsForCourse(courseId)  
27         }  
28  
29     suspend fun refreshLeaderboard(courseId: String): List<LeaderboardRow> =
```

³<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

```
30     withContext(ioDispatcher) {  
31         val rows = api.fetchLeaderboard(courseId)  
32         dao.replaceRows(courseId, rows)  
33         rows  
34     }  
35 }
```

【设计思想】线程约束属于 API 契约

如果一个 suspend 函数只能在后台线程安全调用，它就应该在内部切线程，或者在名字和文档中非常明确地表达限制。Android 项目里更常见的好选择是注入 `CoroutineDispatcher`，让生产环境用 IO，测试环境用 test dispatcher。

【陷阱】IO 与 Default 不是“后台线程”的同义词

`Dispatchers.IO` 适合阻塞 IO：磁盘、网络库阻塞调用、文件读写。`Dispatchers.Default` 适合 CPU 密集计算：排序、大 JSON 纯计算转换、diff 计算。把 CPU 重活扔进 IO 会挤占 IO 线程；把阻塞 IO 扔进 Default 会饿死计算任务。面试里能说出这个差别，是比“不要在主线程”更强的信号。

【追问】“Room 的 suspend DAO 不是已经异步了吗？”

可以回答：很多 Room suspend 查询会在 Room 自己的 executor 上运行，但评审代码时不能把所有封装都假设为理想状态。Repository 里可能还包含文件读写、缓存解析、加密、同步 DAO 或第三方 SDK。把 dispatcher 边界放在 Repository 仍然能表达契约，并让测试更可控。

5.3 不可取消的轮询循环

查错 #3 | 类别：协作式取消

场景：排行榜页打开后每 5 秒刷新一次，用户可以一边做题一边看排名变化。开发者为了快速实现，在 ViewModel 里写了一个轮询 job。

```
1 class LeaderboardViewModel(  
2     private val repository: LeaderboardRepository  
3 ) : ViewModel() {  
4  
5     private val _rows = MutableStateFlow<List<LeaderboardRow>>(emptyList())  
6     val rows: StateFlow<List<LeaderboardRow>> = _rows  
7  
8     private var pollingJob: Job? = null  
9  
10    fun startPolling(leagueId: String) {  
11        pollingJob?.cancel()  
12        pollingJob = viewModelScope.launch(Dispatchers.IO) {
```

```

13         while (true) {
14             try {
15                 val latest = repository.fetchLeagueRows(leagueId)
16                 _rows.value = latest
17                 Thread.sleep(5_000)
18             } catch (e: Exception) {
19                 logger.warn("leaderboard polling failed", e)
20             }
21         }
22     }
23 }
24
25 fun stopPolling() {
26     pollingJob?.cancel()
27 }
28 }

```

你在面试中该怎么说

【话术】取消语义

I would flag this as a cancellation bug and a thread waste. The coroutine uses `while (true)` plus `Thread.sleep`, and it catches `Exception`, so cancellation can be delayed or swallowed. I would make the loop cooperative with `isActive` and `delay`, and avoid catching `CancellationException` as a normal failure.

根因分析：Kotlin 协程取消是协作式的。调用 `cancel()` 只是把 Job 标记为取消；协程要在挂起点、`isActive`、`ensureActive()` 或 `yield()` 处观察到取消。`Thread.sleep` 会阻塞真实线程，不是可取消挂起点；`while (true)` 没有检查 Job 状态；`catch (Exception)` 又可能把 `CancellationException` 当成普通错误吞掉。用户离开排行榜页后，轮询可能继续占着 IO 线程、继续打网络、继续写状态，表现为电量消耗、重复请求或页面回来后看到旧数据闪动。[\[高置信推断\]](#)

```

1 class LeaderboardViewModel(
2     private val repository: LeaderboardRepository
3 ) : ViewModel() {
4
5     private val _rows = MutableStateFlow<List<LeaderboardRow>>(emptyList())
6     val rows: StateFlow<List<LeaderboardRow>> = _rows
7
8     private var pollingJob: Job? = null
9
10    fun startPolling(leagueId: String) {
11        pollingJob?.cancel()
12        pollingJob = viewModelScope.launch {
13            while (isActive) {
14                try {

```



```
15         _rows.value = repository.fetchLeagueRows(leagueId)
16     } catch (e: CancellationException) {
17         throw e
18     } catch (e: IOException) {
19         logger.warn("leaderboard polling failed", e)
20     }
21     delay(5_000)
22 }
23 }
24 }
25
26 fun stopPolling() {
27     pollingJob?.cancel()
28     pollingJob = null
29 }
30 }
31
32 class LeaderboardRepository(
33     private val api: LeaderboardApi
34 ) {
35     fun leagueRows(leagueId: String): Flow<List<LeaderboardRow>> = flow {
36         while (true) {
37             emit(api.fetchLeagueRows(leagueId))
38             delay(5_000)
39         }
40     }
41 }
```

【设计思想】取消点要出现在循环里

长循环有三件套：循环条件用 `isActive`，等待用 `delay` 而不是 `Thread.sleep`，CPU 密集循环中用 `ensureActive()` 或 `yield()` 给取消机会。否则 `cancel()` 只是一个愿望。

【陷阱】catch Exception 吞掉 CancellationException

`CancellationException` 继承自 `Exception`。如果你写 `catch (e: Exception)` 后只打日志不重新抛出，父 scope 会以为子任务已经“正常处理失败”，取消传播被破坏。高级评审应主动指出：要么先 `catch (e: CancellationException) throw e`，要么只捕获你真能恢复的异常类型。

【追问】“用 Flow 轮询是不是就自动生命周期安全？”

不是。`flow while (true) ...` 只是把轮询变成 cold stream；它仍然需要在正确 scope 中收集。Fragment 应结合 `repeatOnLifecycle`，ViewModel 可以用

```
stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), initial)
```

控制上游何时启动和停止。

5.4 async 的异常击穿了父作用域

查错 #4 | 类别: **结构化并发**

场景: 课程加载页希望并行拉取课程内容、用户进度和发音音频清单。内容是必需的；音频清单可以稍后补；进度失败时可以用本地默认值。

```
1 data class LessonBundle(  
2     val lesson: LessonContent,  
3     val progress: UserProgress,  
4     val audio: List<AudioClip>  
5 )  
6  
7 class LessonLoader(  
8     private val lessonRepository: LessonRepository,  
9     private val progressRepository: ProgressRepository,  
10    private val audioRepository: AudioRepository  
11 ) {  
12     suspend fun load(lessonId: String): LessonBundle = coroutineScope {  
13         val lesson = async { lessonRepository.fetchLesson(lessonId) }  
14         val progress = async { progressRepository.fetchProgress(lessonId) }  
15         val audio = async { audioRepository.prefetchAudioList(lessonId) }  
16  
17         LessonBundle(  
18             lesson = lesson.await(),  
19             progress = progress.await(),  
20             audio = audio.await()  
21         )  
22     }  
23 }
```

你在面试中该怎么说

【话术】失败域

I would flag this as a failure-domain issue. All three async children are in the same `coroutineScope`, so if optional audio prefetch fails, it cancels the whole load and the lesson may not open. I would separate required data from degradable data, using `supervisorScope` and per-branch error handling.

根因分析: `coroutineScope` 的语义是结构化并发：任何子协程失败，父 `scope` 失败，兄弟协程被取消。这对“任一失败就整体失败”的事务很好，但课程加载不是纯事务。课程内容失败当然不能继续；音频预加载失败却不应该让整节课打不开；进度失败也许可以使用默认进度或缓存。原

始代码把三个任务放在同一个失败域里，导致最不重要的分支拥有了杀死整个页面的权力。另一个常见变体是创建 `async` 后不 `await()`，异常会在结构化边界或父 `job` 上以不直观的方式出现，让排查更困难。[\[高置信推断\]](#)

```
1 class LessonLoader(  
2     private val lessonRepository: LessonRepository,  
3     private val progressRepository: ProgressRepository,  
4     private val audioRepository: AudioRepository  
5 ) {  
6     suspend fun load(lessonId: String): LessonBundle = supervisorScope {  
7         val lessonDeferred = async { lessonRepository.fetchLesson(lessonId) }  
8         val progressDeferred = async {  
9             runCatching { progressRepository.fetchProgress(lessonId) }  
10            .getOrElse { UserProgress.empty(lessonId) }  
11        }  
12        val audioDeferred = async {  
13            runCatching { audioRepository.prefetchAudioList(lessonId) }  
14            .getOrElse { emptyList() }  
15        }  
16  
17        val lesson = lessonDeferred.await()  
18        LessonBundle(  
19            lesson = lesson,  
20            progress = progressDeferred.await(),  
21            audio = audioDeferred.await()  
22        )  
23    }  
24 }
```

【设计思想】失败域的划分

并行任务里哪些是必需的、哪些是可降级的，这是设计判断，不是语法问题。一个好的 reviewer 会先问业务语义：没有课程内容能不能显示？没有音频能不能先进入？没有进度能不能用默认值？然后再选择 `coroutineScope`、`supervisorScope` 或显式 `try-catch`。

【陷阱】`async` 不是“更高级的 `launch`”

`async` 代表“我要一个结果”，所以结果必须被 `await()`，异常也通常在 `await()` 处处理。只为了并发执行副作用而用 `async`，然后不等待，是代码味道。没有返回值的工作优先用 `launch`，并明确它的异常处理和所有权。

【追问】“launch、async 和 CoroutineExceptionHandler 有什么差异?”

launch 的未捕获异常会立即向父 job 传播; async 的异常保存在 Deferred 中,但仍然受结构化并发约束,父子关系会影响取消。CoroutineExceptionHandler 主要处理 root coroutine 的未捕获异常;它不是给普通子协程做业务恢复的万能 catch。面试里应强调:业务可恢复错误要在对应分支处理,handler 只适合作为最后的日志或崩溃边界。

5.5 在 Fragment 里收集 Flow 却没有绑定生命周期

查错 #5 | 类别: **Flow 收集**

场景: 课程列表页订阅 ViewModel 的 StateFlow, 渲染课程卡片、下载状态和继续学习按钮。
[面经报告] 面经报告过类似 Fragment 中启动协程处理网络响应、但没有正确绑定生命周期的代码。

```
1 class CourseListFragment : Fragment(R.layout.fragment_course_list) {
2
3     private val viewModel: CourseListViewModel by viewModels()
4     private var binding: FragmentCourseListBinding? = null
5     private val adapter = CourseAdapter()
6
7     override fun onCreateView(
8         inflater: LayoutInflater,
9         container: ViewGroup?,
10        savedInstanceState: Bundle?
11    ): View {
12        binding = FragmentCourseListBinding.inflate(inflater, container, false)
13        return binding!!.root
14    }
15
16    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
17        binding!!.courseList.adapter = adapter
18
19        lifecycleScope.launch {
20            viewModel.state.collect { state ->
21                binding!!.progress.isVisible = state.loading
22                adapter.submitList(state.courses)
23                if (state.navigateToLessonId != null) {
24                    findNavController().navigate(
25                        CourseListFragmentDirections.openLesson(state.
navigateToLessonId)
26                    )
27                }
28            }
29        }
30    }
31}
```

```
32     override fun onDestroyView() {  
33         binding = null  
34         super.onDestroyView()  
35     }  
36 }
```

你在面试中该怎么说

【话术】生命周期绑定

I would flag this as a lifecycle collection bug. The coroutine is launched in the Fragment's `lifecycleScope`, not the view lifecycle, so it can keep collecting after `onDestroyView` and touch a null or stale binding. I would collect from `viewLifecycleOwner.lifecycleScope` with `repeatOnLifecycle(STARTED)`.

根因分析：Fragment 的生命周期和它的 view 生命周期不是同一个东西。旋屏或导航时，Fragment 实例可能还活着，但 view 已经销毁并重建。原始代码用 `lifecycleScope`，协程绑定到 Fragment，而不是 view；`binding` 在 `onDestroyView` 置空后，collector 仍可能收到 `StateFlow` 更新并触碰 `binding!!`，导致 crash。即使不 crash，也可能在 view 重建后重复订阅，老 collector 和新 collector 同时处理同一个 state，出现一次导航触发两次、Toast 重复、adapter 收到交错列表等问题。[\[高置信推断\]](#)

```
1  class CourseListFragment : Fragment(R.layout.fragment_course_list) {  
2  
3      private val viewModel: CourseListViewModel by viewModels()  
4      private var binding: FragmentCourseListBinding? = null  
5      private val adapter = CourseAdapter()  
6  
7      override fun onCreateView(  
8          inflater: LayoutInflater,  
9          container: ViewGroup?,  
10         savedInstanceState: Bundle?  
11     ): View {  
12         binding = FragmentCourseListBinding.inflate(inflater, container, false)  
13         return requireNotNull(binding).root  
14     }  
15  
16     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
17         val currentBinding = requireNotNull(binding)  
18         currentBinding.courseList.adapter = adapter  
19  
20         viewLifecycleOwner.lifecycleScope.launch {  
21             viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {  
22                 viewModel.state.collect { state ->  
23                     val activeBinding = binding ?: return@collect  
24                     activeBinding.progress.isVisible = state.loading
```

```

25         adapter.submitList(state.courses)
26     }
27 }
28 }
29
30 viewLifecycleOwner.lifecycleScope.launch {
31     viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
32         viewModel.events.collect { event ->
33             when (event) {
34                 is CourseListEvent.OpenLesson -> findNavController().
navigate(
35                     CourseListFragmentDirections.openLesson(event.
lessonId)
36                 )
37             }
38         }
39     }
40 }
41 }
42
43 override fun onDestroyView() {
44     binding?.courseList?.adapter = null
45     binding = null
46     super.onDestroyView()
47 }
48 }

```

【设计思想】repeatOnLifecycle 是取消并重启

repeatOnLifecycle(STARTED) 不是简单“暂停回调”。当生命周期低于 STARTED，它会取消内部收集；回到 STARTED，再重新启动收集。这能释放上游资源，也能避免后台页面继续渲染。课程列表这类可见 UI 通常用 STARTED：可见时收集，不可见时停止。

【陷阱】StateFlow 不适合一次性导航事件

StateFlow 表达“当前状态”，新 collector 会立刻收到最新值。如果把 navigateToLessonId 放在 state 里，旋屏后新 collector 可能再次导航。一次性事件更适合 Channel 或配置合理的 SharedFlow，并且事件消费要和生命周期配合。

【追问】“stateIn 的 WhileSubscribed(5000) 是干什么的？”

stateIn(scope, SharingStarted.WhileSubscribed(5000), initial) 里的 5000ms 是停止上游前的宽限期。旋屏时旧 collector 取消、新 collector 很快创建；宽限期可以避免上游冷 Flow 立刻停掉又重启，减少重复网络请求或数据库订阅抖动。它不是魔法数字，应该按

上游成本和页面切换习惯设置。

5.6 被多个协程并发修改的共享可变状态

查错 #6 | 类别: **竞态与可见性**

场景：一个用户状态对象同时被 XP 上报、连胜同步和课程进度更新读写。[\[官方\]](#)⁴ 这正是官方 Android Reboot 故事里 DuoState 反模式的缩影：多个产品域共享巨大可变状态，最终带来稳定性和性能代价。

```
1 data class UserState(  
2     val xp: Int,  
3     val streakDays: Int,  
4     val completedLessons: Set<String>  
5 )  
6  
7 class UserStateStore(  
8     private val api: UserApi,  
9     private val dao: UserStateDao  
10 ) {  
11     private val _state = MutableStateFlow(  
12         UserState(xp = 0, streakDays = 0, completedLessons = emptySet())  
13     )  
14     val state: StateFlow<UserState> = _state  
15  
16     suspend fun applyLessonResult(result: LessonResult) = coroutineScope {  
17         launch(Dispatchers.IO) {  
18             api.reportXp(result.sessionId, result.xpEarned)  
19             _state.value = _state.value.copy(  
20                 xp = _state.value.xp + result.xpEarned  
21             )  
22         }  
23  
24         launch(Dispatchers.Default) {  
25             _state.value = _state.value.copy(  
26                 completedLessons = _state.value.completedLessons + result.  
27                 lessonId  
28             )  
29         }  
30  
31         launch(Dispatchers.IO) {  
32             val streak = api.refreshStreak()  
33             _state.value = _state.value.copy(streakDays = streak.days)  
34             dao.save(_state.value)
```

⁴<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

```

34     }
35 }
36 }

```

你在面试中该怎么说

【话术】共享可变状态

I would flag this as a race on shared mutable state. `MutableStateFlow.value = value.copy(...)` is a read-modify-write sequence, and three coroutines on different dispatchers can overwrite each other's updates. I would use atomic `update`, immutable snapshots, or confine all mutation to one owner.

根因分析: `MutableStateFlow.value` 的单次读取和单次写入是线程安全的，但“读旧值、基于旧值 `copy`、再写回”这一串不是自动原子的。三个协程各自从 `_state.value` 读到旧快照，再写回自己的字段更新，后写的人会覆盖先写的人。用户看到的现象可能是：课程完成了但 XP 少加一次；连胜更新后完成课程集合回滚；本地保存的状态和 UI 显示不一致。[\[高置信推断\]](#) 这类问题难复现，因为它依赖网络时序、dispatcher 调度和设备性能。

```

1 初始 state = { xp=100, streak=7, completed=[] }
2
3 协程 A 读到 old = { xp=100, streak=7, completed=[] }
4 协程 B 读到 old = { xp=100, streak=7, completed=[] }
5 A 写回 { xp=115, streak=7, completed=[] }
6 B 写回 { xp=100, streak=7, completed=[lesson-3] }
7
8 结果: lesson 完成被记录了, 但 XP 更新丢失。

```

```

1 data class UserState(
2     val xp: Int,
3     val streakDays: Int,
4     val completedLessons: Set<String>
5 )
6
7 class UserStateStore(
8     private val api: UserApi,
9     private val dao: UserStateDao,
10    private val ioDispatcher: CoroutineDispatcher
11 ) {
12    private val _state = MutableStateFlow(
13        UserState(xp = 0, streakDays = 0, completedLessons = emptySet())
14    )
15    val state: StateFlow<UserState> = _state
16
17    suspend fun applyLessonResult(result: LessonResult) = supervisorScope {

```



```
18     val xpDeferred = async(ioDispatcher) {
19         api.reportXp(result.sessionId, result.xpEarned)
20         result.xpEarned
21     }
22     val streakDeferred = async(ioDispatcher) {
23         runCatching { api.refreshStreak().days }.getOrNull()
24     }
25
26     val earnedXp = xpDeferred.await()
27     val streakDays = streakDeferred.await()
28
29     _state.update { current ->
30         current.copy(
31             xp = current.xp + earnedXp,
32             streakDays = streakDays ?: current.streakDays,
33             completedLessons = current.completedLessons + result.lessonId
34         )
35     }
36
37     withContext(ioDispatcher) {
38         dao.save(_state.value)
39     }
40 }
41 }
```

【设计思想】把“谁能改这个状态”收敛到一个地方

Duolingo 用 8 周停更换来的教训之一，就是巨大共享可变状态会让产品域互相踩踏。[\[官方\]^a](#) 面试里你要把这个原则说出来：状态可以被很多地方观察，但修改入口要少、要有所有者、要用不可变快照或原子更新表达。

^a<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

【陷阱】`_state.value = _state.value.copy(...)` 不等于原子更新

这行看起来很 Kotlin，但本质是 read-modify-write。`MutableStateFlow.update ...` 会基于 CAS 循环重试，让转换函数应用到最新值上。它解决的是同一个 StateFlow 上的丢失更新，不等于解决所有业务事务；跨数据库、网络和内存状态的一致性仍需要事务或单一写入者设计。

【追问】“为什么 @Volatile 不够？AtomicReference 和 update 又是什么关系？”

@Volatile 只保证可见性：一个线程写入后，另一个线程能看到较新的值。它不保证 $x = x + 1$ 这种复合操作的原子性。AtomicReference.compareAndSet 可以把“如果当前还是我读到的旧值，就写入新值”做成原子条件写；MutableStateFlow.update 的思想类似，会在竞争失败时用最新值重新计算。因此它比 volatile 更适合状态快照的并发更新。

Chapter 6

查错题集 B：生命周期、泄漏与列表

生命周期类查错题，在 Android 评审里权重极高。它们不像语法错误那样立刻炸在开发机上，而是潜伏在“用户反复进出页面、低端机内存紧张、长时间学习会话、网络抖动、快速滚动”这些真实条件里。Duolingo 曾公开复盘 Android Reboot：巨型全局可变状态让 ANR 每月恶化，某次发布还带来崩溃率、慢帧和课程启动速度的明显退化；后来团队停更数周，把架构拉回 MVVM、Repository、依赖注入和模块化，才换来 ANR 与掉帧的下降 [官方]¹。所以查错题里看到生命周期、泄漏、列表刷新，不要把它当“小洁癖”，它常常就是用户体验事故的早期形态。

6.1 Fragment 的 binding 没有在 onDestroyView 置空

查错 #7 | 类别：内存泄漏

场景

课程详情页 LessonDetailFragment 展示课程标题、技能树入口、单词复习提醒和一个底部开始按钮。这个页面常被压入回退栈：用户从课程列表点进详情，返回列表，再点另一个课程。看起来只是一个普通 ViewBinding 写法，但如果 Fragment 实例活着而它的 View 已经销毁，旧的整棵 View 树就会被 binding 悄悄拴住。

```
1 class LessonDetailFragment : Fragment(R.layout.fragment_lesson_detail) {
2     private var _binding: FragmentLessonDetailBinding? = null
3     private val binding: FragmentLessonDetailBinding
4         get() = requireNotNull(_binding)
5
6     private val viewModel: LessonDetailViewModel by viewModels()
7     private val args: LessonDetailFragmentArgs by navArgs()
8
9     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
10         super.onCreateView(view, savedInstanceState)
11         _binding = FragmentLessonDetailBinding.bind(view)
12     }
```

¹<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

```

13         binding.toolbar.setNavigationOnClickListener {
14             findNavController().navigateUp()
15         }
16         binding.startLessonButton.setOnClickListener {
17             viewModel.startLesson(args.lessonId)
18         }
19
20         viewModel.lesson.observe(viewLifecycleOwner) { lesson ->
21             binding.title.text = lesson.title
22             binding.description.text = lesson.description
23             binding.xpReward.text = getString(R.string.xp_reward, lesson.xp)
24             binding.reviewBadge.isVisible = lesson.hasDueReview
25         }
26     }
27
28     override fun onDestroyView() {
29         super.onDestroyView()
30         binding.startLessonButton.setOnClickListener(null)
31         // BUG: _binding 仍然引用已经销毁的 View tree
32     }
33 }

```

【话术】面试里怎么说

我会先说影响：这个 Fragment 放进 back stack 后，Fragment 实例可能继续存活，但它的 View 已经销毁；binding 继续持有旧 View，会泄漏整个课程详情页面。Then I would say: “The lifetime of the Fragment is longer than the lifetime of its view. The fix is to clear the binding in onDestroyView, or better, use a view-binding delegate so the rule is enforced structurally.”

根因分析

Fragment 有两条生命周期：Fragment 自己的生命周期，和它所创建的 View 的生命周期。进入回退栈时，Fragment 对象可能还在，因为导航组件需要保留它的状态；但 `onDestroyView` 已经运行，原来的 root view、子 View、drawable、adapter、listener 都应该被释放。`_binding` 是一个强引用，只要它不被置空，旧 View 树就无法被 GC。开发机上单次打开页面通常看不出问题；真实用户连续学习几十分钟、在课程列表和详情间反复切换时，泄漏会逐步累积，低端设备先表现为卡顿，再表现为 OOM 或后台进程频繁被杀。

这里的危险不只在内存。代码在 `onDestroyView` 里还通过 binding 访问旧 View；如果之后某个异步回调继续使用这个 getter，就会把“已销毁的 UI”伪装成“仍可操作的 UI”。评审时要抓住本质：不是少写一行 `_binding = null`，而是所有引用 View 的对象都必须跟 View 生命周期同生共死。

```

1 class LessonDetailFragment : Fragment(R.layout.fragment_lesson_detail) {
2     private var _binding: FragmentLessonDetailBinding? = null

```

```
3     private val binding: FragmentLessonDetailBinding
4         get() = requireNotNull(_binding) { "View binding is only valid between
5         onViewCreated and onDestroyView." }
6
7     private val viewModel: LessonDetailViewModel by viewModels()
8     private val args: LessonDetailFragmentArgs by navArgs()
9
10    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
11        super.onViewCreated(view, savedInstanceState)
12        _binding = FragmentLessonDetailBinding.bind(view)
13
14        binding.toolbar.setNavigationOnClickListener {
15            findNavController().navigateUp()
16        }
17        binding.startLessonButton.setOnClickListener {
18            viewModel.startLesson(args.lessonId)
19        }
20
21        viewModel.lesson.observe(viewLifecycleOwner) { lesson ->
22            binding.title.text = lesson.title
23            binding.description.text = lesson.description
24            binding.xpReward.text = getString(R.string.xp_reward, lesson.xp)
25            binding.reviewBadge.isVisible = lesson.hasDueReview
26        }
27
28    override fun onDestroyView() {
29        binding.startLessonButton.setOnClickListener(null)
30        _binding = null
31        super.onDestroyView()
32    }
33 }
```

【设计思想】把纪律性问题变成结构性问题

手写 `_binding = null` 靠记性，项目大了总会漏。更好的团队级修法，是引入只在 `viewLifecycleOwner` 活跃期可访问的 `viewBinding()` 属性委托：View 销毁时委托自动清空引用，View 不存在时访问直接抛出带信息的异常。面试里这样讲，会让你从“我知道这一行怎么补”上升到“我知道怎样让整个代码库不再反复犯”。

【追问】“lateinit 在这里会怎样？”

`lateinit var binding` 避免了可空写法，却隐藏了释放时机。它既不能表达“View 销毁后不可访问”，也不会自动清空旧 View；除非你在 `onDestroyView` 里重新赋一个哨兵值，但

Kotlin 没有 “unset lateinit” 的正式 API。因此在 Fragment binding 上，lateinit 往往比可空 backing property 更容易泄漏。

【追问】“为什么 LeakCanary 能抓到它？”

LeakCanary 会在 onDestroyView 后观察被销毁的 root View 是否还能被强引用到。如果引用链显示 LessonDetailFragment -> _binding -> root，它就能指出 Fragment 字段保留了本应回收的 View 树。这个工具抓到的是症状；根治仍是明确所有权边界。

6.2 observe 用了 this 而不是 viewLifecycleOwner

查错 #8 | 类别: 重复订阅

场景

连胜卡片 StreakCardView 在课程详情页顶部展示用户今天是否已经完成练习。产品上它很显眼：用户从课程列表进入详情，返回，再进入另一个课程，如果连胜状态更新时卡片闪两次、弹两次动效，用户会以为奖励系统出了问题。这个 bug 的代码在首次进入时完全正常，因此特别像真实面试题。

```
1 class StreakDetailFragment : Fragment(R.layout.fragment_streak_detail) {
2     private var _binding: FragmentStreakDetailBinding? = null
3     private val binding get() = requireNotNull(_binding)
4     private val viewModel: StreakViewModel by viewModels()
5
6     override fun onCreateView(view: View, savedInstanceState: Bundle?) {
7         super.onCreateView(view, savedInstanceState)
8         _binding = FragmentStreakDetailBinding.bind(view)
9
10        viewModel.streak.observe(this) { streak ->
11            binding.streakCard.setDayCount(streak.days)
12            binding.streakCard.setFrozen(streak.isFrozen)
13            if (streak.extendedToday) {
14                binding.streakCard.playCelebration()
15            }
16        }
17
18        viewModel.loading.observe(this) { loading ->
19            binding.progressBar.isVisible = loading
20            binding.practiceButton.isEnabled = !loading
21        }
22
23        binding.practiceButton.setOnClickListener {
24            viewModel.practiceToday()
```

```

25     }
26 }
27
28 override fun onDestroyView() {
29     _binding = null
30     super.onDestroyView()
31 }
32 }

```

【话术】面试里怎么说

我会先讲用户影响：从回退栈回来后，这段代码会累计 observer，一次 streak 更新可能触发多次 UI 更新和多次庆祝动画，甚至访问已经销毁的 binding。Then: “The observer is attached to the Fragment lifecycle, not the view lifecycle. In a Fragment, UI observers should be scoped to viewLifecycleOwner.”

根因分析

this 在 Fragment 里代表 Fragment 的生命周期，而不是 Fragment 的 View 生命周期。Fragment 进回退栈时 View 被销毁，Fragment 本体仍可能处于未销毁状态；下一次 **onViewCreated** 会创建新 View，同时又注册一组新的 observer。旧 observer 没死，新 observer 又来了，于是同一份 LiveData 变化会驱动多份回调。

这类 bug 的触发条件很产品化：单次进入页面没问题；“进入详情、返回列表、再进入详情、网络刷新连胜状态”才复现。用户看到的是卡片闪烁、按钮状态跳动、动画重复播放。如果旧回调捕获了旧 binding，还可能在 **_binding** 已经为 null 时崩溃。它的评分价值在于考你是否把“观察 UI 状态”理解成 View 的资源，而不是 Fragment 的资源。

```

1 class StreakDetailFragment : Fragment(R.layout.fragment_streak_detail) {
2     private var _binding: FragmentStreakDetailBinding? = null
3     private val binding get() = requireNotNull(_binding)
4     private val viewModel: StreakViewModel by viewModels()
5
6     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
7         super.onViewCreated(view, savedInstanceState)
8         _binding = FragmentStreakDetailBinding.bind(view)
9
10        viewModel.streak.observe(viewLifecycleOwner) { streak ->
11            binding.streakCard.setDayCount(streak.days)
12            binding.streakCard.setFrozen(streak.isFrozen)
13            if (streak.extendedToday) {
14                binding.streakCard.playCelebration()
15            }
16        }
17    }

```

```
18     viewModel.loading.observe(viewLifecycleOwner) { loading ->
19         binding.progressBar.isVisible = loading
20         binding.practiceButton.isEnabled = !loading
21     }
22
23     binding.practiceButton.setOnClickListener {
24         viewModel.practiceToday()
25     }
26 }
27
28 override fun onDestroyView() {
29     binding.practiceButton.setOnClickListener(null)
30     _binding = null
31     super.onDestroyView()
32 }
33 }
```

【陷阱】单次路径测不出来

这个 bug 在本地最容易漏，因为“打开一次、数据回来、UI 更新”全是对的。必须测试“打开 A、返回、打开 B、触发同一个 LiveData 更新”，才会看到 observer 累积。查错题里凡是 Fragment 注册观察者、callback、adapter listener，都要主动问一句：它跟 View 还是 Fragment 同寿命？

【设计思想】UI 订阅属于 View

只要回调里直接读写 View，它就应该绑定到 `viewLifecycleOwner`。这条规则比“LiveData 用哪个 owner”更广：Flow 的收集、menu provider、back callback、文本监听器，本质上都要问同一个问题：谁拥有这个 UI 副作用，谁负责释放它？

【追问】“如果改用 StateFlow 呢？”

StateFlow 不会自动解决生命周期问题。Fragment 里应该在 `viewLifecycleOwner.lifecycleScope` 中配合 `repeatOnLifecycle(Lifecycle.State.STARTED)` 收集，让 View 停止时取消收集、重启时重新收集。协程版细节在前一章已经展开；这里的对照点是：API 变了，所有权规则没变。

6.3 ViewModel 持有 Activity Context

查错 #9 | 类别：泄漏与可测试性

场景

个人主页展示本周 XP、连续学习天数和下一枚成就徽章。ProfileViewModel 为了拼出“本周获得 120 XP”这类文案，构造函数里直接拿了 Activity 的 Context。这在功能上很顺手，却把 ViewModel 的长生命周期和 Activity 的短生命周期绑到了一起。

```
1 class ProfileViewModel(  
2     private val context: Context,  
3     private val userRepository: UserRepository  
4 ) : ViewModel() {  
5     private val _profile = MutableLiveData<ProfileUiModel>()  
6     val profile: LiveData<ProfileUiModel> = _profile  
7  
8     fun loadProfile(userId: String) {  
9         viewModelScope.launch {  
10             val user = userRepository.getUser(userId)  
11             val weeklyXp = userRepository.getWeeklyXp(userId)  
12             val nextBadge = userRepository.getNextBadge(userId)  
13  
14             _profile.value = ProfileUiModel(  
15                 name = user.displayName,  
16                 avatarUrl = user.avatarUrl,  
17                 weeklyXpText = context.getString(R.string.weekly_xp_format,  
18                     weeklyXp),  
19                 streakText = context.getString(R.string.streak_days_format, user  
20                     .streakDays),  
21                 badgeText = context.getString(R.string.next_badge_format,  
22                     nextBadge.title)  
23             )  
24         }  
25     }  
26  
27     override fun onCleared() {  
28         super.onCleared()  
29         userRepository.cancelUserRequests()  
30     }  
31 }
```

【话术】面试里怎么说

我会先说影响：ViewModel 会跨旋屏存活，如果这里传进来的是 Activity context，旋屏后旧 Activity 不能被回收。Then: “This is not only a leak; it is a layering smell. The ViewModel is formatting UI copy instead of exposing state. I would either use Application context for pure resources, or preferably move string resolution to the UI layer.”

根因分析

ViewModel 的设计目标就是跨配置变更保留状态。Activity 旋转后旧实例应被销毁，新实例重新观察同一个 ViewModel。如果 ViewModel 持有旧 Activity context，旧 Activity 连同它的 Window、View、资源引用都可能被保留下来。这个问题不像空指针那样立刻崩；它常在多次旋屏、分屏、语言切换或低内存回收后放大。

更深一层，它暴露的是分层错误：ViewModel 为了 getString 了解了“文案长什么样”。一旦要做复数、A/B 文案、可访问性描述或 Compose stringResource，测试就变得笨重。评审时可以给三条路：一是 AndroidViewModel 拿 Application context，只适合非常纯的资源访问；二是 ViewModel 暴露资源 ID 与参数，让 UI 解析；三是注入 StringProvider 抽象。面试中我会偏向第二条，因为它让 ViewModel 表达“状态是什么”，让 UI 决定“状态如何说给用户听”。

```
1  data class ProfileUiState(  
2      val name: String,  
3      val avatarUrl: String?,  
4      val weeklyXp: Int,  
5      val streakDays: Int,  
6      val nextBadgeTitle: String  
7  )  
8  
9  class ProfileViewModel(  
10     private val userRepository: UserRepository  
11 ) : ViewModel() {  
12     private val _profile = MutableLiveData<ProfileUiState>()  
13     val profile: LiveData<ProfileUiState> = _profile  
14  
15     fun loadProfile(userId: String) {  
16         viewModelScope.launch {  
17             val user = userRepository.getUser(userId)  
18             val weeklyXp = userRepository.getWeeklyXp(userId)  
19             val nextBadge = userRepository.getNextBadge(userId)  
20  
21             _profile.value = ProfileUiState(  
22                 name = user.displayName,  
23                 avatarUrl = user.avatarUrl,  
24                 weeklyXp = weeklyXp,  
25                 streakDays = user.streakDays,  
26                 nextBadgeTitle = nextBadge.title  
27             )  
28         }  
29     }  
30 }  
31  
32 class ProfileFragment : Fragment(R.layout.fragment_profile) {  
33     private val viewModel: ProfileViewModel by viewModels()
```

```
34
35     private fun render(state: ProfileUiState, binding: FragmentProfileBinding) {
36         binding.name.text = state.name
37         binding.weeklyXp.text = getString(R.string.weekly_xp_format, state.
weeklyXp)
38         binding.streak.text = getString(R.string.streak_days_format, state.
streakDays)
39         binding.nextBadge.text = getString(R.string.next_badge_format, state.
nextBadgeTitle)
40     }
41 }
```

【设计思想】泄漏常常是分层问题的症状

如果一个长生命周期对象持有短生命周期对象，先别急着问“换成 Application 行不行”，要追问“它为什么需要这个对象”。很多泄漏修到最后不是加 weak reference，而是重新划分责任：ViewModel 管状态，UI 管资源与渲染，Repository 管数据。

【陷阱】Application context 不是万能止痛药

把 Activity context 换成 Application context 可以消除这条泄漏链，但不一定消除设计味道。Application context 不能弹主题化 dialog，不能拿 Activity 级样式，也让 ViewModel 更难做纯 JVM 单元测试。只有当资源解析确实是平台无关的基础能力时，才考虑包一层 StringProvider。

【追问】“那 ViewModel 暴露资源 ID 会不会让 UI 变笨？”

可以用 sealed class 或小型 copy model 把 UI 文案表达成结构化值，例如 TextSpec.Resource(id, args) 与 TextSpec.Raw(value)。这样 ViewModel 仍不持有 Context，UI 只做一次统一解析，测试也能断言资源 ID 和参数。

6.4 ViewHolder 在 bind 里起协程却不在回收时取消

查错 #10 | 类别：列表与协程

场景

排行榜每行展示头像、昵称、本周 XP 和升降名次。为了让列表先出现文字，再异步补头像与 XP，LeaderboardViewHolder.bind 里直接起协程加载。快速滚动时，RecyclerView 会复用 ViewHolder；如果旧请求回来后写进新位置，用户就会看到头像错位、XP 串行，像列表里出现了“鬼影”。

```
1 class LeaderboardAdapter(
2     private val repository: LeaderboardRepository,
3     private val scope: CoroutineScope
```

```
4 ) : RecyclerView.Adapter<LeaderboardViewHolder>() {
5     private val players = mutableListOf<LeaderboardRow>()
6
7     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
LeaderboardViewHolder {
8         val binding = RowLeaderboardBinding.inflate(LayoutInflater.from(parent.
context), parent, false)
9         return LeaderboardViewHolder(binding, repository, scope)
10    }
11
12    override fun onBindViewHolder(holder: LeaderboardViewHolder, position: Int)
{
13        holder.bind(players[position])
14    }
15
16    override fun getItemCount(): Int = players.size
17 }
18
19 class LeaderboardViewHolder(
20     private val binding: RowLeaderboardBinding,
21     private val repository: LeaderboardRepository,
22     private val scope: CoroutineScope
23 ) : RecyclerView.ViewHolder(binding.root) {
24
25     fun bind(row: LeaderboardRow) {
26         binding.rank.text = row.rank.toString()
27         binding.name.text = row.displayName
28         binding.avatar.setImageResource(R.drawable.avatar_placeholder)
29         binding.weeklyXp.text = binding.root.context.getString(R.string.
loading_xp)
30
31         scope.launch {
32             val avatar = repository.loadAvatar(row.userId)
33             val xp = repository.loadWeeklyXp(row.userId)
34             binding.avatar.setImageBitmap(avatar)
35             binding.weeklyXp.text = binding.root.context.getString(R.string.
xp_amount, xp)
36         }
37     }
38 }
```

【话术】面试里怎么说

我会先说用户现象：快速滚动后，旧用户的头像或 XP 可能写到新用户行上。Then: “A ViewHolder is reusable, but the coroutine launched from bind outlives that binding operation. We need to cancel work when the holder is rebound or recycled, or move the async ownership out of the item.”

根因分析

RecyclerView 的核心优化就是复用 ViewHolder：第 3 行滚出屏幕后，同一个 holder 可能立刻拿去显示第 30 行。bind 不是构造函数，它会被调用很多次。上面的协程捕获了 binding 和旧的 row.userId，但没有任何机制说明“这次绑定已经过期”。当网络慢、磁盘缓存 miss、用户快速滑动时，旧协程晚回来，就把旧头像写到当前 holder 上。

这个 bug 的可怕之处在于它不一定崩溃，而是造成可见但间歇性的错行：排行榜头像错、XP 错、名次动效错。用户很难描述，日志也未必有异常。评审时要指出两个层次：最低限度是在 holder 里保存 Job，重新 bind 和回收时取消；更好的架构是头像加载交给 Coil 这类知道 ImageView 生命周期的库，XP 数据在 ViewModel 或 Repository 层合并好，Adapter 只负责渲染不可变行模型。

```
1 class LeaderboardAdapter(  
2     private val repository: LeaderboardRepository,  
3     private val scope: CoroutineScope  
4 ) : RecyclerView.Adapter<LeaderboardViewHolder>() {  
5     private val players = mutableListOf<LeaderboardRow>()  
6  
7     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):  
8         LeaderboardViewHolder {  
9         val binding = RowLeaderboardBinding.inflate(LayoutInflater.from(parent.  
10             context), parent, false)  
11         return LeaderboardViewHolder(binding, repository, scope)  
12     }  
13  
14     override fun onBindViewHolder(holder: LeaderboardViewHolder, position: Int)  
15     {  
16         holder.bind(players[position])  
17     }  
18  
19     override fun onViewRecycled(holder: LeaderboardViewHolder) {  
20         holder.clear()  
21         super.onViewRecycled(holder)  
22     }  
23  
24     override fun getItemCount(): Int = players.size  
25 }  
26  
27 class LeaderboardViewHolder(  
28     private val binding: RowLeaderboardBinding,  
29     private val repository: LeaderboardRepository,  
30     private val scope: CoroutineScope
```

```
25     private val binding: RowLeaderboardBinding,
26     private val repository: LeaderboardRepository,
27     private val scope: CoroutineScope
28 ) : RecyclerView.ViewHolder(binding.root) {
29     private var bindJob: Job? = null
30     private var boundUserId: String? = null
31
32     fun bind(row: LeaderboardRow) {
33         bindJob?.cancel()
34         boundUserId = row.userId
35         binding.rank.text = row.rank.toString()
36         binding.name.text = row.displayName
37         binding.avatar.setImageResource(R.drawable.avatar_placeholder)
38         binding.weeklyXp.text = binding.root.context.getString(R.string.
loading_xp)
39
40         bindJob = scope.launch {
41             val avatar = repository.loadAvatar(row.userId)
42             val xp = repository.loadWeeklyXp(row.userId)
43             if (boundUserId == row.userId && bindingAdapterPosition !=
RecyclerView.NO_POSITION) {
44                 binding.avatar.setImageBitmap(avatar)
45                 binding.weeklyXp.text = binding.root.context.getString(R.string.
xp_amount, xp)
46             }
47         }
48     }
49
50     fun clear() {
51         bindJob?.cancel()
52         bindJob = null
53         boundUserId = null
54         binding.avatar.setImageDrawable(null)
55     }
56 }
```

【设计思想】列表项不该拥有异步任务的所有权

ViewHolder 是渲染缓存，不是业务任务 owner。它的生命周期由 RecyclerView 为性能而频繁复用，天然不适合持有长任务。能在 ViewModel 里把行模型准备好，就不要在 item 里发请求；能交给图片库管理请求取消，就不要手写头像协程。

【陷阱】取消不等于校验可以省略

即使保存 Job 并取消, 也建议在写回前比对 boundUserId 或请求 tag。取消是协作式的, 某些库调用不可取消, 或者在取消发生前已经拿到结果。写 UI 前再确认“我还是为同一个 userId 服务”, 是防止错行的最后一道闸。

【追问】“如果必须在 item 里发请求呢?”

把请求和绑定令牌绑定起来: bind 时记录 userId 或递增 token, 返回时比对; 回收时取消 Job; 写回前检查 bindingAdapterPosition != NO_POSITION。如果是图片, 优先用 Coil 的 ImageView.load(url), 让库在新请求覆盖旧请求时自动取消。

6.5 每次数据变化都 notifyDataSetChanged, 且 stable ID 不稳定

查错 #11 | 类别: 列表性能

场景

排行榜是 Duolingo 产品里极适合查错的列表: 实时刷新、排名变化、用户头像、徽章、滚动位置、动画全都在同一屏。社区里明确有人报告过类似真题: “Review this code that handles list updates in a RecyclerView —any performance or crash risks you’d improve?” [\[面经报告\]](#)。下面这段代码正是那类题的核心: 看似只是刷新列表, 实则同时破坏性能、动画和身份语义。

```
1 class LeaderboardAdapter : RecyclerView.Adapter<LeaderboardAdapter.RowViewHolder>() {
2     private val rows = mutableListOf<LeaderboardRow>()
3
4     init {
5         setHasStableIds(true)
6     }
7
8     fun updateRows(newRows: List<LeaderboardRow>) {
9         rows.clear()
10        rows.addAll(newRows)
11        notifyDataSetChanged()
12    }
13
14    override fun getItemId(position: Int): Long {
15        return position.toLong()
16    }
17
18    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
19    RowViewHolder {
20        val binding = RowLeaderboardBinding.inflate(LayoutInflater.from(parent.
21        context), parent, false)
```

```
20         return ViewHolder(binding)
21     }
22
23     override fun onBindViewHolder(holder: ViewHolder, position: Int) {
24         holder.bind(rows[position])
25     }
26
27     override fun getItemCount(): Int = rows.size
28
29     class ViewHolder(private val binding: RowLeaderboardBinding) :
30     RecyclerView.ViewHolder(binding.root) {
31         fun bind(row: LeaderboardRow) {
32             binding.rank.text = row.rank.toString()
33             binding.name.text = row.displayName
34             binding.weeklyXp.text = binding.root.context.getString(R.string.
35             xp_amount, row.weeklyXp)
36             binding.badge.isVisible = row.isInPromotionZone
37         }
38     }
39 }
```

【话术】面试里怎么说

我会先说影响：实时榜单每次更新都全量重绑，会造成闪烁、掉帧和滚动位置不稳定；更严重的是 stable ID 返回 position，排名一变，同一个人就变成了另一个 ID。Then: “This code confuses position with identity. I would use ListAdapter and DiffUtil, with areItemsTheSame based on a stable business id and areContentsTheSame based on value equality.”

根因分析

`notifyDataSetChanged()` 等于告诉 RecyclerView：“我不知道哪里变了，你全重来吧。”结果是所有可见行重新绑定，默认动画失去增量信息，图片可能重新加载，TalkBack 的焦点也更容易跳。排行榜每秒或每几秒更新时，这会直接变成慢帧和闪烁。Duolingo 的 Android Reboot 复盘里，慢帧和滚动性能曾是官方点名的用户体验问题 [官方]²，所以在它的 Android 查错轮里，列表刷新绝不是小题。

`setHasStableIds(true)` 本来是告诉 RecyclerView：每个 item 有跨位置、跨刷新都稳定的身份。但 `getItemId` 返回 position，恰好违背这个承诺。排行榜里用户从第 10 名升到第 3 名，业务身份没变，position 变了；另一个用户补到第 10 名，position 一样但身份变了。RecyclerView 会基于错误身份复用 holder 或运行动画，于是出现动画错乱、内容闪烁、滚动锚点漂移。

DiffUtil 的两个方法也常被写反。`areItemsTheSame` 问“是不是同一个实体”，应该看 `userId`、`lessonId`、`orderId` 这类稳定业务 ID；`areContentsTheSame` 问“同一个实体显示内容有没有变”，可以用 data class 的 `equals`。如果把内容相等当身份，昵称或 XP 一变就被当成新用户；如果把

²<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

userId 当内容, XP 变化又不会刷新。

```
1  data class LeaderboardRow(  
2      val userId: String,  
3      val rank: Int,  
4      val displayName: String,  
5      val weeklyXp: Int,  
6      val isInPromotionZone: Boolean  
7  )  
8  
9  class LeaderboardAdapter : ListAdapter<LeaderboardRow, LeaderboardAdapter.  
    ViewHolder>(DIFF) {  
10  
11      override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):  
    ViewHolder {  
12          val binding = RowLeaderboardBinding.inflate(LayoutInflater.from(parent.  
    context), parent, false)  
13          return ViewHolder(binding)  
14      }  
15  
16      override fun onBindViewHolder(holder: ViewHolder, position: Int) {  
17          holder.bind(getItem(position))  
18      }  
19  
20      override fun getItemId(position: Int): Long {  
21          return getItem(position).userId.hashCode().toLong()  
22      }  
23  
24      class ViewHolder(private val binding: RowLeaderboardBinding) :  
    RecyclerView.ViewHolder(binding.root) {  
25          fun bind(row: LeaderboardRow) {  
26              binding.rank.text = row.rank.toString()  
27              binding.name.text = row.displayName  
28              binding.weeklyXp.text = binding.root.context.getString(R.string.  
    xp_amount, row.weeklyXp)  
29              binding.badge.isVisible = row.isInPromotionZone  
30          }  
31      }  
32  
33      companion object {  
34          private val DIFF = object : DiffUtil.ItemCallback<LeaderboardRow>() {  
35              override fun areItemsTheSame(oldItem: LeaderboardRow, newItem:  
    LeaderboardRow): Boolean {  
36                  return oldItem.userId == newItem.userId  
37              }  
38          }  
39      }
```

```

39         override fun areContentsTheSame(oldItem: LeaderboardRow, newItem:
LeaderboardRow): Boolean {
40             return oldItem == newItem
41         }
42     }
43 }
44 }
45
46 fun LeaderboardAdapter.render(rows: List<LeaderboardRow>) {
47     submitList(rows)
48 }

```

【陷阱】“data class equals” 也可能让 Diff 退化

如果 `LeaderboardRow` 里塞了可变引用、每次刷新都更新的 `fetchAt` 时间戳，或一个不参与 UI 的 `debug` 字段，`areContentsTheSame` 会永远 `false`，`DiffUtil` 表面存在，效果却接近全量刷新。行模型应尽量不可变，只包含真正影响渲染的字段；非 UI 元数据不要混进 `item equality`。

【设计思想】身份、内容、位置三件事要分开

列表 bug 的总开关，是把 `position` 当 `identity`。`position` 只是当前排序下的坐标，内容是用户看到的字段，身份是业务实体本身。查错时先问：这个 `item` 的稳定业务 ID 是什么？如果答不出来，后面的动画、缓存、局部刷新都会摇晃。

【追问】“如果排行榜是每秒都在变的实时数据呢？”

第一，节流或合并上游更新，例如 500ms 或一屏刷新一次，不要让 UI 承担服务端推送频率。第二，只刷新变化字段，可用 `payload` 让 XP 文本局部更新，而不是整行重绑头像。第三，用户正在快速滚动时可以延后非关键刷新，等 `idle` 再提交；这是产品取舍，不是纯技术题。`DiffUtil` 默认由 `AsyncListDiffer` 在后台算 `diff`，但你仍要控制提交频率。

6.6 注册了监听器却没有解注册

查错 #12 | 类别：资源泄漏

场景

单词复习模块有离线队列：用户在地铁里完成练习，网络恢复时自动同步。`WordReviewActivity` 注册了 `ConnectivityManager.NetworkCallback`，一旦可用网络出现就触发 `sync`。代码能工作，但注册和释放不配对，长会话后同一个同步动作可能被触发多次，`Activity` 也可能被系统 `callback` 持有。

```

1 class WordReviewActivity : AppCompatActivity() {

```

```
2     private lateinit var binding: ActivityWordReviewBinding
3     private val syncRepository: WordReviewRepository by lazy {
4         WordReviewRepository.getInstance(this) }
5     private lateinit var connectivityManager: ConnectivityManager
6
7     private val networkCallback = object : ConnectivityManager.NetworkCallback()
8     {
9         override fun onAvailable(network: Network) {
10             lifecycleScope.launch {
11                 syncRepository.syncOfflineQueue()
12                 binding.syncStatus.text = getString(R.string.synced)
13             }
14         }
15     }
16
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         binding = ActivityWordReviewBinding.inflate(layoutInflater)
20         setContentView(binding.root)
21         connectivityManager = getSystemService(ConnectivityManager::class.java)
22         connectivityManager.registerDefaultNetworkCallback(networkCallback)
23     }
24
25     override fun onResume() {
26         super.onResume()
27         binding.reviewCard.startListening()
28     }
29
30     override fun onDestroy() {
31         binding.reviewCard.stopListening()
32         super.onDestroy()
33     }
34 }
```

【话术】面试里怎么说

我会先说影响：系统服务持有 callback，Activity 退出后仍可能被引用；如果注册路径重复，还会导致一次网络恢复触发多次 sync。Then: “Registration and unregistration must be paired at the same lifecycle level. I would register in onStart and unregister in onStop, or wrap it in a lifecycle observer so acquire and release stay together.”

根因分析

系统服务、广播、传感器、全局布局监听器都有共同点：注册后，外部对象会保存你的 callback。这个 callback 是匿名内部类，隐式持有 WordReviewActivity，又访问 binding 和 repository。只

要不解注册, Activity 就可能在销毁后仍被系统服务引用。更糟的是, 如果代码后来搬到 `onResume` 注册、仍在 `onDestroy` 解注册, 暂停再恢复多次就会重复注册, 网络恢复时重复同步, 服务端看到重复请求, 用户看到状态文案跳动。

生命周期配对的关键是“在哪个回调获取, 就在哪个对称回调释放”。可见期间需要网络监听, 就用 `onStart/onStop`; 前台交互期间需要传感器, 就用 `onResume/onPause`; 整个进程级能力, 则不该挂在 Activity 上, 而应挂在 Application 或专门 manager 上。

```
1 class WordReviewActivity : AppCompatActivity() {
2     private lateinit var binding: ActivityWordReviewBinding
3     private val syncRepository: WordReviewRepository by lazy {
4         WordReviewRepository.getInstance(applicationContext) }
5     private lateinit var connectivityManager: ConnectivityManager
6     private var networkCallbackRegistered = false
7
8     private val networkCallback = object : ConnectivityManager.NetworkCallback()
9     {
10         override fun onAvailable(network: Network) {
11             lifecycleScope.launchWhenStarted {
12                 syncRepository.syncOfflineQueue()
13                 binding.syncStatus.text = getString(R.string.synced)
14             }
15         }
16     }
17
18     override fun onCreate(savedInstanceState: Bundle?) {
19         super.onCreate(savedInstanceState)
20         binding = ActivityWordReviewBinding.inflate(layoutInflater)
21         setContentView(binding.root)
22         connectivityManager = getSystemService(ConnectivityManager::class.java)
23     }
24
25     override fun onStart() {
26         super.onStart()
27         if (!networkCallbackRegistered) {
28             connectivityManager.registerDefaultNetworkCallback(networkCallback)
29             networkCallbackRegistered = true
30         }
31         binding.reviewCard.startListening()
32     }
33
34     override fun onStop() {
35         if (networkCallbackRegistered) {
36             connectivityManager.unregisterNetworkCallback(networkCallback)
37             networkCallbackRegistered = false
38         }
39     }
40 }
```

```
37         binding.reviewCard.stopListening()  
38         super.onStop()  
39     }  
40 }
```

【设计思想】获取与释放要在语法上相邻

资源类 bug 的通用解法，是让 acquire 和 release 靠近：onStart/onStop 成对出现，DefaultLifecycleObserver 把注册与解注册封在一个类里，Compose 用 DisposableEffect 在同一个代码块里返回 onDispose。当释放逻辑离获取逻辑很远，评审时就该提高警惕。

【陷阱】生命周期不配对比完全不释放更隐蔽

onResume 注册、onDestroy 解注册，看起来“最终会释放”，但中间每次暂停恢复都会多注册一份。查错题里不要只搜索有没有 unregister，还要检查 register 与 unregister 是否在同一层级配对。

【追问】“用 DefaultLifecycleObserver 怎么写？”

可以把网络监听封成 OfflineSyncObserver，在 onStart(owner) 注册，在 onStop(owner) 解注册，然后由 Activity lifecycle.addObserver(observer)。这样 Activity 不再散落资源管理细节，测试也能单独验证 observer 的配对行为。

【要点】生命周期所有权口诀

谁创建，谁释放；谁读取 View，谁跟 View 同寿命；谁注册外部 callback，谁在对称生命周期解注册；谁启动异步任务，谁负责取消。查错时不要被 API 名字带着走，先画出对象之间的强引用和生命周期长短，答案通常就浮出来了。

Chapter 7

查错题集 C：Compose、空安全与 Kotlin 惯用性

这一章看的是两类容易被低估的问题。第一类是 Compose 副作用与重组：它们不一定崩溃，却会带来重复请求、旧数据、掉帧和难以复现的 UI 抖动。第二类是空安全与 Kotlin 惯用性：Duolingo 官方写过，Android 端花了两年迁移到 100% Kotlin，并配套 detekt、ktlint、Android Lint 与自研 Splinter；历史上 NPE crash 也是迁移动机之一 [官方]¹。因此在它的 Android 专属 Kotlin Code Review 里，代码“像不像 Kotlin”不是审美题，而是可靠性信号。Duolingo 也公开提到在 Shop、Purchase Flow 等 SDUI 场景使用 Compose [官方]²，所以 Compose 查错要按生产代码标准看。

7.1 把 MutableStateFlow 暴露出去，并用 SharedFlow 存持久状态

查错 #13 | 类别：状态暴露

场景

商店页 ShopViewModel 展示宝石余额、道具卡、订阅入口和购买结果 toast。页面从 SDUI 配置拿卡片，用户旋屏或从支付页返回时应立即看到当前状态。下面代码把可写 StateFlow 直接公开，又把持久 UI 状态和一次性导航事件混在 Flow 里，两个方向都会出错。

```
1 class ShopViewModel(  
2     private val shopRepository: ShopRepository,  
3     private val purchaseTracker: PurchaseTracker  
4 ) : ViewModel() {  
5     val uiState = MutableStateFlow(ShopState())  
6     val purchaseEvents = MutableStateFlow<PurchaseEvent?>(null)  
7     val cards = MutableSharedFlow<List<ShopCard>>()  
8  
9     fun loadShop() {  
10         viewModelScope.launch {
```

¹<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

²<https://blog.duolingo.com/server-driven-ui/>

```
11         uiState.value = uiState.value.copy(loading = true)
12         val response = shopRepository.loadShopCards()
13         cards.emit(response.cards)
14         uiState.value = ShopState(
15             loading = false,
16             gemBalance = response.gemBalance,
17             subscriptionVisible = response.showSubscription
18         )
19     }
20 }
21
22 fun buy(card: ShopCard) {
23     viewModelScope.launch {
24         val receipt = shopRepository.purchase(card.id)
25         purchaseTracker.trackSuccess(card.id)
26         purchaseEvents.value = PurchaseEvent.NavigateToReceipt(receipt.id)
27         uiState.value = uiState.value.copy(gemBalance = receipt.
newGemBalance)
28     }
29 }
30 }
```

【话术】面试里怎么说

我会先说影响：外部 UI 可以直接改 `uiState`，破坏 ViewModel 单向数据流；`SharedFlow` 没有 replay 时旋屏后新订阅者拿不到 cards，可能白屏；用 `StateFlow` 存导航事件又会在重新订阅时重复导航。Then: “State is replayable; events are not. I would expose read-only `StateFlow` and send one-off effects through a Channel or a `SharedFlow` with replay zero.”

根因分析

`MutableStateFlow` 是可写权限。把它作为 `public val` 暴露出去，Fragment、Composable、测试替身甚至另一个 ViewModel 都能随手改商店状态。状态来源一旦不唯一，评审时就很难推理“宝石余额为什么变了”。这违反了单向数据流，也让并发更新更容易互相覆盖。

第二个问题是状态和事件的语义混乱。状态的判据是：新观察者加入时，重放当前值是正确的。商店卡片、余额、loading 都是状态；旋屏后应该立刻拿到。事件的判据是：重放会产生副作用。导航到收据页、弹 toast、震动反馈都不该因为旋屏重新发生。`SharedFlow` 默认不保留值，用它存 cards 会让新 UI 看不到当前商店；`StateFlow` 永远有最新值，用它存导航事件会让返回后再次跳转。

```
1 data class ShopState(
2     val loading: Boolean = false,
3     val gemBalance: Int = 0,
4     val subscriptionVisible: Boolean = false,
```

```

5     val cards: List<ShopCard> = emptyList()
6 )
7
8 sealed interface ShopEvent {
9     data class NavigateToReceipt(val receiptId: String) : ShopEvent
10    data class ShowPurchaseError(val message: String) : ShopEvent
11 }
12
13 class ShopViewModel(
14     private val shopRepository: ShopRepository,
15     private val purchaseTracker: PurchaseTracker
16 ) : ViewModel() {
17     private val _uiState = MutableStateFlow(ShopState())
18     val uiState: StateFlow<ShopState> = _uiState.asStateFlow()
19
20     private val _events = Channel<ShopEvent>(Channel.BUFFERED)
21     val events: Flow<ShopEvent> = _events.receiveAsFlow()
22
23     fun loadShop() {
24         viewModelScope.launch {
25             _uiState.update { it.copy(loading = true) }
26             val response = shopRepository.loadShopCards()
27             _uiState.value = ShopState(
28                 loading = false,
29                 gemBalance = response.gemBalance,
30                 subscriptionVisible = response.showSubscription,
31                 cards = response.cards
32             )
33         }
34     }
35
36     fun buy(card: ShopCard) {
37         viewModelScope.launch {
38             runCatching { shopRepository.purchase(card.id) }
39                 .onSuccess { receipt ->
40                     purchaseTracker.trackSuccess(card.id)
41                     _uiState.update { it.copy(gemBalance = receipt.newGemBalance
42 ) }
43                     _events.send(ShopEvent.NavigateToReceipt(receipt.id))
44                 }
45                 .onFailure { _events.send(ShopEvent.ShowPurchaseError("Purchase
46 failed")) }
47         }
48     }
49 }

```


【设计思想】可写性是一种权限

默认只暴露只读接口：StateFlow 而不是 MutableStateFlow，List 而不是 MutableList，领域服务方法而不是裸字段。权限越小，状态变化路径越少，Code Review 更容易判断正确性。

【陷阱】状态与事件的判据

不要按“它来自 ViewModel”来分类，要按“能不能安全重放”分类。余额可以重放，导航不可以；错误 banner 如果屏幕重建后仍应显示，是状态；toast 如果只说一次，是事件。这个判据比具体选 Channel 还是 SharedFlow 更重要。

【追问】“为什么不用 LiveData ?”

LiveData 也能承载状态，但 StateFlow 更贴近协程、组合操作和 Kotlin 只读包装。关键不在名字，而在是否维持单向数据流、是否有清晰的 replay 语义。团队如果仍用 LiveData，同样应暴露 LiveData 而不是 MutableLiveData。

【追问】“SingleLiveEvent 为什么是错的方向 ?”

SingleLiveEvent 试图在状态容器上补一个“只消费一次”的补丁，常遇到多观察者、旋屏竞态和测试不确定性。更清晰的方向是类型上分流：持久状态走 StateFlow，一次性 effect 走 Channel 或 replay 为 0 的 SharedFlow。

7.2 LaunchedEffect 的 key 用错，副作用写在 composable 函数体里

查错 #14 | 类别：Compose 副作用

场景

课程详情页迁到 Compose 后，需要进入页面时上报曝光，并在 lessonId 变化时重新拉取内容。产品允许用户在同一个容器里切换不同课程；如果 effect key 写错，就会出现两种相反问题：重组时重复请求，或切课后还显示旧课程。

```
1 @Composable
2 fun LessonDetailScreen(
3     lessonId: String,
4     viewModel: LessonDetailViewModel,
5     onStartLesson: (String) -> Unit
6 ) {
7     val state by viewModel.state.collectAsState()
8
9     viewModel.loadLesson(lessonId)
10    viewModel.trackLessonImpression(lessonId)
11
12    LaunchedEffect(Unit) {
```

```

13     viewModel.refreshRecommendations(lessonId)
14 }
15
16 Column(modifier = Modifier.fillMaxSize()) {
17     LessonHeader(
18         title = state.title,
19         subtitle = state.subtitle,
20         locked = state.locked
21     )
22     RecommendationCarousel(items = state.recommendations)
23     Button(
24         enabled = !state.locked,
25         onClick = { onStartLesson(lessonId) }
26     ) {
27         Text(text = stringResource(R.string.start_lesson))
28     }
29 }
30 }

```

【话术】面试里怎么说

我会先说影响：函数体里的 `load` 和曝光上报会在每次重组时重复执行；而 `LaunchedEffect(Unit)` 只跑一次，`lessonId` 变化时推荐内容不会刷新。Then: “Composable functions should be idempotent. Side effects must be placed in effect APIs with keys that match the lifecycle of the work.”

根因分析

Compose 可以因为很多原因重组：state 变化、父组件重组、配置变化、列表 item 移动。Composable 函数体应该像“描述 UI 的纯函数”，同样输入多次执行不应产生额外副作用。直接在函数体调用 `loadLesson` 和 `trackLessonImpression`，等于把网络请求与埋点绑定到重组次数；一个 loading 状态变化就可能触发下一次 load，形成隐蔽循环。

`LaunchedEffect(Unit)` 是另一种错：它确实避免了每次重组都跑，但 key 是 `Unit`，表示“这个调用点进入 composition 后只启动一次”。当 `lessonId` 从西语课程变成法语课程，同一个 Composable 仍在，effect 不会重启，用户看到旧推荐。正确 key 应该表达副作用依赖的身份：加载课程依赖 `lessonId`，就用 `LaunchedEffect(lessonId)`。

几个 effect 的选择可以这样讲：`LaunchedEffect` 启动可挂起任务，key 变则取消旧任务并重启；`SideEffect` 在成功重组后把 Compose 状态同步到非 Compose 对象，不能挂起；`DisposableEffect` 管注册与解注册，key 变或离开 composition 时释放；`rememberUpdatedState` 用来在长生命周期 effect 中拿到最新 lambda，而不把 lambda 作为重启 key。

```

1 @Composable
2 fun LessonDetailScreen(
3     lessonId: String,

```

```

4     viewModel: LessonDetailViewModel,
5     onStartLesson: (String) -> Unit
6 ) {
7     val state by viewModel.state.collectAsStateWithLifecycle()
8     val latestOnStartLesson by rememberUpdatedState(onStartLesson)
9
10    LaunchedEffect(lessonId) {
11        viewModel.loadLesson(lessonId)
12        viewModel.refreshRecommendations(lessonId)
13        viewModel.trackLessonImpression(lessonId)
14    }
15
16    Column(modifier = Modifier.fillMaxSize()) {
17        LessonHeader(
18            title = state.title,
19            subtitle = state.subtitle,
20            locked = state.locked
21        )
22        RecommendationCarousel(items = state.recommendations)
23        Button(
24            enabled = !state.locked,
25            onClick = { latestOnStartLesson(lessonId) }
26        ) {
27            Text(text = stringResource(R.string.start_lesson))
28        }
29    }
30 }

```

【陷阱】lambda 不一定适合当 key

如果把 `onStartLesson` 或 `onImpression` 这类 lambda 直接放进 key，而父组件每次重组都创建新 lambda，effect 就会被反复取消重启。需要“effect 不重启，但内部使用最新 lambda”时，用 `rememberUpdatedState` 保存最新引用。

【设计思想】key 是副作用的生命周期声明

写 `LaunchedEffect(x)` 不是为了让 lint 安静，而是在声明：这项工作属于 x；x 变了，旧工作就过期。查错时把每个 effect 翻译成这句话，很多 key 错误会立刻显形。

【追问】“如果曝光只想在整个屏幕生命周期内发一次呢？”

那就把曝光和加载拆开：加载用 `LaunchedEffect(lessonId)`，屏幕级曝光用 `LaunchedEffect(Unit)`，或者让导航层在 screen entry 时上报。一个 effect 只承载一种生命周期语义，不要把“随 lesson 变化”和“整个屏幕一次”揉在同一个块里。

7.3 collectAsState 而非 collectAsStateWithLifecycle，以及不稳定参数引发的过度重组

查错 #15 | 类别：重组性能

场景

主界面学习路径 PathScreen 是长列表：顶部有连胜、宝石计数，下面是几十个 lesson node。后台同步 XP、宝石、课程解锁时，上游 Flow 频繁变化。Compose 代码如果不按生命周期收集、不给 LazyColumn key、把不稳定列表和新建 lambda 一路传下去，就会造成后台仍工作、前台整屏重组和滚动位置跳动。

```
1 @Composable
2 fun PathScreen(
3     viewModel: PathViewModel,
4     navigator: LessonNavigator
5 ) {
6     val state by viewModel.uiState.collectAsState()
7     val lessons = state.lessons.toMutableList()
8
9     Column(modifier = Modifier.fillMaxSize()) {
10         TopStatsBar(
11             streak = state.streakDays,
12             gems = state.gemBalance,
13             onShopClick = { navigator.openShop() }
14         )
15
16         LazyColumn(modifier = Modifier.fillMaxSize()) {
17             items(lessons) { lesson ->
18                 LessonNode(
19                     lesson = lesson,
20                     isCurrent = lesson.id == state.currentLessonId,
21                     onClick = { navigator.openLesson(lesson.id) }
22                 )
23             }
24         }
25     }
26 }
27
28 @Composable
29 fun LessonNode(lesson: PathLesson, isCurrent: Boolean, onClick: () -> Unit) {
30     Row(modifier = Modifier.fillMaxWidth().clickable(onClick = onClick)) {
31         Text(text = lesson.title)
32         if (isCurrent) Text(text = stringResource(R.string.current_lesson))
33     }
34 }
```

34 }

【话术】面试里怎么说

我会先说影响: `collectAsState` 可能在后台继续收集, 列表没有 key 会让 item 身份随位置漂移; 每次重组新建可变 list 和 lambda, 会让本可跳过的子树继续重组。Then: “I would collect with lifecycle awareness, provide stable item keys, and move state reads closer to the leaf composables so recomposition is scoped.”

根因分析

`collectAsState` 不知道 Android Lifecycle。屏幕进入后台后, 上游 Flow 仍可能被收集, 触发数据库查询、网络合并或状态计算; 对学习路径这种重页面, 后台工作会挤占电量和主线程时间。`collectAsStateWithLifecycle` 会在合适生命周期状态下收集, 停止时取消, 回到前台再恢复。

重组性能的另一半是稳定性与读取范围。Compose 编译器会尽量跳过参数没变且稳定的 Composable; 但普通 List 接口和可变集合容易被视为不稳定, 每次 `toMutableList()` 又创建新对象。`LazyColumn.items` 不给 key 时, item 身份默认跟位置走, 插入、解锁、排序会导致状态和动画错位。每行 `onClick = { navigator.openLesson(lesson.id) }` 都是新 lambda, 也会影响跳过判断。根本解法不是到处撒 `remember`, 而是让状态读取发生在最小 UI 单位、让参数身份稳定、让事件从下往上抬。

```

1  @Composable
2  fun PathRoute(
3      viewModel: PathViewModel,
4      navigator: LessonNavigator
5  ) {
6      val state by viewModel.uiState.collectAsStateWithLifecycle()
7      val onLessonClick: (String) -> Unit = remember(navigator) {
8          { lessonId -> navigator.openLesson(lessonId) }
9      }
10     val onShopClick: () -> Unit = remember(navigator) {
11         { navigator.openShop() }
12     }
13
14     PathScreen(
15         state = state,
16         onLessonClick = onLessonClick,
17         onShopClick = onShopClick
18     )
19 }
20
21 @Composable
22 fun PathScreen(

```

```

23     state: PathUiState,
24     onLessonClick: (String) -> Unit,
25     onShopClick: () -> Unit
26 ) {
27     Column(modifier = Modifier.fillMaxSize()) {
28         TopStatsBar(
29             streak = state.streakDays,
30             gems = state.gemBalance,
31             onShopClick = onShopClick
32         )
33         LazyColumn(modifier = Modifier.fillMaxSize()) {
34             items(
35                 items = state.lessons,
36                 key = { lesson -> lesson.id }
37             ) { lesson ->
38                 LessonNode(
39                     lesson = lesson,
40                     isCurrent = lesson.id == state.currentLessonId,
41                     onClick = { onLessonClick(lesson.id) }
42                 )
43             }
44         }
45     }
46 }
47
48 @Immutable
49 data class PathLesson(val id: String, val title: String, val locked: Boolean)

```

【设计思想】重组的单位是读取状态的最小 composable

真正有效的优化，是让会变的状态只被需要它的最小节点读取。顶部宝石变化不该让整条路径列表都重新计算；某个 lesson 的进度变化也不该让顶部栏重组。状态下沉、事件上提，比机械加 `remember` 更根本。

【陷阱】稳定集合不是语法糖

如果列表很大且频繁变化，可以考虑 `kotlinx immutable collections` 或明确的 `@Immutable` 模型，让 Compose 更有信心跳过未变化节点。但不要用注解掩盖真实可变对象；如果对象内部会被原地修改，Compose 看不到变化，UI 反而会 stale。

【追问】“怎么定位过度重组？”

先用 Android Studio Layout Inspector 看重组计数和跳过次数；再用 Compose Compiler Metrics 看哪些类型不稳定；必要时在可疑 Composable 周围用 `Modifier.drawWithContent` 或

日志打点。定位顺序是先找“谁读了太大的状态”，再看“哪些参数不稳定”。

7.4 !! 强制解包、Java 平台类型与未初始化的 lateinit

查错 #16 | 类别: 空安全

场景

课程 JSON 从后端下发，LessonRepository 解析后交给 UI 渲染。服务端偶尔省略 subtitle，某个 Java SDK 返回平台类型，adapter 又在快速回调路径上提前访问。Duolingo 官方迁移 Kotlin 时提到历史 NPE crash 是重要背景之一 [官方]³；这类代码正是 Kotlin 项目里最该被评审拦下的 NPE 入口。

```
1 class LessonRepository(  
2     private val api: LessonApi,  
3     private val jsonParser: JavaLessonParser  
4 ) {  
5     suspend fun loadLesson(id: String): LessonUiModel {  
6         val response = api.getLesson(id)  
7         val body = response.body()!!  
8         val dto = body.data!!  
9         val metadata = jsonParser.parse(dto.metadataJson)  
10  
11         return LessonUiModel(  
12             id = dto.id!!,  
13             title = dto.title!!,  
14             subtitle = metadata.subtitle.uppercase(),  
15             unitName = dto.unit!!.name!!,  
16             firstExercisePrompt = dto.exercises!![0].prompt!!,  
17             authorNameLength = dto.author!!.name!!.length  
18         )  
19     }  
20 }  
21  
22 class LessonListFragment : Fragment(R.layout.fragment_lesson_list) {  
23     private lateinit var adapter: LessonAdapter  
24  
25     fun onLessonsLoaded(lessons: List<LessonUiModel>) {  
26         adapter.submitList(lessons)  
27     }  
28  
29     override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
30         adapter = LessonAdapter()
```

³<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

```

31     requireView().findViewById<RecyclerView>(R.id.lessons).adapter = adapter
32 }
33 }

```

【话术】面试里怎么说

我会先说影响：这里有多条 NPE 路径，服务端缺字段、HTTP body 为空、Java parser 返回 null、回调早于 adapter 初始化都会崩。Then: “Kotlin null-safety only helps if we model nullability at the boundary. I would remove bang-bang operators, validate DTOs in a mapper, and expose non-null domain models to the UI.”

根因分析

!! 是把 Kotlin 的空安全关掉。它没有恢复数据正确性，只是把“这里可能为空”的信息推迟到运行期崩溃。`response.body()!!` 遇到 204、错误 body 或反序列化失败会崩；`dto.exercises!![0]` 同时假设列表非空；`user.name!!.length` 假设后端永不回 null。移动端面对的是灰度服务端、旧缓存、弱网重试和版本不一致，所有这些假设都会被真实用户击穿。

Java 平台类型是另一个洞。Kotlin 看到 Java 方法返回 `String!`，既可以当可空，也可以当非空；编译器不会替你兜底。`metadata.subtitle.uppercase()` 看起来像安全 Kotlin，实际上 `subtitle` 可能来自 Java null。`lateinit` 也类似：它绕过构造期初始化检查，如果某条异步回调先于 `onViewCreated` 到达，访问 `adapter` 就会抛 `UninitializedPropertyAccessException`。

```

1  class LessonRepository(
2      private val api: LessonApi,
3      private val jsonParser: JavaLessonParser
4  ) {
5      suspend fun loadLesson(id: String): LessonUiModel {
6          val body = requireNotNull(api.getLesson(id).body()) {
7              "Lesson response body is null for lessonId=$id"
8          }
9          val dto = requireNotNull(body.data) {
10              "Lesson response data is null for lessonId=$id"
11          }
12          return dto.toDomain(jsonParser)
13      }
14  }
15
16  private fun LessonDto.toDomain(jsonParser: JavaLessonParser): LessonUiModel {
17      val metadata = jsonParser.parse(metadataJson)
18      val exercises = exercises.orEmpty()
19      val firstPrompt = exercises.firstOrNull()?.prompt ?: ""
20
21      return LessonUiModel(
22          id = requireNotNull(id) { "Lesson id is required" },

```



```

23     title = title.orEmpty(),
24     subtitle = metadata.subtitle?.uppercase().orEmpty(),
25     unitName = unit?.name.orEmpty(),
26     firstExercisePrompt = firstPrompt,
27     authorNameLength = author?.name?.length ?: 0
28 )
29 }
30
31 class LessonListFragment : Fragment(R.layout.fragment_lesson_list) {
32     private var adapter: LessonAdapter? = null
33
34     fun onLessonsLoaded(lessons: List<LessonUiModel>) {
35         adapter?.submitList(lessons)
36     }
37
38     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
39         adapter = LessonAdapter()
40         view.findViewById<RecyclerView>(R.id.lessons).adapter = adapter
41     }
42
43     override fun onDestroyView() {
44         adapter = null
45         super.onDestroyView()
46     }
47 }

```

【设计思想】空值应该在系统边界被消灭

不要让 UI 层到处写 `?.`。防御后端。DTO 可以忠实表达接口可空性，mapper 负责把可空 DTO 收敛成非空 Domain：必需字段用 `requireNotNull` 明确失败，非关键文案用默认值或兜底模型。这样业务层才能重新享受 Kotlin 的非空类型。

【陷阱】不是所有 `!!` 都同罪

`!!` 偶尔可以表达“这里为 `null` 就是程序 bug”，例如测试中强行取 fixture，或某个生命周期不变量已经被前置检查保证。但生产代码里更推荐 `requireNotNull` 或 `checkNotNull`，因为它带错误信息，能告诉崩溃分析系统是哪条不变量被破坏。评审时要区分“该报错”和“该兜底”。

【追问】“`::adapter.isInitialized` 可以吗？”

可以作为过渡防线，但它没有解决生命周期所有权。Fragment View 销毁后 adapter 也应解绑；如果回调可能晚到，可空字段加 `onDestroyView` 置空，比 `lateinit` 更能表达“这东西只在 View 存活期存在”。

7.5 一整段 Java 味的 Kotlin

查错 #17 | 类别: 惯用性综合

场景

成就动画页有一段从 Java 机械转换来的 helper：根据用户状态决定 banner 文案、动画类型和下一步动作。它能编译，也能跑，但停在官方迁移文章所说的“自动转换、修编译错误”阶段，还没有进入惯用化重构 [官方]⁴。查错题里遇到这种代码，不要只说“风格不好”，要把风格和编译期安全、空安全、可维护性连起来。

```
1 class AchievementFormatter private constructor() {
2     companion object {
3         const val STATE_LOCKED = 0
4         const val STATE_UNLOCKED = 1
5         const val STATE_CLAIMED = 2
6
7         @Volatile private var instance: AchievementFormatter? = null
8
9         fun getInstance(): AchievementFormatter {
10             if (instance == null) {
11                 synchronized(AchievementFormatter::class.java) {
12                     if (instance == null) {
13                         instance = AchievementFormatter()
14                     }
15                 }
16             }
17             return instance!!
18         }
19     }
20
21     fun format(user: User?, achievements: List<Achievement>): ArrayList<String>
22     {
23         val result = ArrayList<String>()
24         if (user != null) {
25             result.add(String.format("%s has %d gems", user.getName(), user.
26             getGems()))
27         }
28         if (achievements.size > 0) {
29             for (i in 0 until achievements.size) {
30                 val achievement = achievements[i]
31                 if (achievement.getState() == STATE_UNLOCKED) {
32                     result.add(String.format("Unlocked: %s", achievement.
33                     getTitle()))
```

⁴<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

```

31         } else if (achievement.getState() == STATE_CLAIMED) {
32             result.add(String.format("Claimed: %s", achievement.getTitle
33         ))
34         } else if (achievement.getState() == STATE_LOCKED) {
35             result.add(String.format("Locked: %s", achievement.getTitle
36         ))
37         }
38     }
39     return result
40 }
41 fun animationFor(event: Any): String {
42     if (event is StreakEvent) {
43         return "streak_" + event.days
44     } else if (event is BadgeEvent) {
45         return "badge_" + event.badgeId
46     } else if (event is LeagueEvent) {
47         return "league_" + event.rank
48     }
49     return "none"
50 }
51 }

```

【话术】面试里怎么说

我会避免说“我个人不喜欢这种风格”，而是说：这段 Kotlin 还保留 Java 迁移痕迹，问题不只是审美；手写单例、int 状态和 nullable getter 都把错误留到运行期。In English: “I would frame this as type-safety and readability, not personal taste. Kotlin idioms let the compiler enforce invariants that this code currently checks by convention.”

根因分析

Java 味 Kotlin 的危险在于，它看起来“已经迁移”，实际没有获得 Kotlin 的安全收益。手写双检锁单例又长又容易错，Kotlin `object` 一行就能表达线程安全单例。`STATE_LOCKED = 0` 这类 int 常量无法阻止非法状态 3 进入系统，`when` 也不能做穷尽性检查；sealed class 或 enum 能把状态空间交给编译器。

`if (user != null) { user.getName() }` 可以工作，但 Kotlin 更自然的写法是安全调用、属性和作用域函数。`achievements.size > 0` 不如 `isNotEmpty()` 表达意图；按下标遍历容易引入越界和无意义的索引，直接遍历或 `forEachIndexed` 更贴近集合语义。`String.format` 在简单拼接上噪声大，字符串模板更清楚。`if (event is A) else if` 可以改成 `when`；如果事件是 sealed interface，新增事件类型时编译器会提醒你补分支。

```

1 object AchievementFormatter {

```

```
2 fun format(user: User?, achievements: List<Achievement>): List<String> {
3     val result = mutableListOf<String>()
4
5     user?.let {
6         result += "${it.name} has ${it.gems} gems"
7     }
8
9     achievements.forEach { achievement ->
10         val label = when (achievement.state) {
11             AchievementState.Unlocked -> "Unlocked"
12             AchievementState.Claimed -> "Claimed"
13             AchievementState.Locked -> "Locked"
14         }
15         result += "$label: ${achievement.title}"
16     }
17
18     return result
19 }
20
21 fun animationFor(event: AchievementEvent): String = when (event) {
22     is AchievementEvent.Streak -> "streak_${event.days}"
23     is AchievementEvent.Badge -> "badge_${event.badgeId}"
24     is AchievementEvent.League -> "league_${event.rank}"
25     AchievementEvent.None -> "none"
26 }
27 }
28
29 enum class AchievementState { Locked, Unlocked, Claimed }
30
31 data class Achievement(
32     val title: String,
33     val state: AchievementState
34 )
35
36 sealed interface AchievementEvent {
37     data class Streak(val days: Int) : AchievementEvent
38     data class Badge(val badgeId: String) : AchievementEvent
39     data class League(val rank: Int) : AchievementEvent
40     data object None : AchievementEvent
41 }
```

【设计思想】惯用性的价值不是审美

Kotlin idiom 的核心收益，是把运行期错误搬到编译期：sealed class 让 when 穷尽，非空类型减少 NPE，data class 给出稳定 equality，object 免掉手写并发单例。评审里这样说，风格意见就不再像挑剔，而像风险控制。

【陷阱】不要把所有 Java 写法一棒打死

有些 Java 风格来自互操作边界，例如老 SDK 的 getter、String.format 处理本地化格式、框架要求空构造。评审时要区分边界妥协和业务代码内部惯性。能用类型系统表达的业务规则，不应继续靠注释和 int 常量维持。

【追问】“如果团队正在迁移，是否值得专门做惯用化？”

值得，但要分层推进。官方迁移经验也不是“自动转换就结束”，而是先转 Kotlin，再修编译，再做 idiomatic refactor。优先改高 churn、高 crash、高业务核心的文件；对低风险老代码，可以借触碰时顺手消除 Java-ism，避免大爆炸式重写。

【要点】Compose 与 Kotlin 查错主线

Compose 题看副作用归属：函数体是否纯，effect key 是否表达正确生命周期，状态读取是否被限制在最小范围。Kotlin 题看类型边界：可写性是否最小，null 是否在边界收敛，状态空间是否由 sealed class、enum、不可变 data class 交给编译器。

把第 7 到第 17 题连起来，其实仍是前面方法论里的严重度阶梯：会崩溃的 NPE、会泄漏的生命周期错误、会错行的列表异步、会重复请求的 Compose effect、会退化体验的全量刷新，都比单纯格式问题更高优先级。而所谓“惯用 Kotlin”，最终也不是为了好看，是为了让这些错误更早、更便宜地暴露出来。

Part IV

设计轮：Android 架构设计

Chapter 8

Android 架构设计方法论

[官方] Duolingo 官方工程面试博客明确写到：Design Interview 是 60 分钟、无需写代码；对 iOS/Android 岗位，它考的是移动端架构设计，而不是后端分布式系统设计¹。这句话要先刻进脑子里。后端 System Design 常问的是“你怎么扛住流量”：分片、缓存、复制、一致性、QPS、热点。Android 架构设计问的是另一件事：**你怎么在一台随时会断网、随时会被系统杀掉、存储与电量都受限的设备上，保证用户看到的東西是对的。**

这不是难度降低，而是坐标轴换了。你仍然要谈一致性，但一致性不再只是数据库副本之间的一致；它变成了 UI 状态、本地数据库、内存缓存、离线队列、服务端状态之间的一致。你仍然要谈可用性，但可用性不再只是服务 99.9% 在线；它变成了用户在地铁里学完一课、重启 App 后进度不能丢。你仍然要谈性能，但性能不再只是 p99 API 延迟；它变成了入门级 Android 设备上首帧、滚动、Lesson start 的体感。

[官方] Duolingo 公开复盘过一次 Android 重构：巨大的全局可变 **DuoState** 被 XP、streak、lessons 等模块共享，ANR 每月恶化 5–10%；某次发布让 crash 增加 10%、帧渲染慢 25%、入门级设备 lesson start 慢 70%。他们用 8 周 feature freeze 推进 MVVM、Dagger/Hilt、Repository 与模块化，换来 ANR 下降 41%、missed frame 下降 28%、滚动提升 40%²。这段材料几乎就是 Android 设计轮的评分 rubrics：状态边界、依赖边界、生命周期、低端设备、可测试性。

¹<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

²<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

维度	后端 System Design	Android 架构设计
核心问题	如何承载海量请求与数据	如何在不可靠设备上保持正确体验
扩展性	分库分表、缓存、队列、读写分离	分层、模块化、状态隔离、可测试 ViewModel
一致性	副本一致、事务、CAP、最终一致	UI/内存/Room/DataStore/服务端之间的收敛
失败模式	节点宕机、网络分区、热点、消息重复	断网、进程被杀、低内存、磁盘满、电量限制
性能指标	QPS、p95/p99、吞吐、容量	首帧、滚动掉帧、ANR、启动耗时、流量与电量
写路径	幂等 API、事务、消息队列	乐观更新、离线队列、WorkManager、回滚策略

【设计思想】移动端设计轮的主语

不要把 Android 设计题讲成“客户端只是调 API”。移动端的主语是**本地状态机**：用户动作先进本地状态机，状态机决定 UI、持久化、网络同步、失败回滚。服务端是权威来源，但用户体验发生在设备上。

8.1 六步答题框架

下面这套六步法，是本书后面所有 Duolingo Android 设计题的骨架。你不需要机械背诵每一句，但要形成固定顺序：先定边界，再画数据流，再选存储，再讲写路径，再讲状态与乐观更新，最后主动打失败场景。

8.1.1 第一步：澄清需求（5 分钟）

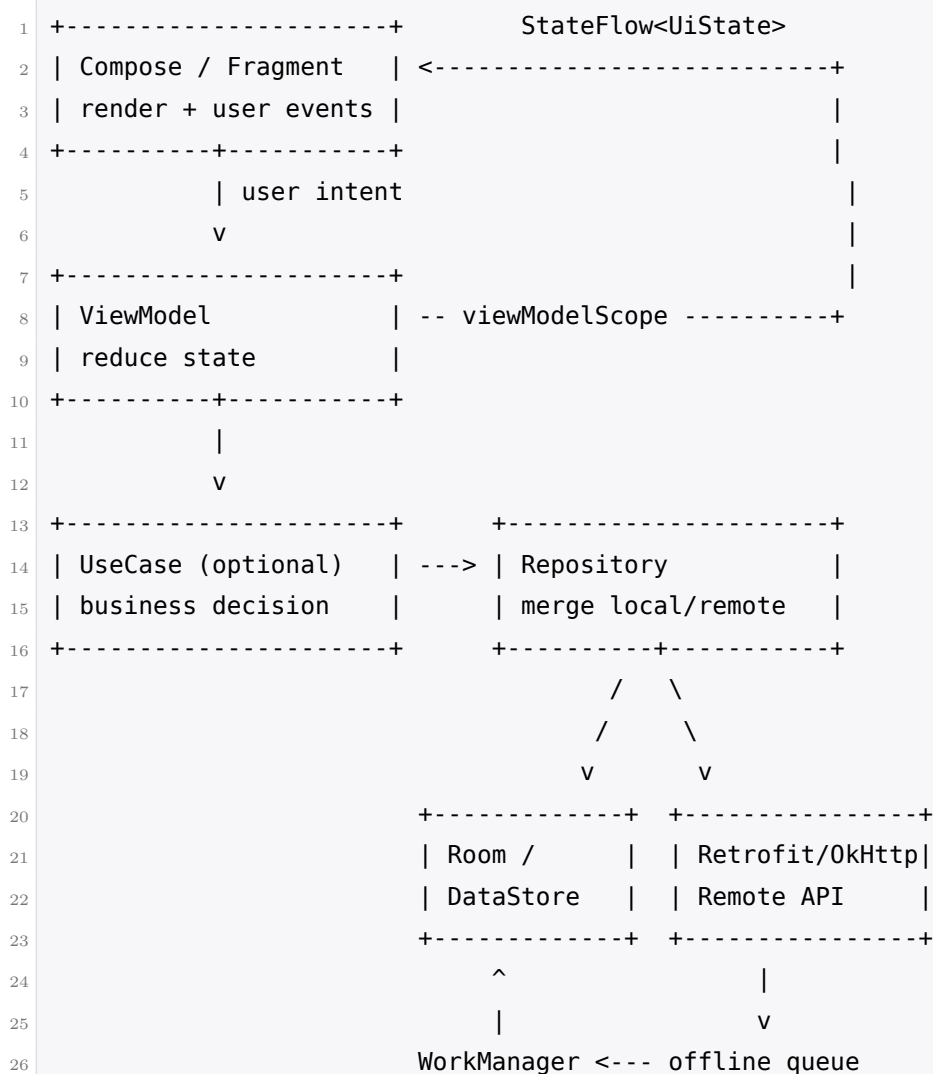
开场不要急着画 Repository。先问四类问题。

- **功能边界**：用户要完成哪一个主路径？是只展示，还是要写入？要不要支持取消、撤销、重试？
- **离线要求**：断网时是只读旧数据，还是允许继续学习并稍后同步？如果离线写入失败，用户是否已经看到成功 UI？
- **实时性与一致性**：这个数据错 5 秒会怎样？错一天会怎样？是否涉及钱、排名晋级、公平性、不可逆资源？
- **设备档位**：目标设备是否包括低内存、慢存储、弱网络的入门级 Android？[\[官方\]](#) Duolingo 明确关注入门级设备体验。

好的澄清不是“我问完了”，而是“我把后面的复杂度裁剪出来”。如果面试官说 leaderboard 晋级必须公平，你就不能用纯乐观更新；如果面试官说课程包必须离线可学，你就必须谈内容版本、断点续传、校验和磁盘配额。

8.1.2 第二步：分层与数据流 (8 分钟)

[官方] Duolingo 公开技术栈包含 Kotlin、MVVM、Dagger/Hilt、Repository，并在推进 Compose；**[高置信推断]** 标准 Jetpack 组件通常会包括 Room、DataStore、WorkManager、Retrofit/OkHttp 等。面试里不要堆名词，要把箭头画清楚：UI 不直接碰网络，ViewModel 不直接碰数据库细节，Repository 负责把 Local 与 Remote 收敛成一个可观察的数据源。



这张图有三个暗含约束。第一，数据流尽量单向：UI 发 intent，ViewModel 变更状态，UI 观察状态。第二，本地存储不是缓存的代名词；在离线题里它是写路径的一部分。第三，WorkManager 与离线队列不是“补充组件”，而是进程死亡后的持久化执行边界。

8.1.3 第三步：本地存储选型 (5 分钟)

移动端设计题很爱看你是否知道“该把什么放哪里”。不要说“都放 SQLite”。用下面这张表。

选项	适合什么	不适合什么
Room	结构化数据、需要查询、需要关系、需要 Flow 观察；如课程、进度、离线队列	大文件、极少量偏好开关
DataStore	少量键值、用户偏好、实验 assignment、streak 快照；异步且类型安全	复杂查询、大量列表、频繁分页
SharedPreferences	维护遗留代码时遇到；简单同步读取	新代码首选；同步 IO 与一致性风险较高
文件与目录	音频、图片、课程包、可校验资源；便于 LRU 清理	需要 SQL 查询的业务实体
内存缓存	当前会话内的热数据、解码后对象、图片内存层	进程被杀后必须保留的数据

[高置信推断] 对 Duolingo 这类学习产品，课程内容、题目、进度、pending operations 通常适合 Room；streak 当前值、提醒时间、实验 assignment 适合 DataStore；音频与图片适合文件缓存加索引。

8.1.4 第四步：同步与写路径（10 分钟）

写路径是移动端设计轮的分水岭。只会说“调用 API，失败 toast”是不够的。你要回答：用户点按钮后，UI 何时变？本地何时落盘？网络何时发？失败时怎么重试？重复请求怎么去重？

```

1  用户动作
2  |
3  +-- 是否必须马上得到服务端确认？
4  |
5  +-- 是：直接 suspend 调 API，显示 loading，失败不落最终状态
6  |     例：购买 gems、排行榜晋级确认
7  |
8  +-- 否：先写本地事务 + pending operation
9  |
10 +-- UI 从 Room/StateFlow 立刻看到新状态
11 +-- WorkManager 在网络可用时上传
12 +-- 服务端用 idempotency key 去重
13 +-- ACK 后删除 pending operation

```

WorkManager、前台 Service、裸协程的选择可以用一句话定调：

- **裸协程**：用户还停留在当前页面，任务短、可丢弃或可由本地状态恢复。
- **WorkManager**：需要保证“最终会执行”，可延迟，受网络/电量约束，进程被杀后仍要续跑。

- **前台 Service**: 用户明确知道有一个长任务正在进行, 必须持续执行并展示通知, 例如大课程包下载。

幂等要主动说。客户端为每次逻辑写操作生成 UUID: lesson session、follow action、streak freeze consume 都有自己的 idempotency key。服务端看到重复 key 返回同一结果, 而不是重复加 XP 或重复消耗资源。

8.1.5 第五步: 状态管理与乐观更新 (10 分钟)

UI 状态不要只写

codeLoading/Success/Error 三态。真实业务常见第四态: **有旧数据的 Loading**。例如排行榜 tab 上次有数据, 这次进入页面刷新中; 如果你清空列表显示 spinner, 体验会抖。更好的状态是旧列表继续显示, 顶部或下拉区域显示 refreshing。

```
1 sealed interface LessonUiState {  
2     data object EmptyLoading : LessonUiState  
3     data class Data(  
4         val lesson: Lesson,  
5         val isRefreshing: Boolean = false,  
6         val pendingSync: Boolean = false,  
7         val errorMessage: String? = null,  
8     ) : LessonUiState  
9     data class FatalError(val message: String) : LessonUiState  
10 }
```

乐观更新是 Duolingo 设计题里最高信号的话题。[\[官方\]](#) 官方 frontend prediction 文章定义了这个模式: 客户端在网络返回前, 基于预期结果先更新 UI; 真实请求在后台发送; 如果结果不一致, 再协调³。课程进度、排行榜 XP、Follow 按钮都是官方讨论过的例子。

但乐观更新不是免费午餐。官方文章点出了两个成本: **状态重复**, 因为前后端各维护一份看起来相同的状态; **逻辑重复**, 因为 increment、XP 计算、boost 判断等规则要在两侧实现, 业务变更时容易漂移。面试官真正想听的不是“我会 optimistic UI”, 而是“我知道这笔交易何时划算”。

8.1.6 第六步: 边界与失败 (10 分钟)

最后十分钟主动扫失败场景。不要等面试官问。

- **断网**: 哪些操作允许排队? 哪些必须失败? 网络恢复由 `ConnectivityManager.NetworkCallback` 触发同步, 真正执行交给 `WorkManager`。
- **进程被杀**: `ViewModel` 状态会丢, `Room/DataStore` 与 `WorkManager request` 不会丢; 临时表单用 `SavedStateHandle`。
- **多设备冲突**: 服务端是 source of truth; 客户端用保守合并, 学习进度一般不 downgrade。

³<https://blog.duolingo.com/frontend-prediction/>

- **时区**: streak、每日目标、提醒都不能只信设备时间; 用服务端返回的账号时区定义 day boundary。
- **存储配额**: 课程包、音频、图片必须有大小上限、LRU、校验与重新下载策略。
- **低内存**: 列表分页、图片解码尺寸、避免把大课程包全读进内存。
- **版本兼容与迁移**: Room migration、DataStore schema 演进、老客户端与新服务端字段兼容。

8.2 什么时候该用乐观更新

[官方] Duolingo 把 frontend prediction 用在三类场景: 课程进度离线递增、排行榜 XP 预测、Follow 按钮即时反馈。机制很简单: 先把 UI 改成“我们预计会成功的样子”, 再发请求, 最后对账。但决定是否使用它, 必须同时看用户心理、可逆性、成本和公平性。

【设计思想】乐观更新的本质

乐观更新是把“延迟”换成“不一致的风险”。你必须能说出这笔交易在这个场景下划不划算: 用户是否强烈期待立刻反馈? 错误是否可逆? 短暂不一致是否会伤害信任、公平或金钱?

官方课程进度例子里, 离线时无法访问服务器, 客户端先把本地 counter 加一, 等恢复网络再同步。这里的产品判断是: 用户学完课, 进度条立刻前进比“等网络确认”更重要; 即使最后服务端发现细节不一致, 也可以保守合并。代价是状态重复与逻辑重复: 前端和后端都知道如何 increment, 规则一改就可能漂移。

排行榜 XP 更复杂。**[官方]** 官方文章说, 等待后端计算 XP 会让 session end card 变慢, 因为 XP 可能受 bonus、boost、accuracy 等因素影响。客户端可以预测并立即展示, 但不一致来源很多: 前端和后端对“答对题数”的口径不同; 前端认为 XP boost 生效, 后端却不认。于是回滚策略成为核心。

策略	用户可见结果	风险与成本
Never roll back	用户一直看到预测值; 体验最顺	风险最高。用户可能看到“你是第一名”, 真实服务端却不是, 信任与公平受损
Immediate rollback	session end card 先显示预测; leaderboard tab 立刻显示真实	同一时间两处 UI 不一致, 用户困惑: “刚才不是加了 20 XP 吗?”
Deferred rollback	本次会话内都显示预测; App 重启或下次冷启动后替换为真实	体验连续, 但调试困难; bug 可能只在重启后显现

我的面试答案通常是: **对 session end card 可以预测; 对 leaderboard tab 可短时间采用 deferred rollback; 对晋级、奖励发放、结算必须等后端确认。**原因是 card 是即时反馈, 预测错误的心理成本较低; leaderboard 排名牵涉社交比较, 不能让用户短暂看到极端错误; 晋级涉及公平性, 官方明确说不适合 prediction。

[官方] 官方也给了反例: 购买 gems 等 monetized resources 不适合预测, 因为用户期望精确; 排行榜晋级不适合预测, 因为公平性必须由后端确认。

【追问】“前后端算出来的 XP 不一样怎么办？”

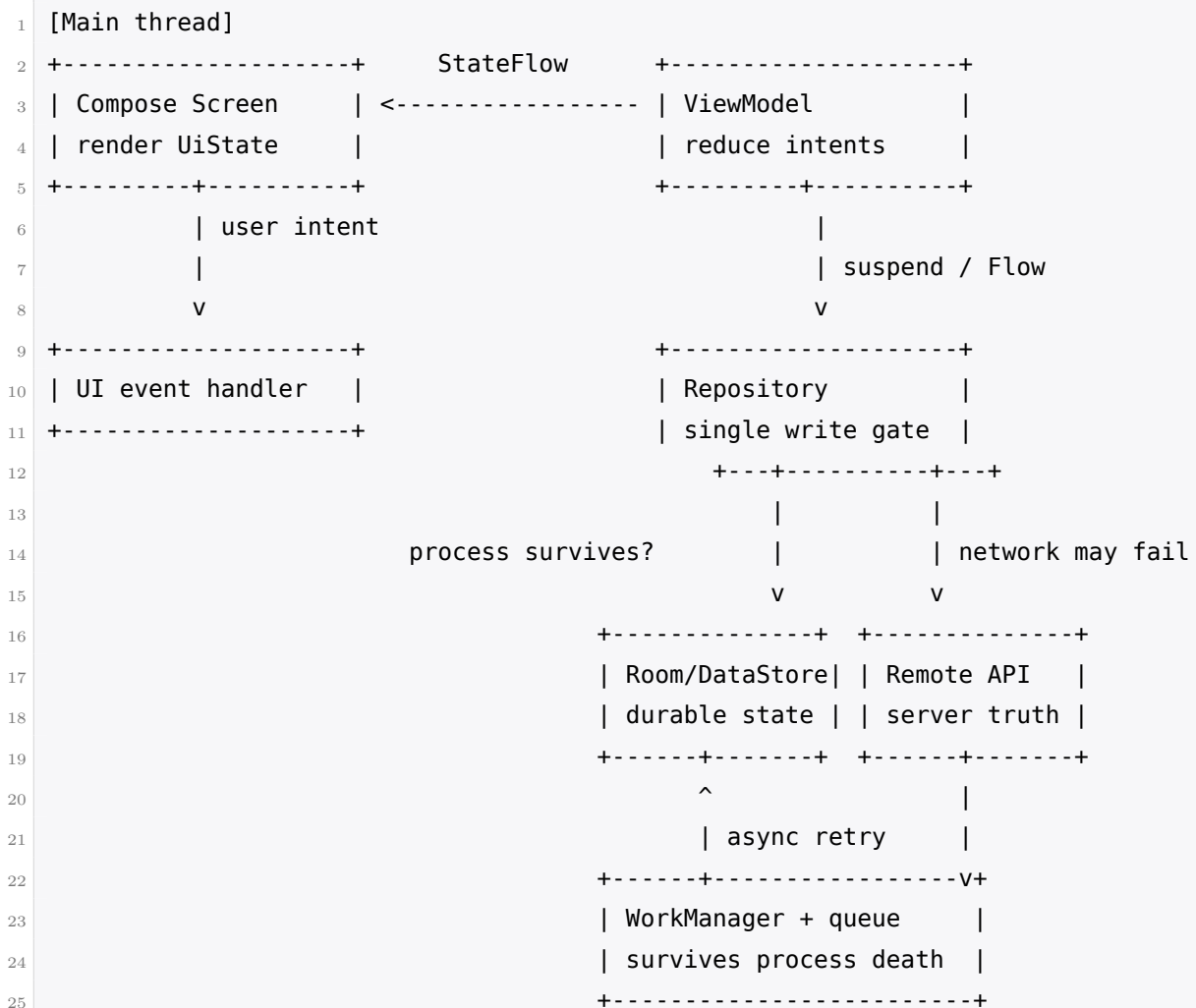
先承认这是 prediction 的核心风险，而不是甩锅给后端。可选解法有三层：第一，把计算规则收敛到一处，例如服务端返回可配置规则或共享规则版本，客户端只预测能完整复现的部分；第二，服务端为准，但延迟替换，避免同屏立即打脸；第三，只预测低风险量，例如 session card 上的临时 XP，不预测晋级和奖励发放。面试里要明确说：如果规则复杂到客户端无法可靠复现，就缩小预测范围。

【要点】乐观更新决策规则

可逆、低代价、用户心理预期“立刻发生”的操作，可以乐观：Follow、点赞、课程完成后的本地进度。涉及钱、涉及公平排名、涉及不可逆资源、涉及风控的操作，不乐观：购买 gems、排行榜晋级、订阅权限生效。

8.3 画图与表达

移动端架构图要比后端图多标三件事：线程边界、持久化边界、进程死亡边界。面试官很吃这一套，因为它说明你真的很在 Android 上踩过坑。



每条边最好标同步还是异步。UI 到 ViewModel 是同步 intent；ViewModel 到 Repository 可能是 suspend；Repository 到 Room 是本地事务；Repository 到 Remote 是网络；WorkManager 到 Remote 是异步重试。再用虚线或文字标出“进程被杀后还在”的组件：Room、DataStore、WorkManager request、文件缓存。

【话术】60 分钟开场话术

“我先按 Android 架构题来回答，而不是后端分布式系统题。我的重点会放在本地数据源、离线写路径、状态一致性、进程被杀后的恢复，以及哪些地方可以乐观更新。最后我会扫一遍低端设备、存储、电量和迁移边界。”

【话术】从高层图切到深挖

“高层图到这里能跑通主路径。接下来我会深挖两个最容易出错的点：第一是写路径如何在断网和重复提交下保持幂等；第二是 UI 先更新后回滚时，用户会看到什么。”

8.4 自查表

拿到任何移动端设计题，都用这张表检查自己的答案是否完整。

- 我是否先澄清了离线、实时性、一致性、设备档位？
- 图里是否有 UI、ViewModel、Repository、Local、Remote、WorkManager？箭头是否单向？
- 我是否说明了 Room、DataStore、文件、内存缓存各放什么？
- 写操作是否有 idempotency key？失败是否可重试？重复请求是否安全？
- UI 状态是否区分 empty loading、data refreshing、error with old data？
- 如果用了乐观更新，我是否说清了状态重复、逻辑重复、回滚策略？
- 进程被杀后，哪些状态仍然存在？哪些需要从 SavedStateHandle 或 Room 恢复？
- 多设备冲突时，服务端是否是 source of truth？客户端是否避免 downgrade？
- 是否考虑了时区、磁盘配额、低内存、电量、Doze、版本迁移？
- 我是否明确指出哪些场景不能乐观：钱、公平晋级、不可逆资源？

Chapter 9

设计题精讲（上）：离线、连胜、排行榜与实验

这一章进入真正的 60 分钟 Android 设计题。四道题都按同一结构展开：题面、澄清、架构图、数据模型、关键机制、边界、追问。请注意来源标注：**[面经报告]** 表示社区面经反复报告；**[高置信推断]** 表示没有官方题面，但由 Duolingo 官方技术材料、职位描述和产品形态高置信推断；**[官方]** 表示来自 Duolingo 官方公开材料。

9.1 离线课程下载与进度同步

设计题 #1 | 难度：**Hard**

[面经报告] 多份社区面经报告过离线课程、进度同步、学习 session 同步类题目；**[官方]** Duolingo 官方 frontend prediction 文章也直接讨论了课程进度在离线时先本地递增、恢复网络后同步的场景¹。因此这题既像真实题，也高度贴合官方关注点。

9.1.1 题面与常见变体

设计一个 Android 客户端能力：用户可以下载课程内容，离线完成 lesson；App 恢复网络后把进度、答题结果和 XP 同步到服务端。常见变体有三种：

- “如何支持离线学习，并保证进度最终同步？”
- “课程包很大，下载中断后怎么办？课程内容更新怎么办？”
- “同一个账号在两台设备离线完成同一课，回来后如何合并？”

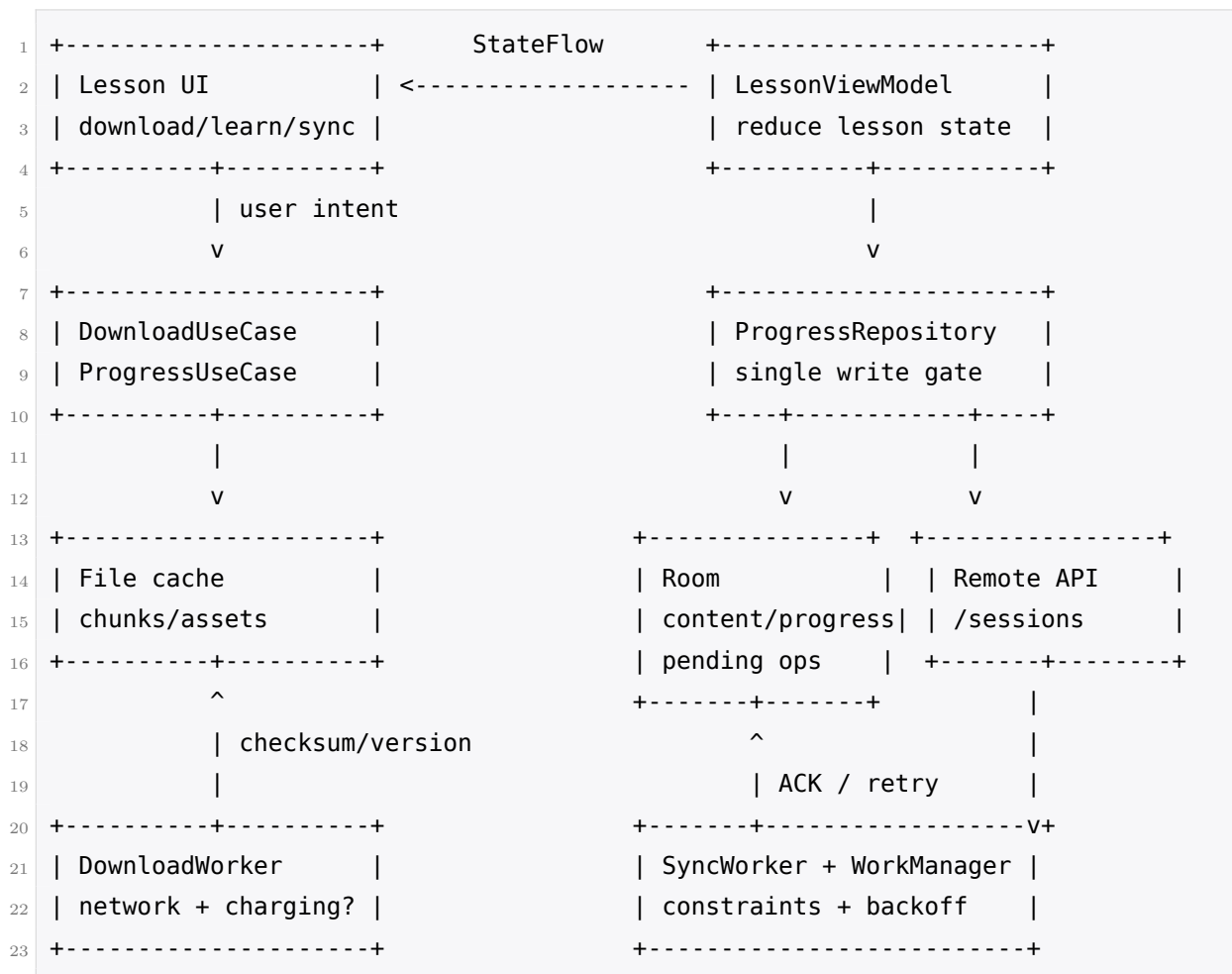
9.1.2 需求澄清：你该问什么

- 问：离线时只允许学习已下载课程，还是也要浏览新课程？答：只保证已下载内容可学。
- 问：进度是否允许乐观显示？答：允许，学习产品里完成反馈要即时；但服务端最终为准。

¹<https://blog.duolingo.com/frontend-prediction/>

- 问：课程内容更新时，正在学习旧版本的用户是否要被打断？答：不要打断，当前 session 跑完旧版本。
- 问：同步失败是否阻塞用户继续学习？答：不阻塞，但要可见地标记 pending sync。
- 问：目标设备？答：包括低端 Android；磁盘、电量和弱网都要考虑。[\[官方\]](#) Duolingo 公开强调入门级设备体验。

9.1.3 架构总览



关键是把“课程内容下载”和“学习进度同步”拆成两条写路径。内容包是大文件，重点是断点续传、校验、版本和清理；进度是小而关键的业务事件，重点是本地事务、离线队列、幂等、合并。

9.1.4 数据模型

```

1  @Entity(tableName = "lesson_content")
2  data class LessonContentEntity(
3      @PrimaryKey val lessonId: String,
4      val courseId: String,
5      val version: Long,
  
```



```

6     val manifestUrl: String,
7     val localPath: String?,
8     val sizeBytes: Long,
9     val checksum: String,
10    val status: DownloadStatus,
11    val lastAccessAt: Long,
12 )
13
14 @Entity(
15     tableName = "lesson_progress",
16     primaryKeys = ["lessonId", "contentVersion"]
17 )
18 data class LessonProgressEntity(
19     val lessonId: String,
20     val contentVersion: Long,
21     val completedQuestionIds: Set<String>,
22     val lastSessionUuid: String?,
23     val isComplete: Boolean,
24     val updatedAt: Long,
25     val pendingSync: Boolean,
26 )
27
28 @Entity(tableName = "pending_operations")
29 data class PendingOperationEntity(
30     @PrimaryKey val opId: String,           // client-generated UUID
31     val type: String,                       // COMPLETE_LESSON, ANSWER_BATCH
32     val idempotencyKey: String,
33     val payloadJson: String,
34     val createdAt: Long,
35     val attempt: Int,
36     val nextRunAt: Long,
37     val lastError: String?,
38 )

```

Room 存结构化内容索引、进度和 pending operations；文件目录存实际媒体与题目包；DataStore 可以存全局配额、最近同步时间、用户选择的自动下载策略。[\[高置信推断\]](#) 这是标准 Jetpack 选型，不是 Duolingo 官方确认的具体实现。

9.1.5 关键机制逐个击破

课程内容包：分片、断点续传、校验、版本化

课程包不要建模成“一个 URL 下完就完”。更稳的设计是 manifest 描述版本、总大小、chunk 列表和校验值；DownloadWorker 逐 chunk 下载到临时文件，成功后校验 checksum，再原子 rename 到正式目录。断点续传可以靠 HTTP range 或 chunk 粒度状态。低端设备上不要边下边解压到大

量小文件而没有配额控制，否则存储碎片和 IO 会拖慢 lesson start。

内容版本号是这题的灵魂。用户开始 lesson 时, ViewModel 绑定 `lessonId + contentVersion`。如果服务端发布新版本，已开始的 session 继续使用旧版本；下次进入再提示更新。这样可以避免“做到第 8 题，题目忽然被替换，答题结果无法解释”。

进度写路径：本地事务先行，后台同步

用户完成一课时，Repository 在一个 Room transaction 中同时更新 `lesson_progress` 与插入 `pending_operations`。UI 从 Room Flow 立刻看到完成状态和 pending sync 标记。随后 WorkManager 在网络可用时上传。这个设计的好处是：进程在“用户看到成功 UI”和“网络请求发出”之间被杀，也不会丢进度。

```
1 complete lesson locally:
2 BEGIN Room transaction
3     upsert lesson_progress(isComplete=true, pendingSync=true)
4     insert pending_operations(opId=sessionUuid, idempotencyKey=sessionUuid)
5 COMMIT
6 enqueue unique sync work
```

WorkManager 约束要具体：同步小进度只要求 network connected；大内容下载可要求 unmetered 或 charging，取决于用户设置。ExistingWorkPolicy 用 KEEP 或 APPEND_OR_REPLACE，避免每次完成一题都启动一堆重复 worker。失败重试使用指数退避，并把 lastError 落库给调试与 UI。

幂等与逐条 ACK

每个学习 session 由客户端生成 UUID，作为 idempotency key。服务端看到同一 key 的 PUT /sessions 或 POST /sessions 重试，返回同一结果，不重复加进度或 XP。同步队列不要“一批失败全失败”；更好的协议是服务端逐条 ACK。客户端只删除成功 ACK 的 pending operation，失败项保留并更新 attempt。这样上线后如果某类 payload 因服务端校验失败，不会堵住整个队列。

冲突合并：服务端为准，但保守合并

多设备离线是必问追问。假设手机 A 和平板 B 都离线完成同一 lesson。A 先上线，服务端记录 session A；B 后上线，服务端发现 lesson 已完成。学习产品里我的建议是保守合并：进度取并集或最大值，绝不 downgrade。也就是说，如果客户端说完成了更多题，服务端可以验证题目属于该 lesson 后合并；如果服务端已有完成状态，B 的重复完成不应该把状态回退。

这背后是产品判断：[\[高置信推断\]](#) 对语言学习产品，宁可多给一点进度，也不要因为同步冲突让用户“学过的消失”。但这不适用于钱和公平晋级；XP 与 leaderboard 仍要以服务端计算为准。

网络恢复与队列调度

ConnectivityManager.NetworkCallback 适合触发“可以同步了”的信号，但真正执行交给 WorkManager。原因是 callback 只说明此刻网络可用，不保证 App 进程一直活着；WorkManager

才能跨进程死亡持久化任务。同步 worker 每次从 Room 取 nextRunAt 到期的一小批操作，按创建时间上传，避免一次性读出几千条导致内存压力。

磁盘配额与 LRU 清理

课程包、音频、图片都必须有上限。Room 的 content 表记录 sizeBytes 与 lastAccessAt；清理器在总量超过阈值时删除未 pinned、未 active session、最久未访问的包。删除文件后同步更新 Room，避免索引指向不存在的路径。正在学习的版本要 pin 住，直到 session 结束。

9.1.6 边界情况

- 上传成功一半：逐条 ACK；成功删除，失败保留，不阻塞后续可独立操作。
- 课程版本过期：当前 session 继续旧版本；完成 payload 带 contentVersion，服务端按版本解释。
- 磁盘满：下载前预估空间；失败后清理 LRU；仍不足则给用户明确提示。
- App 被杀：Room transaction 已提交，pending operation 仍在；WorkManager 之后续跑。
- 服务端返回 new_progress 与预期不符：服务端为准，但如果会 downgrade 学习完成状态，要走产品确认或保守合并规则。

【追问】“WorkManager、前台 Service、裸协程怎么选？”

小同步用 WorkManager：可延迟、要跨进程死亡、需要网络约束。大课程包下载如果用户明确点了“下载整套课程”，且期望持续进行，用前台 Service 或 expedited work 并展示通知；否则容易被系统限制。裸协程只适合页面内短任务，例如保存草稿或发一个可重试的即时请求，不能承担离线最终一致的保证。

【追问】“进程被杀时正在上传 session 怎么办？”

上传前 operation 已在 Room。若请求已经到达服务端但客户端没收到响应，WorkManager 重试时携带同一 idempotency key，服务端返回第一次结果。若请求根本没发出，重试就是第一次执行。两种情况客户端逻辑相同，这就是幂等键的价值。

【追问】“如何测试离线逻辑？”

单元测试用 fake Repository 和 fake clock 验证 Room transaction、队列状态、合并规则；instrumentation 测试用 MockWebServer 模拟断网、500、超时、重复 ACK；端到端测试覆盖“完成 lesson 后杀进程、重启、恢复网络仍能同步”。还要有 migration 测试，确保 pending operations 在版本升级后不丢。

【设计思想】可迁移心法

离线设计不是“缓存一点数据”，而是“把写操作变成可持久化、可重试、可幂等、可合并的事件”。只要你能说清事件的生命周期，离线课程、离线评论、离线收藏都是同一题。

【陷阱】只存最终状态，不存操作

如果只在本地把 progress 改成 complete，却不记录是哪一次 session、何时完成、用哪个内容版本完成，恢复网络后就无法解释这次写入，也无法幂等重试。离线写路径必须存 operation log。

9.2 连胜 (Streak) 客户端

设计题 #2 | 难度: **Medium**

[面经报告] 社区面经报告过 Streak 客户端设计；**[高置信推断]** Streak 是 Duolingo 核心产品机制，Android 端必然涉及本地展示、提醒、离线预测与服务端校准。

9.2.1 题面与常见变体

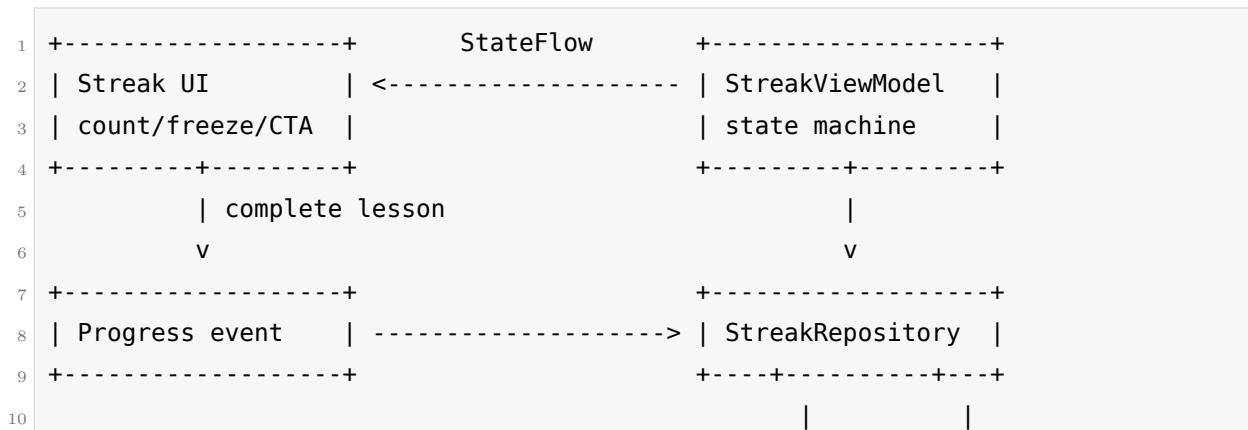
设计 Duolingo Android 的 streak 客户端：用户每天完成学习后连胜加一；支持 Streak Freeze；支持本地提醒；断网和跨时区时仍给出合理体验。变体包括：

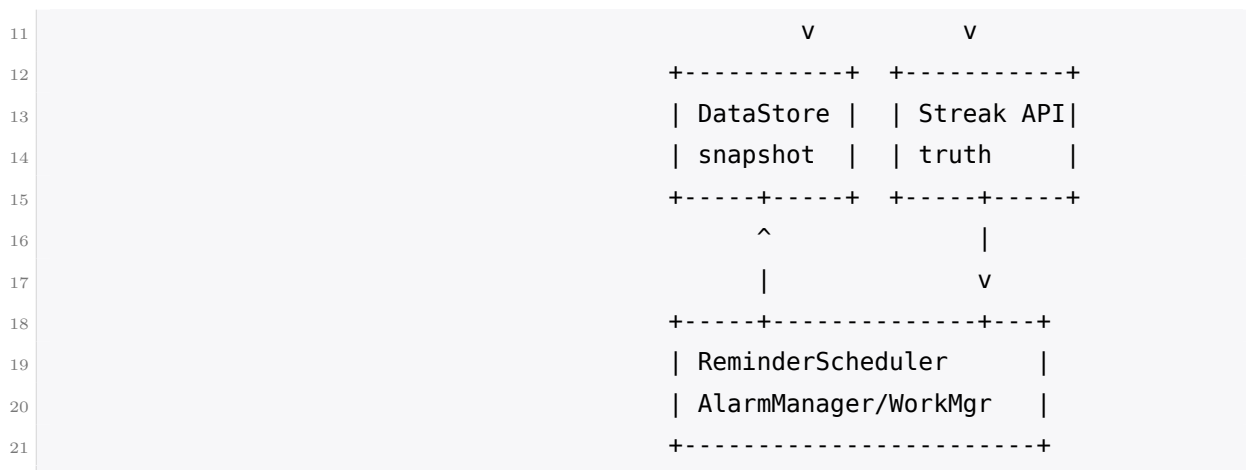
- “今天”的边界按什么算？
- 离线完成课程后 streak 是否立刻加一？
- 如何调度“快失去连胜了”的提醒？

9.2.2 需求澄清：你该问什么

- 问：day boundary 按设备时区还是账号时区？答：按服务端返回的用户时区。
- 问：离线完成是否可以先显示 streak 增加？答：可以乐观展示，但最终以服务端校准。
- 问：Streak Freeze 是付费/库存资源吗？答：是有限资源，消耗必须幂等且服务端确认。
- 问：提醒是否必须精确到分钟？答：普通提醒不必，临近截止的紧急提醒尽量准但要尊重 Android 限制。

9.2.3 架构总览





Streak 的业务数据很小，DataStore 足够：当前 streak、最后完成日期、freeze 库存、账号时区、服务端校准时间。真正复杂的不是存储，而是“今天”的定义、时间作弊、freeze 幂等和提醒调度。

9.2.4 数据模型

```

1  @Serializable
2  data class StreakSnapshot(
3      val currentStreak: Int,
4      val lastCompletedLocalDate: String?, // date in account timezone
5      val accountTimeZone: String,
6      val freezeCount: Int,
7      val pendingOptimisticDay: String?,
8      val serverVersion: Long,
9      val updatedAtMillis: Long,
10 )
11
12 data class FreezeConsumeOp(
13     val dateKey: String, // e.g. userId + yyyy-MM-dd
14     val idempotencyKey: String, // scoped to that streak day
15     val createdAtMillis: Long,
16 )

```

9.2.5 关键机制逐个击破

什么算“今天”：账号时区优先

不要按设备当前时区直接算 streak。用户旅行、系统时间错误、手动改时间都会让结果不稳定。更稳的规则是：服务端返回账号时区与当前 streak 状态；客户端用账号时区计算展示上的 local date 和剩余时间。设备时间只用于倒计时 UI，不能作为最终判定。

用户跨时区旅行时，客户端可以提示“你的连胜按账号时区计算”。如果产品允许用户改账号时区，改动应由服务端确认，并有频率限制，避免通过频繁改时区延长一天。

完成课程后的乐观加一

课程完成是用户心理预期立刻发生的操作。[\[官方\]](#) 官方 frontend prediction 的课程进度例子支持“离线先本地递增，之后同步”。[\[高置信推断\]](#) 对 streak 也可以采用类似策略：如果今天还未完成，完成 lesson 后本地 snapshot 把 currentStreak 加一，并记录 pendingOptimisticDay。若服务端之后确认，清掉 pending；若服务端拒绝，通常不应立刻把用户刚看到的 streak 打回去，而是展示“同步中/需要修复”的状态，并在下一次刷新时校准。

这里要诚实：streak 的最终规则可能包括作弊检测、lesson 有效性、订阅修复等，客户端不能完全复现。客户端的乐观值是体验层，不是权威账本。

Streak Freeze 的幂等

Freeze 是有限库存，不能因为重试消耗两次。同一天触发 freeze 的操作必须有 date-scoped key，例如 `consume-freeze:userId:2026-08-13`。客户端离线时可以展示“freeze pending”，但真正扣库存应以后端确认为准。服务端若看到同一天重复请求，返回第一次消耗结果。

【要点】同一天只能消耗一次

Streak Freeze 的幂等粒度不是“请求 UUID”这么简单，而是“用户 + streak day”。如果每次重试都生成新 UUID，服务端无法知道它们是同一天同一次逻辑消耗。

本地提醒调度

提醒有两类。第一类是用户设定的每日提醒，比如晚上 8 点学习；这类可以用 WorkManager 或普通 Alarm，允许一定不精确。第二类是“还有 3 小时保住连胜”的紧急提醒；它更需要接近截止时间。

Android 12+ 对 exact alarm 有权限限制，Doze 也会延后后台任务。我的结论是：默认用 WorkManager/非精确 alarm 调度普通提醒；对临近 streak 截止的高价值提醒，如果产品确实要求准点，再申请 exact alarm 权限或使用可解释的降级策略。不要为了所有提醒都申请精确闹钟，这会增加权限摩擦和电量成本。

里程碑动画与状态机

Streak UI 不只是一个数字。可能有状态：normal、today completed、freeze protected、lost、repair available、milestone celebration。动画要由状态机驱动，且“庆祝已播放”最好记录在 DataStore，避免旋转屏幕或进程重建重复播放。

9.2.6 边界情况

- 手动改系统时间：UI 可受影响但不提交权威结果；下次服务端校准纠正。
- 离线 7 天：本地可展示 pending days，但服务端回来后逐日验证；freeze 按 date-scoped key 幂等消耗。
- 多设备：服务端 streakVersion 更新；客户端收到较新 version 后覆盖本地权威字段，保留本地 pending UI 标记。

- 断连胜 UI：不要只显示失败；提供修复连胜入口，但购买或消耗资源必须服务端确认。

【追问】“用户把系统时间调到明天，能不能提前加 streak？”

不能把设备时间作为权威。客户端可以根据账号时区和最近服务端时间估算展示，但提交给服务端的 lesson session 带真实 server receipt 或同步时由服务端判定属于哪一天。若离线期间设备时间异常，回来后服务端校准。

【追问】“离线 7 天回来怎么算？”

客户端上传 7 天的 completion events，每条带 session UUID、内容版本和客户端观察到的账号日期。服务端按账号时区、有效 lesson、freeze 库存逐日计算。客户端可以先展示乐观连续，但最终用服务端返回的 streak timeline 替换，并避免立刻造成突兀 downgrade：用解释性 UI 告诉用户哪些天已同步、哪些需要修复。

【追问】“客户端和服务端 streak 不一致，以谁为准？”

服务端为准，因为 streak 涉及奖励、修复、可能的付费资源和作弊防护。客户端可以乐观展示，但必须标记 pending，并在权威结果回来后收敛。若收敛会降低用户刚看到的数字，要用产品化过渡，而不是静默跳变。

【设计思想】可迁移心法

凡是“每日一次”的功能，都先定义 day boundary，再定义幂等粒度。Streak、每日任务、签到、免费宝箱，本质都是时间分桶下的状态机。

【陷阱】直接信设备时间

设备时间可以被用户改，也可能因为旅行而变化。用它做最终 streak 判定，会制造客服无法解释的边界 bug。

9.3 排行榜客户端与 XP 预测

设计题 #3 | 难度：Hard

[高置信推断] 这题未被官方确认为面试原题，但 **[官方]** Duolingo 官方 frontend prediction 文章详细描述了排行榜 XP 预测、session end card、leaderboard tab 以及三种回滚策略，因此它是高概率、高价值设计题。

9.3.1 题面与常见变体

设计 Android 排行榜客户端：用户完成 lesson 后获得 XP，session end card 立即展示 XP 与排名变化；leaderboard tab 展示同组用户排名；弱网下也要体验顺滑。变体包括：

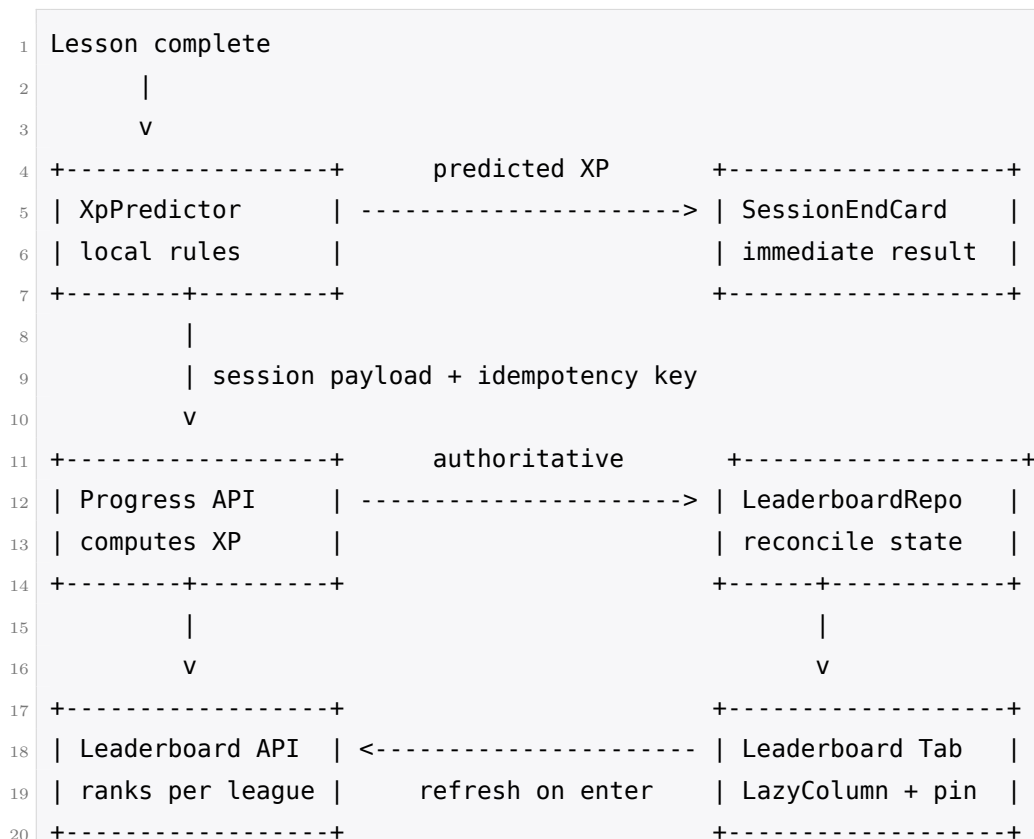
- “如何在后端计算 XP 较慢时快速展示结果？”
- “预测 XP 和服务端 XP 不一致怎么办？”

- “排行榜晋级能不能也预测?”

9.3.2 需求澄清：你该问什么

- 问：XP 计算是否完全能在客户端复现？答：不完全，可能有 boost、accuracy、实验规则。
- 问：排行榜 tier 多大？答：[高置信推断] Duolingo league 同 tier 通常是数十人规模，客户端分页压力不大，但仍要处理头像和刷新。
- 问：晋级/降级是否可以预测？答：不能。[官方] 官方明确说排行榜晋级不适合 prediction，因为公平性需要后端确认。
- 问：用户离线完成 lesson 后是否显示 XP？答：可以显示 pending/predicted XP，但不能发放权威排名奖励。

9.3.3 架构总览



这题的主线是两套 UI：session end card 需要快，leaderboard tab 需要可信。它们可以共享 Repository，但不能共享“所有值都同等可信”的模型。

9.3.4 数据模型

```

1 data class XpPrediction(
2     val sessionUuid: String,
3     val predictedXp: Int,

```



```
4     val ruleVersion: Long,
5     val confidence: PredictionConfidence,
6 )
7
8 @Entity(tableName = "leaderboard_entries")
9 data class LeaderboardEntryEntity(
10     @PrimaryKey val userId: String,
11     val displayName: String,
12     val avatarUrl: String?,
13     val xp: Int,
14     val rank: Int,
15     val isCurrentUser: Boolean,
16     val source: ScoreSource, // PREDICTED or SERVER
17     val updatedAt: Long,
18 )
19
20 data class LeaderboardUiState(
21     val entries: List<LeaderboardEntry>,
22     val pinnedMe: LeaderboardEntry?,
23     val isRefreshing: Boolean,
24     val hasPredictedScore: Boolean,
25 )
```

9.3.5 关键机制逐个击破

数据流：课程结束到排行榜

课程结束后，客户端先用 `XpPredictor` 算预测 XP，session end card 立即展示。这一步的价值是减少等待，符合官方 prediction 文章的动机。随后上传 session，后端计算 authoritative XP。leaderboard tab 进入时拉取真实榜单，也可以在当前用户行上叠加 pending delta，但必须明确标记来源。

XP 预测为什么会错

[官方] 官方给出的不一致来源很真实：前端和后端对“答对题数”的计算可能不同；前端认为 XP boost 生效而后端不认。再加上 A/B 实验、反作弊、补偿规则，客户端很难完整复现。正确表达不是“保证预测一致”，而是“缩小预测范围并设计回滚”。

三种回滚策略完整推演

Never roll back: session end card 和 leaderboard tab 都继续显示预测值。体验最平滑，但风险最大。用户可能看到自己短暂成为第一名，甚至截图分享；服务端真实值回来后如果不改，就破坏公平；如果后台又改，用户无法理解。

Immediate rollback: card 显示预测值，但 leaderboard tab 一进来就显示服务端真实值。技术上简单、数据可信，但两处 UI 同时不一致：刚结束一课说 +20 XP，点进榜单只多了 +10 XP。

用户会怀疑 bug。

Deferred rollback: 本次会话内两个地方都显示预测值, 并标记 pending; App 重启、下次进入或服务端确认窗口后替换为真实值。体验连续, 缺点是调试困难, 因为错误可能跨进程或跨启动显现。我的推荐是对“普通 XP 展示”使用 bounded deferred rollback: 有时间上限、pending 标记和埋点; 对晋级、奖励、最终排名使用服务端值。

策略	体验	正确性	推荐场景
Never	最顺	最差	几乎不用, 除非纯装饰数字
Immediate	数据可信	同屏打架	调试工具或低风险内部 UI
Deferred	连续性好	需上限与标记	session XP、短期 leaderboard 展示

为什么晋级必须等后端

[官方] 官方明确把 leaderboard promotion 列为 prediction non-goal。原因是公平性: 晋级影响用户之间的相对结果, 不能由每台客户端各自猜。即使客户端预测“你将晋级”, 也只能写成“当前看来在晋级区, 等待确认”, 不能发放奖励或展示最终晋级动画。

列表实现与刷新策略

同 tier 数十人规模简化了设计: 不需要复杂无限滚动, 但仍可用 Paging 3 或一次拉取加 Lazy-Column。当前用户即使不在当前可见区域, 也应该 pin 一行在顶部或底部, 帮助用户理解自己距离上一名多少 XP。滚动定位到自己要等数据加载后执行, 避免 Compose 首帧阻塞。

刷新策略上, 我不会用 WebSocket 常连。排行榜不是聊天, 实时性要求没有高到值得牺牲电量。更合理的是: 进入 tab 刷新、下拉刷新、完成 lesson 后触发一次刷新、后台按较低频率轮询或由 push 提示重大变化。头像加载用 Coil 一类图片库[\[高置信推断\]](#), 内存和磁盘缓存即可; 不要把头像二进制塞进 Room。

9.3.6 边界情况

- XP 相同: 服务端定义 tie-breaker, 例如先达到该 XP 的时间更靠前; 客户端只展示服务端 rank。
- 飞行模式完成课程: 本地显示 pending XP; 恢复网络后上传 session, leaderboard 刷新真实值。
- boost 过期边界: 客户端预测必须带 ruleVersion 与 observed boost 状态; 服务端返回差异后埋点分析。
- 当前用户不在第一页: pin my rank; 拉取 me 周围窗口。

【追问】“两个用户 XP 相同怎么排？”

客户端不自行决定。服务端返回 rank 和 tie-breaker 解释字段，例如 lastXpAt 或 joinedLeagueAt。客户端最多在 UI 上显示“同分按先达到排序”。因为任何客户端本地排序都可能和其他设备不一致。

【追问】“用户飞行模式下完成课程，回来后排行榜怎么补？”

完成时生成 session UUID，保存 pending operation 和 predicted XP。leaderboard tab 可以显示“+X pending”或把自己的行临时上移但标记 syncing。恢复网络后上传；服务端 ACK 后用真实 XP 替换。晋级与奖励等到真实榜单确认。

【追问】“如何避免短暂看到自己第一名然后掉下去？”

不要对高影响排名文案做强断言。预测阶段可以说“你可能上升到第 1，正在同步”，或在 leaderboard tab 中只显示 pending delta，不触发“你是第一名”的庆祝。对最终第一名、晋级区、奖励动画，等后端确认。

【设计思想】可迁移心法

同一个数字在不同 UI 里的可信等级可以不同。session card 要快，可以预测；leaderboard 要可信，要校准；晋级要公平，必须权威。设计时先给数据标注 source，再决定 UI 文案。

【陷阱】把预测值当作真值写进全局状态

如果 predicted XP 直接覆盖全局 user profile，后续所有页面都会把它当真，回滚范围不可控。预测值应带 source、sessionId、过期时间，并限制在需要它的 UI 域内。

9.4 客户端 A/B 实验 SDK

设计题 #4 | 难度: **Hard**

[高置信推断] 这题没有官方题面，但 **[官方]** Duolingo 相关 JD 强调跨 iOS、Android、backend、web 持续 A/B tests、reactive Android apps、clean APIs、unit tests；**[官方]** 公司公开文化也高度重视实验。因此客户端实验 SDK 是高置信设计方向。

9.4.1 题面与常见变体

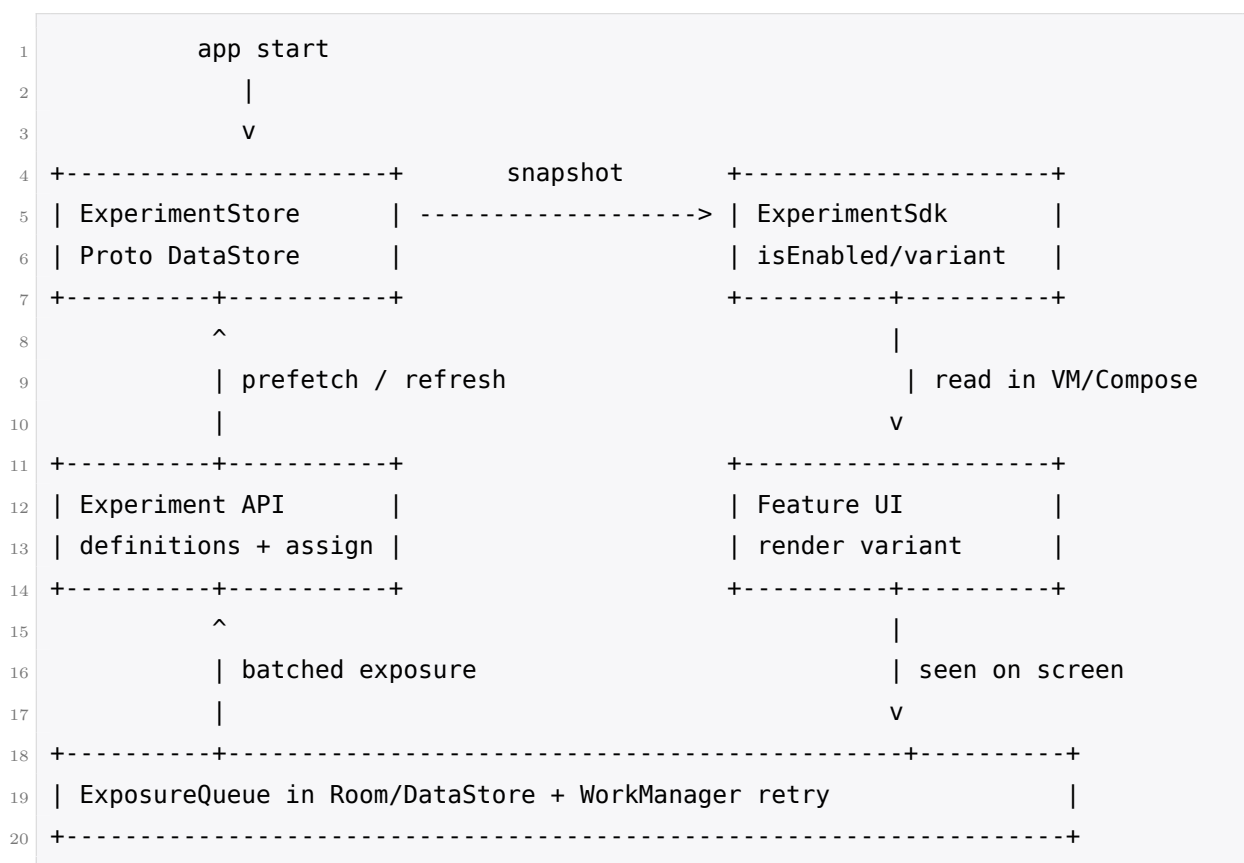
设计 Android A/B 实验 SDK：业务方可以读取实验 variant，Compose UI 根据 variant 渲染；assignment 稳定、可离线；曝光准确上报；实验定义变更后能失效。变体包括：

- “客户端自己哈希分桶，还是服务端下发 assignment？”
- “如何避免冷启动先显示 control 又闪成 treatment？”
- “曝光什么时候上报，如何离线去重？”

9.4.2 需求澄清：你该问什么

- 问：实验是否需要跨平台一致？答：有些需要，例如订阅 paywall；有些只在 Android 内部。
- 问：离线时是否要能读取实验？答：要，至少使用最近缓存的 assignment 或默认 control。
- 问：实验读取是否在冷启动关键路径？答：是，不能明显拖慢首帧。
- 问：曝光定义是什么？答：用户真正看到 treatment UI，而不是代码读取了 variant。

9.4.3 架构总览



SDK 的难点不是 if (enabled)，而是稳定 assignment、冷启动无闪烁、准确曝光、类型安全和可测试。

9.4.4 数据模型与 API

```

1 sealed interface Experiment<T> {
2     val name: String
3     val default: T
4 }
5
6 object NewPaywallExperiment : Experiment<PaywallVariant> {
7     override val name = "new_paywall"
8     override val default = PaywallVariant.Control
  
```

```

9  }
10
11  enum class PaywallVariant { Control, ShortTrial, AnnualFirst }
12
13  interface ExperimentSdk {
14      fun <T> getVariant(experiment: Experiment<T>): T
15      fun isEnabled(experiment: Experiment<Boolean>): Boolean
16      fun recordExposure(experimentName: String, variant: String, surface: String)
17  }
18
19  @Serializable
20  data class AssignmentSnapshot(
21      val userId: String,
22      val definitionsVersion: Long,
23      val assignments: Map<String, String>,
24      val fetchedAtMillis: Long,
25  )

```

默认值就是 control。这样离线、未命中、解析失败时都有安全回退。业务代码依赖接口，测试中注入 fake assignment。

9.4.5 关键机制逐个击破

分配：服务端下发 vs 客户端哈希

客户端哈希分桶的优点是稳定、离线可用、无需等待网络。规则可以是 `hash(userId + experimentName) mod 10000` 落入区间。只要 `userId` 不变，同一用户跨会话稳定。

但不是所有实验都适合客户端分配。涉及跨平台一致、后端逻辑、付费、风控、互斥层、复杂受众过滤的实验，应由服务端下发 assignment。服务端能保证同一用户在 Android、iOS、web、backend 拿到一致 variant，也能即时停实验。我的推荐是混合：SDK 支持本地哈希作为 fallback 和 Android-only 实验；权威 assignment 由服务端快照覆盖。

本地缓存与冷启动首帧

实验读取很容易制造闪烁：首帧用默认 control，DataStore 异步读完后变 treatment，UI 突然换。解决有两种。

第一，启动 splash 期间预取 assignment snapshot，设置一个很短预算，例如几十毫秒；读到就进入主界面，读不到用上次内存快照或默认值。第二，把实验读取做成同步内存快照：Application 启动时尽早异步加载 DataStore，一旦进入业务 UI，SDK 的 `getVariant` 只读 immutable map，不做 IO。这样业务方不会在 Compose recomposition 中触发磁盘读取。

【陷阱】在 Composable 里 suspend 读取实验

如果每个 Composable 自己从 DataStore collect variant，首帧、重组、闪烁和测试都会变复杂。实验 SDK 应该向 UI 暴露稳定快照；需要动态变更的远程配置另说。

类型安全 API 与线程安全

字符串 API 容易拼错实验名, 也容易把 variant 当 Boolean。用 typed experiment definition 可以让默认值、variant 类型和解析逻辑集中。SDK 内部持有 `AtomicReference<AssignmentSnapshot>` 或 `immutable StateFlow`; 读取无锁或轻锁, 刷新在后台替换整份 snapshot。业务代码只看到稳定接口。

曝光上报: 看到才算

曝光不能在 `getVariant` 时自动上报, 因为很多代码会提前读取但用户未看到。曝光应该由 UI 在变体真正进入屏幕时记录, 例如 Compose 的 `LaunchedEffect(variant, surface)` 配合可见性判断。事件写入本地队列, 批量上报, 离线暂存。去重 key 至少包含 `userId + experimentName + variant + surface + assignmentVersion`, 避免旋转屏幕重复曝光。

实验定义变更与过期

服务端返回 `definitionsVersion`、实验状态、过期时间、互斥层。客户端刷新后, 如果某 assignment 对应实验已结束或 variant 不再存在, 就失效并回到默认 control, 同时上报一次配置解析错误。过期实验不能永远躺在 DataStore 里, 否则老用户会长期停留在不存在的 treatment。

与 Compose 集成

ViewModel 读取 SDK, 产出普通 `UiState`; Composable 只渲染 `UiState`。对于纯 UI 实验, 也可以在 Composable 读取稳定 snapshot, 但仍要避免 IO。曝光用生命周期感知方式: 页面可见且 variant 实际渲染后记录一次。测试中用 fake SDK 固定 variant, 截图测试覆盖各分支。

9.4.6 边界情况

- 用户登出/切换账号: 清掉 user-scoped assignment, 回到 anonymous 或默认 control。
- 设备离线很久: 使用缓存但检查 ttl; 过期后对高风险实验回 control。
- 同一实验跨平台: 服务端 assignment 为准, 客户端哈希只做 fallback。
- 曝光队列满: 按时间批量上传; 超限时丢弃低价值曝光并记录计数, 不阻塞 UI。

【追问】“实验读取在冷启动关键路径上, 如何不拖慢启动?”

不在业务 UI 里做磁盘或网络读取。Application 阶段异步加载上次 snapshot; Splash 给一个很小预算尝试预取; SDK 对外是同步内存 map。网络刷新放后台, 刷新后是否立即影响当前页面要由产品决定, 通常下一次进入页面生效以避免闪烁。

【追问】“同一用户多设备必须拿到同一个变体吗?”

如果实验影响订阅、价格、后端行为或跨平台漏斗, 必须一致, 服务端 assignment。纯 Android UI copy、布局、动画实验, 可以客户端哈希, 只要同一设备和同一用户稳定即可。面试里要按实验风险分层, 而不是绝对回答。

【追问】“实验与 SDUI/远程配置的边界在哪里？”

实验回答“谁看到哪个 variant，并如何上报效果”；远程配置回答“某个参数当前值是多少”；SDUI 回答“界面结构由服务端如何描述”。三者可以组合：实验决定使用哪套 SDUI schema 或 config key，但曝光、assignment、互斥和统计仍属于实验 SDK。

【设计思想】可迁移心法

实验 SDK 的核心不是随机数，而是“稳定分配 + 无闪烁读取 + 准确曝光 + 可测试默认值”。任何增长团队、订阅团队、推荐团队都会问这四件事。

【要点】面试收束

最后主动总结：assignment 要稳定，读取不能拖慢首帧，曝光要在真正可见时上报，离线要排队，默认 control 要安全，测试要能注入 fake variant。这六句话基本覆盖客户端实验 SDK 的评分点。

Chapter 10

设计题精讲（下）：SDUI、资产、通知与状态重构

本章继续第七章的方法论与第八章的前四道设计题。请记住一个定调：**[官方]** Duolingo 官方面试博文说明，移动端 Design Interview 是 60 分钟、无需写代码的 Android/iOS 架构设计轮，重点不是后端分布式系统，而是本地存储、离线与同步、乐观更新与回滚、状态管理、进程被杀、设备约束。¹ 所以下面每道题都从客户端边界出发：你要能画清楚模块、状态、缓存、降级和测试，而不是把所有难点推给服务器。

10.1 Server-Driven UI 客户端解析与渲染框架

设计题 #5 | 难度：**Hard**

[官方] Duolingo 已公开介绍其 Server-Driven UI：响应包含 `screens`、`stylesheet` 与 `data_model`，并已用于 Shop 与 Purchase Flow。² **[高置信推断]** 因为这是官方上线且公开撰文的移动端架构系统，本题是本书里“推断题中概率最高的一道”。

10.1.1 题面与常见变体

- 设计一个 Android 客户端 SDUI 框架，可以解析服务端下发的页面描述并渲染成原生 UI。
- 如果服务端想免发版改 Shop 或购买流程，客户端要提供哪些能力集、缓存和版本兼容策略？
- 旧客户端不认识新组件时怎么办？如果响应有条件展示、点击动作和样式系统，又该如何测试？

10.1.2 需求澄清：你该问什么

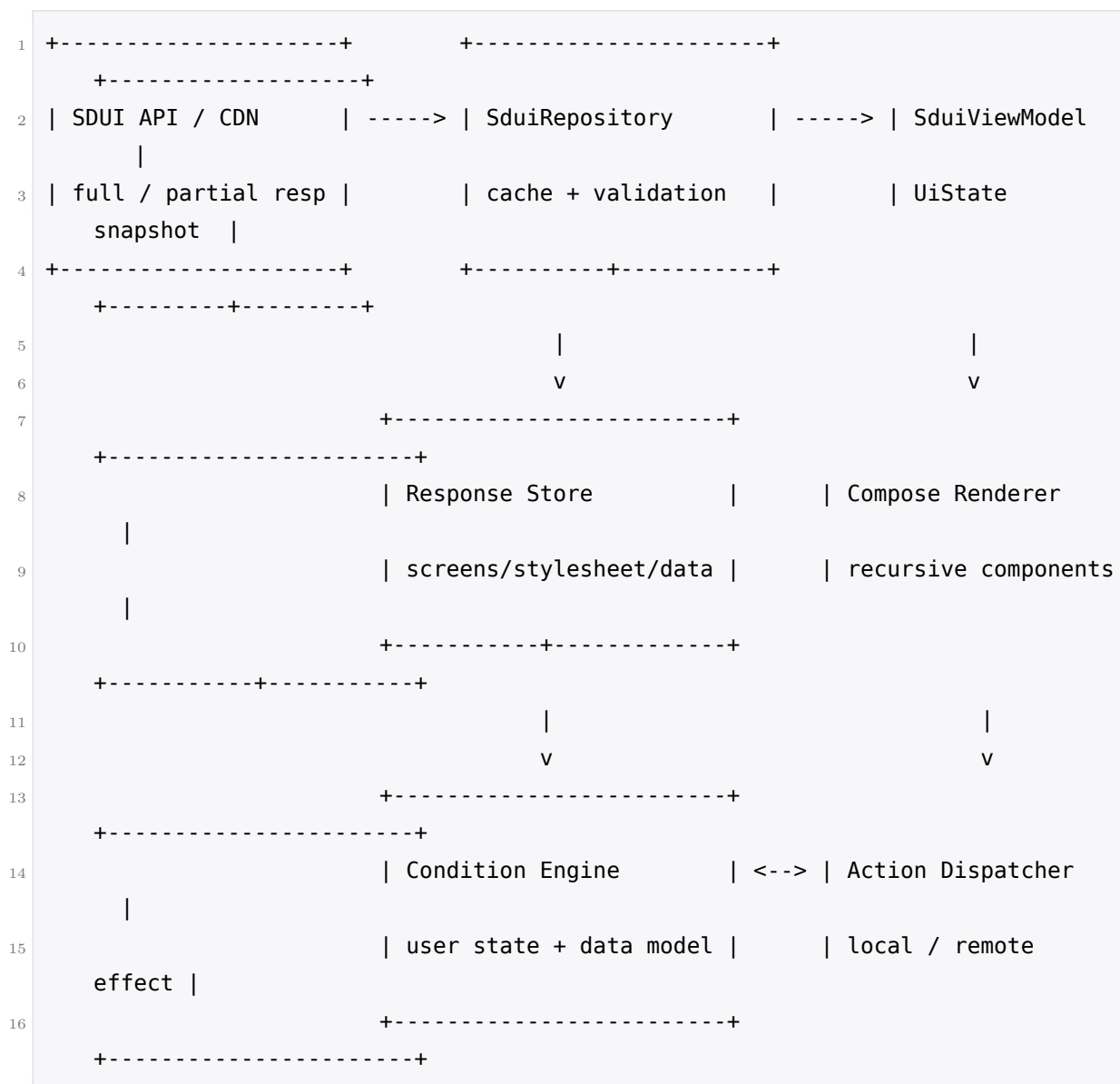
- “范围是全 App SDUI，还是只覆盖 Shop、订阅、活动页这类业务页面？”面试官常会回答：先覆盖低风险、高迭代页面，课程主流程和复杂动画仍保持原生。

¹<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

²<https://blog.duolingo.com/server-driven-ui/>

- “响应是否包含行为，还是只包含布局？” 合理回答：包含 tap/show/dismiss 等 Actions，可触发后端调用或本地操作，但能力由客户端白名单控制。
- “是否要求离线可用？” 通常不要求完整离线编辑，但至少缓存上一次可渲染的 UI config 与 data_model，网络失败时能展示降级页。
- “如何处理旧版本？” 官方策略是新客户端拿完整 SDUIResponse，旧客户端拿 Partial Response：只更新 data_model，UI config 用本地缓存的版本。[\[官方\]](#)

10.1.3 架构总览



10.1.4 数据模型 / 关键接口

```

1 @Serializable
2 sealed interface Component {

```

```
3     val id: String
4     val children: List<Component>
5 }
6
7 @Serializable @SerialName("label")
8 data class Label(
9     override val id: String,
10    val textKey: String,
11    val styleId: String? = null,
12    override val children: List<Component> = emptyList()
13 ) : Component
14
15 @Serializable @SerialName("stack")
16 data class Stack(
17     override val id: String,
18     val axis: Axis,
19     override val children: List<Component>
20 ) : Component
21
22 @Serializable @SerialName("unknown")
23 data class UnknownComponent(
24     override val id: String,
25     val rawType: String,
26     override val children: List<Component> = emptyList()
27 ) : Component
28
29 data class SduiResponse(
30     val screens: List<Screen>,
31     val stylesheet: Map<String, StyleSpec>,
32     val dataModel: Map<String, JsonElement>,
33     val uiConfigVersion: String
34 )
```

10.1.5 关键机制逐个击破

组件树建模与多态反序列化。 [官方] 官方文章提到 Duolingo 的 SDUI 使用约十二种基础组件, 如 labels、images、stacks、buttons、carousels, 由服务端组合。客户端不应该把 JSON 当成随手解析的 `Map<String, Any>`, 而应建模成一个小而稳定的 sealed interface `Component` 能力集。[高置信推断] 在 Kotlin 中可以用 `kotlinx.serialization` 的多态反序列化, 或 Moshi 的 polymorphic adapter, 以 `type` 字段判别具体组件。这样做的收益是: 渲染器只处理强类型组件; 测试 fixture 可以直接覆盖每个类型; 新增组件需要显式注册, 评审时更容易发现兼容性风险。

未知组件必须优雅降级。 这是评分关键点。服务端配置比客户端发布快, 旧客户端迟早会遇到未知 `type`。正确策略不是崩溃, 也不是静默吞掉整页, 而是分层降级: 第一, 低价值装饰组件可以

跳过，但保留兄弟节点；第二，影响布局的容器可渲染为保守占位并继续遍历已知子节点；第三，关键业务入口如购买按钮不能被未知组件承载，必须有 data model 驱动的本地兜底实现。[\[高置信推断\]](#) 如果未知组件出现在结账确认页的核心路径，宁可显示“请更新 App”或回退到原生购买流程，也不能展示半个不可用页面。

【陷阱】未知组件导致整页崩溃

SDUI 的价值是免发版迭代，但它也把“服务端配置错误”变成了“所有客户端同时收到坏 UI”的风险。面试时一定要主动说：解析层用 `UnknownComponent` 承接未知类型，渲染层按组件重要性跳过、占位或回退，监控层上报 raw type 与 config version。只说“我们会和后端约定好”是不够的。

递归渲染与 Compose。 [\[高置信推断\]](#) Compose 很适合递归组件树：`Render(component)` 根据类型分派，再对子组件递归。真正的难点是边界控制。第一，`key` 不能只用列表下标，应该优先用服务端稳定 `id`，否则 carousel 插入一个元素会导致状态错位。第二，要设置最大深度与最大节点数，例如深度 30、节点 1000，超过即拒绝渲染并展示降级页。第三，递归函数不能把 data model 读取散落在每个 Composable 中，最好先由 ViewModel 计算出可渲染快照，避免每次 recomposition 都重复求值复杂条件。

```
1 @Composable
2 fun RenderComponent(node: Component, ctx: RenderContext, depth: Int = 0) {
3     if (depth > ctx.maxDepth) return ctx.RenderFallback("too_deep")
4     key(node.id) {
5         when (node) {
6             is Label -> DuoLabel(ctx.text(node.textKey), ctx.style(node.styleId)
7         )
8             is Stack -> DuoStack(node.axis) {
9                 node.children.forEach { RenderComponent(it, ctx, depth + 1) }
10            }
11            is UnknownComponent -> ctx.RenderUnknown(node)
12        }
13    }
```

stylesheet 的解析、缓存与主题切换。 [\[官方\]](#) 官方响应把 stylesheet 与组件树分开。客户端应把样式看成“组件 ID 或 style ID 到 StyleSpec 的映射”，渲染时只解析一次成内部类型，例如 spacing token、字体 token、颜色 token。[\[高置信推断\]](#) 不建议让服务端直接下发任意像素、任意字体或任意颜色；更稳妥的是服务端选择客户端内置 token，如 `duo.primary.button`、`spacing.md`。这样暗色模式、无障碍字体缩放、品牌色更新都仍由客户端统一控制。缓存上，UI config 与 stylesheet 绑定版本号，按页面或 flow 存储；主题切换时不需要重新拉网络，只需要把 token 重新映射到当前主题。

data model 与 Conditions。 [官方] `data_model` 是动态数据,可本地缓存;Conditions 可基于用户状态决定组件是否显示,例如 hearts 数量。客户端可以把 data model 暴露成受控 key-value store,再实现一个小型表达式求值器:支持比较、布尔组合、存在性检查,禁止任意脚本。[\[高置信推断\]](#) 例如 `hearts < maxHearts` 可以决定是否展示“购买 hearts”模块。关键是表达式求值必须纯函数、可测试、可限制复杂度;不要在 condition 中发网络请求,也不要允许服务端下发可执行 Kotlin/JS。

Versioning 与 Partial Response。 [官方] 官方策略的精妙处在于:新客户端拿完整 `SDUIResponse` 并缓存 UI config;旧客户端只拿 Partial Response,也就是只发新的 `data_model`,让它继续用本地缓存的 UI config 渲染。因此服务端不必硬编码判断“客户端版本 X 支持组件 Y”。本质上,客户端缓存的 UI config 就是旧客户端已知能力集下的最后安全布局。

追问会落在三个细节:第一,本地缓存从哪来?可以在 App 包内预置一个默认 config,同时首次成功拉取完整响应后覆盖。第二,首次安装且没有网络怎么办?展示预置原生兜底或包内默认 config,不能白屏。第三,缓存与 App 版本如何绑定?缓存 key 应包含 `screenId + appMajorVersion + schemaVersion + uiConfigVersion`;升级后如果能力集改变,可以迁移或丢弃旧 config,避免新渲染器解释旧语义。

Actions 分发。 [官方] Actions 由 tap/show/dismiss 等事件触发后端调用或本地操作。[\[高置信推断\]](#) 客户端应使用 `ActionDispatcher` 白名单: `Navigate`、`OpenUrl`、`PostEvent`、`UpdateLocalData` 等。所有远程副作用都要有幂等键、loading 状态和失败回滚;所有本地副作用都要经过 `ViewModel`,而不是让 `Composable` 直接改全局状态。购买 gems 等 monetized 资源不适合随意乐观更新,这一点可引用第七章的 frontend prediction 边界。

可测试性。 SDUI 的测试核心是 JSON fixture。每个页面保留一组完整响应、partial 响应、未知组件响应、坏 condition 响应。解析层做 schema test,渲染层做 screenshot 或 semantics snapshot test,Action 层用 fake dispatcher 验证点击后发出的意图。[\[高置信推断\]](#) 真正上线前还要有 config linter: 服务端配置提交时就检查未知 style、循环引用、节点数上限和必填 action。

10.1.6 边界情况

- 本地只有旧 config,服务端 data model 缺字段:使用默认值并上报,关键字段缺失则降级。
- 组件树过深、节点过多或 condition 循环引用:拒绝渲染该 screen,展示 fallback。
- Action 重复点击:按钮本地去抖,远程请求带幂等键。
- 进程被杀后恢复:ViewModel 状态可丢,但 Repository 中的 cached response 与 pending action 结果必须能重建 UI。

【追问】“SDUI 与原生实现的边界该划在哪?”

适合 SDUI 的是内容和布局迭代快、逻辑可枚举、失败可降级的页面:Shop、活动页、购买引导、设置页局部模块。[\[官方\]](#) Duolingo 已用于 Shop 与 Purchase Flow。[\[高置信推断\]](#) 不适合 SDUI 的是高性能动画、课程答题核心交互、复杂离线编辑、强安全或强一致交易。回答要体

现“能力集固定在客户端”，不是把 App 变成浏览器。

【追问】“SDUI 如何配合 A/B 实验免发版？”

服务端按实验分桶下发不同 screen tree 或 stylesheet，客户端只负责渲染已支持能力集，并把曝光、点击、转化事件带上 experiment id。优势是实验启动不用发版；风险是同一客户端版本承载太多配置，所以必须有 config lint、灰度、kill switch 和 snapshot test。A/B SDK 本身见第八章。

【追问】“如果服务端下发会导致布局无限递归的响应怎么办？”

不要尝试“相信服务端不会”。客户端用解析时的节点数上限、渲染时的深度上限、引用解析的 visited set 三层保护。超过限制就停止渲染该子树并上报 config version。性能上可以缓存解析后的组件树，但缓存前必须先通过这些校验。

【设计思想】能力集与决策的分离

把“能显示什么”的能力集固化在客户端，把“显示什么”的决策交给服务端。版本兼容问题的本质是能力集协商：服务端只能组合客户端承诺支持的积木，客户端必须在遇到未知积木时安全降级。

10.2 图片与音频资产的缓存系统

设计题 #6 | 难度：Medium

[高置信推断] Duolingo 是重资产 App：每个词的发音音频、大量插画、角色动画和课程图片都会影响首屏、课程启动与离线体验，因此资产缓存是高概率 Android 设计题。

10.2.1 题面与常见变体

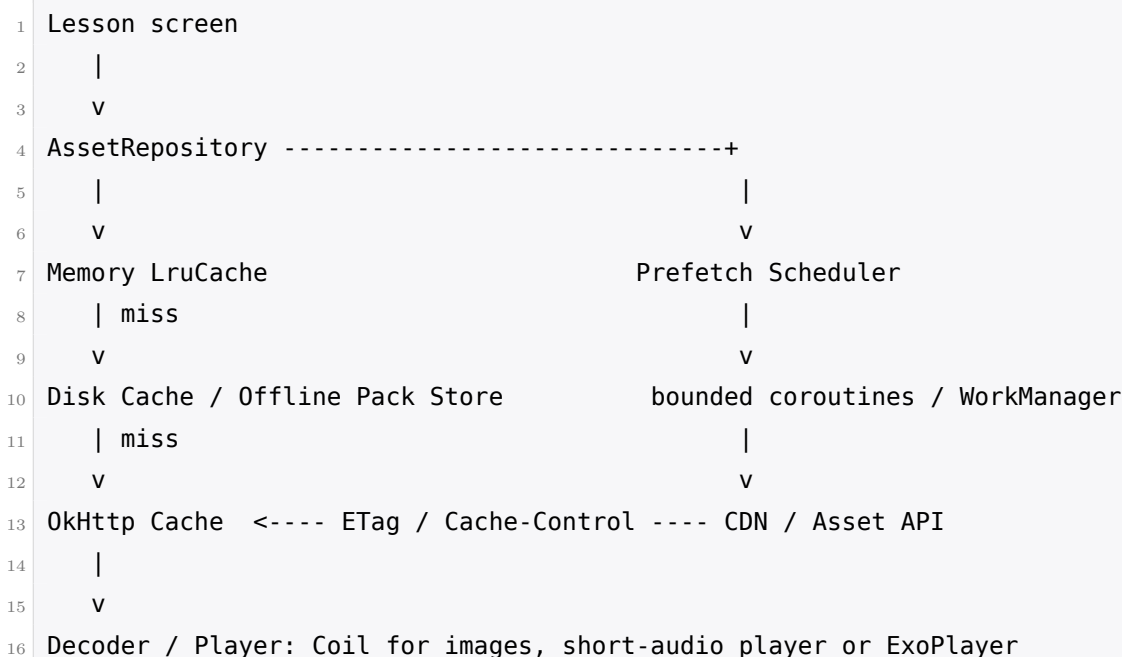
- 设计一个图片与音频缓存系统，支持课程中高频播放单词发音。
- 用户进入一节课前如何预取本节音频，同时不拖慢前台交互？
- 课程内容更新后，如何避免播放旧音频或错口音音频？

10.2.2 需求澄清：你该问什么

- “资产有哪些类型？”回答可假设：短音频、图片、轻量动画、少量视频；本题重点短音频和图片。
- “是否支持离线课程包？”如果支持，课程包是用户显式下载内容，不能放在系统可清理的 cacheDir。
- “缓存一致性要求多强？”音频和图片通常最终一致即可，但不能串语言、串词条、串说话人。

- “低端设备是否重点考虑?” Duolingo 官方 Android Reboot 提到入门设备课程启动曾慢 70%, 因此必须考虑内存与主线程。[\[官方\]](#)³

10.2.3 架构总览



10.2.4 数据模型 / 关键接口

```

1 data class AssetKey(
2     val language: String,
3     val entryId: String,
4     val voice: String,
5     val assetType: AssetType,
6     val contentVersion: String
7 )
8
9 interface AssetRepository {
10     suspend fun getAudio(key: AssetKey): Result<LocalAsset>
11     suspend fun getImage(key: AssetKey): Result<LocalAsset>
12     fun prefetch(keys: List<AssetKey>, priority: PrefetchPriority)
13 }

```

10.2.5 关键机制逐个击破

三级缓存。 [\[高置信推断\]](#) 标准链路是内存 LruCache -> 磁盘 DiskLruCache -> 网络。内存层服务于一节课内反复播放或反复展示，命中必须很快；磁盘层服务于跨进程、跨启动复用；网络层

³<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

依赖 CDN 与 OkHttp 缓存。图片可以直接使用 Coil，因为它 Kotlin/协程友好、生命周期感知、内置内存与磁盘缓存；音频则需要自建 key 与预取策略，因为“同一词条不同语言或口音”是业务语义，不是 URL 字符串能自然表达的。

复合 cache key。 资产 key 必须至少包含语言、词条、口音或说话人、资产版本。少一维就会出错：少语言，英语 course 与西语 course 可能共用同一个 surface form；少口音，拉美西语与西班牙西语会串音；少版本，课程内容更新后仍播放旧发音；少资产类型，图片和音频可能在路径归一化后冲突。[\[高置信推断\]](#) 面试中说出“复合 key 防串音”非常加分，因为它显示你理解语言学习 App 的业务约束。

版本化 key 优于失效通知。 课程内容更新时，不要依赖“服务端发一个 invalidate 消息让客户端删旧文件”。移动端可能离线、进程被杀、通知丢失。更稳的方式是让 contentVersion 或 assetVersion 参与 key：新课程引用新 key，自然下载新资产；旧资产由磁盘 LRU 慢慢淘汰。[\[高置信推断\]](#) 这是典型的 immutable asset 思路，牺牲一点磁盘空间，换来简单可靠的一致性。

HTTP 缓存与业务缓存的关系。 OkHttp 的 ETag 和 Cache-Control 解决的是“同一个 URL 的网络再验证”；业务缓存解决的是“这个词条、这个说话人、这个版本对应哪个本地文件”。两层不冲突。[\[高置信推断\]](#) 如果资产 URL 已经带内容哈希，可以设置很长的 max-age；如果 URL 稳定但内容可能变，用 ETag 做 revalidation。业务层仍要记录 AssetKey -> file，不能把整个系统建立在 URL 字符串拼接上。

音频播放器选择。 短发音音频高频、低延迟、文件小，优先考虑轻量播放器池或平台音频 API；如果涉及流式长音频、可变速、缓存分段、复杂音轨，再用 ExoPlayer/Media3。[\[高置信推断\]](#) 一句好的面试话术是：课程内单词发音更像 sound effect，不像视频流；我会避免为每个 300ms 音频创建重量级播放器，但如果产品后来加入长听力材料，我会把接口抽象好，底层切到 ExoPlayer。

预加载策略。 进入一节课前，客户端知道本节 lesson 的 challenge 列表，就可以预取全部短音频和首屏图片。预取应使用受限并发的协程，例如 2-4 个并发，并且只在 Wi-Fi 或网络良好时扩大窗口。后台大批量预取可交给 WorkManager，但前台 lesson 预取更适合 ViewModel scope 下的短任务。[\[高置信推断\]](#) 关键原则：预取不能抢占用户正在答题的前台带宽；预取失败不应阻塞开课，只标记哪些资产需要 lazy fetch。

低端设备内存。 [\[官方\]](#) Android Reboot 把入门设备性能作为重要背景。[\[高置信推断\]](#) 内存 LRU 大小应按 ActivityManager.memoryClass 动态计算，例如取可用堆的 1/8 或更保守比例，并根据低内存回调裁剪。图片解码要按目标尺寸采样，不要把 4K 图解码成全尺寸 Bitmap 再缩小。音频内存缓存只适合极短片段，更多时候缓存的是文件路径和已准备好的播放器状态。

磁盘配额与目录语义。 cacheDir 表示系统可在空间紧张时清理，适合可重新下载的普通缓存；filesDir 或应用管理的持久目录适合用户显式下载的离线课程包。[\[高置信推断\]](#) 这是加分点：如果用户点了“下载课程离线学习”，系统偷偷清理后地铁里打不开，会被视为产品 bug。因此普通图片和音频缓存放 cacheDir，离线包放持久目录并在 UI 中显示占用空间、允许用户手动删除。

10.2.6 边界情况

- 磁盘满：停止预取，保留当前 lesson 必需资产，淘汰最久未使用普通缓存。
- 用户切换账号或课程：按 user/course 维度清理离线包元数据，公共资产可按 key 复用。
- CDN 返回 404：记录 manifest 错误，跳过该音频并在 UI 提供重试，不能卡死课程。
- 进程被杀：预取任务可丢，已写入磁盘的完整文件保留；写文件采用临时名完成后原子 rename。

【追问】“用户在地铁里断网，预取到一半怎么办？”

已经完整下载并校验的资产可用；未完成文件不进入索引。课程开始时按 challenge 检查缺失资产：短音频可在 UI 上允许静音降级或提示稍后重试，核心答题文本仍可继续。若是用户显式离线包，下载状态必须是 partial，不能显示为已完成。

【追问】“同一个词在两门课程里复用，如何避免重复下载？”

把 AssetKey 设计成内容语义 key，而不是 course-local path。相同语言、词条、说话人、版本的音频只存一份；课程 manifest 引用同一个 key。离线包只增加引用计数或 manifest 关系，删除某课程时不立即删除仍被其他课程引用的资产。

【追问】“如何度量缓存命中率并定位每次都走网络的 bug？”

在 Repository 层打点：memory hit、disk hit、HTTP 304、HTTP 200、decode failure、eviction reason，并按 asset type、course、app version 聚合。若每次都走网络，优先查 key 是否包含不稳定字段、HTTP header 是否禁止缓存、文件是否写到 cacheDir 后被频繁清理、或 URL 是否每次带随机 query。

【设计思想】缓存首先是命名问题

缓存系统最难的不是 LRU，而是“什么东西算同一个资产”。语言学习 App 的资产 key 必须携带业务语义；只用 URL 或词面文本做 key，迟早会串音、串图或版本错乱。

【要点】目录选择

普通可再下载缓存放 cacheDir；用户显式下载、离线承诺的一整包课程放持久目录。系统可能清理 cacheDir，所以不能把“离线可用”的产品承诺建立在它上面。

10.3 推送通知的客户端接收、调度与落地

设计题 #7 | 难度：Medium

[面经报告] 多个面经汇总提到通知、提醒与 Android 生命周期问题；**[高置信推断]** 对 Duolingo 这种依赖每日学习习惯和连胜提醒的产品，推送通知设计非常自然。

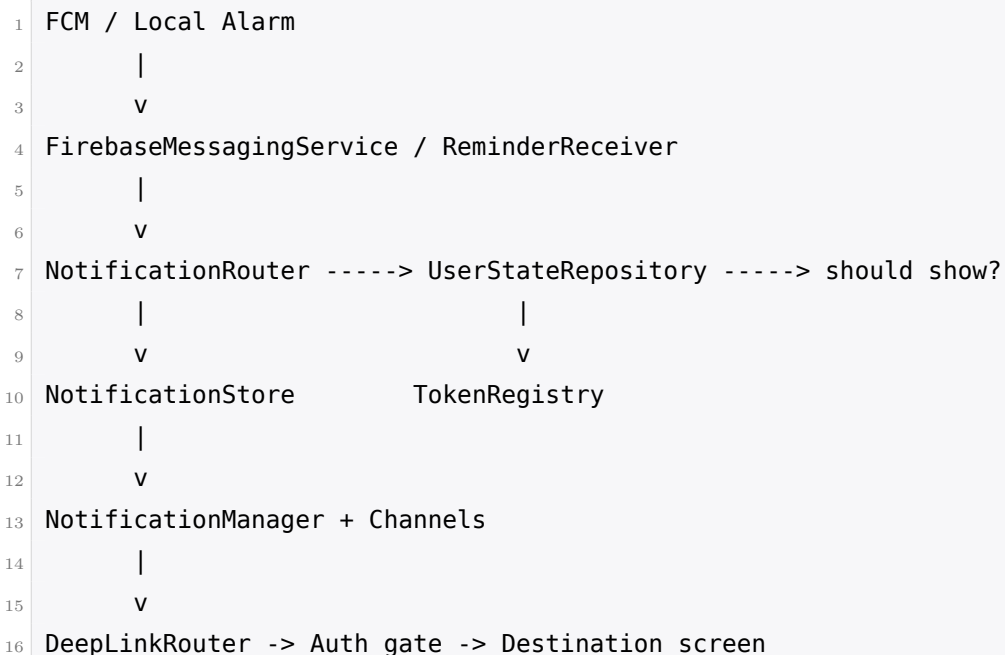
10.3.1 题面与常见变体

- 设计 Duolingo Android 的学习提醒、连胜挽救和营销通知系统。
- FCM 消息到达后，客户端如何决定展示、去重、落地到具体课程？
- Android 13 通知权限、Android 12 精确闹钟和 Doze 会如何影响设计？

10.3.2 需求澄清：你该问什么

- “通知类型有哪些？” 学习提醒、连胜挽救、排行榜、好友互动、营销活动，应分渠道和优先级。
- “哪些由服务端推，哪些本地调度？” 服务端适合跨设备一致和大规模策略；本地适合用户本机每日固定提醒。
- “是否需要精确到分钟？” 每日提醒通常不需要；连胜截止前最后提醒可能更接近精确，但仍要尊重权限和电量策略。
- “点击后落到哪里？” 可能是课程、排行榜、商店或登录页；必须校验参数和登录态。

10.3.3 架构总览



10.3.4 数据模型 / 关键接口

```

1  data class PushPayload(
2      val messageId: String,
3      val type: NotificationType,
4      val userId: String,
  
```

```
5     val deepLink: String,
6     val expiresAt: Instant?,
7     val dedupeKey: String
8 )
9
10 interface NotificationDecider {
11     suspend fun shouldShow(payload: PushPayload): Decision
12 }
13
14 interface TokenRegistry {
15     suspend fun bindToken(userId: String, token: String)
16     suspend fun unbindToken(userId: String, token: String)
17 }
```

10.3.5 关键机制逐个击破

FCM 的 notification payload 与 data payload。 [\[高置信推断\]](#) 这是最常被问倒的点。notification payload 在 App 后台时可能由系统直接展示, 客户端拿不到完整回调, 因此无法做复杂去重、登录态判断和本地状态检查。data payload 才会进入 `FirebaseMessagingService.onMessageReceived`, 客户端能自己建通知。对于“用户已经学过就不要再提醒”“点击要落到指定课程”这类需求, 应优先使用 data payload 或混合策略, 并接受系统限制: 后台执行时间短, 不能在回调里做长任务。

Token 注册、刷新与账号绑定。 FCM token 不是账号, 必须绑定到当前登录 user。 [\[高置信推断\]](#) 登录后上传 token 与 userId, token 刷新时重新绑定; 登出时解绑, 否则下一个使用这台设备的人可能收到前一个账号的连胜提醒。多账号切换时, 客户端本地只允许当前 active user 接收可展示通知; 服务端也要维护 token-user 关系的有效期和最后登录设备。

Notification Channel。 Android 8+ 要按类型建 Channel。 [\[高置信推断\]](#) 连胜挽救可放高重要度, 普通学习提醒中等, 营销低重要度。关键细节: Channel 一旦创建, 重要度由用户掌控, App 不能强行改高; 如果产品确实需要新的默认重要度, 只能创建新 channel id, 并清楚说明原因。不要把所有通知塞进一个 channel, 否则用户只能一刀切关闭。

本地提醒调度。 用户设置“每天晚上 8 点提醒我”时, 本地调度能更贴近设备时区和用户偏好。 [\[高置信推断\]](#) WorkManager 适合不精确、可延迟、带约束的任务; 它不能保证精确到 20:00。AlarmManager 的 `setExactAndAllowWhileIdle` 更精确, 但 Android 12+ 对精确闹钟权限更严格, 且 Doze/App Standby 仍会影响执行。我的决策: 普通每日提醒用 WorkManager 或非精确 Alarm, 允许一定窗口; 连胜即将过期这类高价值提醒优先由服务端推送, 并在客户端有权限时设置一个本地兜底 alarm; 没有精确权限就降级为非精确提醒, 不用为了学习提醒强迫用户授权。

Android 13 通知权限。 Android 13+ POST_NOTIFICATIONS 是运行时权限。 [\[高置信推断\]](#) 不要冷启动就弹权限框; 最佳时机是在用户完成第一节课、设置每日目标、或主动打开提醒设置时解释价值再请求。被拒后不要反复打扰, 可以在提醒设置页给出开启入口和系统设置 deep link。

Deep Link 落地。 点击通知时 App 可能冷启动、已在后台、或任务栈中已有多个 Activity。[\[高置信推断\]](#) 推荐统一 DeepLinkRouter：解析 URI，校验参数，检查登录态，拉取必要数据，最后导航到课程或排行榜。返回键行为要符合用户预期：从通知打开课程，返回可回到 Home，而不是退出到空白任务。所有 deep link 参数都视为不可信，尤其是 courseId、lessonId、userId 必须和当前账号匹配。

去重与时序。 “用户已经学过了就不要再提醒”可能发生竞态：服务端在 19:55 生成提醒，用户 19:58 完成课程，20:00 推送到达。[\[高置信推断\]](#) 分工是：服务端尽量基于最新状态调度，客户端到达时再做最后一跳判断。payload 带 expiresAt 与 dedupeKey；客户端查本地 today learned 状态，若已完成则丢弃。这样即使服务端旧消息到达，也不会打扰用户。

10.3.6 边界情况

- 用户关闭 channel：App 不绕过，只在设置页解释影响。
- token 刷新失败：本地重试并让服务端 token 过期机制兜底。
- payload 过期或账号不匹配：丢弃并上报。
- App 被强制停止：本地 alarm 可能失效，服务端推送仍是主要兜底。

【追问】“服务端调度和客户端本地调度各自优缺点是什么？”

服务端调度适合跨设备、全局频控、实验策略和时区批量计算；缺点是可能不了解设备权限和最新本地行为。客户端本地调度适合用户个人固定提醒和离线兜底；缺点是受 Doze、权限、进程状态影响。Duolingo 这种固定时区提醒可服务端主导、本地补充：服务端负责策略，客户端负责最后判断与用户偏好。

【追问】“用户关闭通知权限后如何优雅降级？”

不要用弹窗轰炸。产品上可在 Home 用非侵入卡片提示“开启提醒可保护连胜”，允许设置日历提醒或邮件提醒；技术上继续维护本地状态和 in-app banner。行为上尊重用户选择，这也符合 Learners First。

【追问】“如何避免同一时刻推送风暴？”

服务端按时区、用户活跃窗口和随机 jitter 分散；客户端对同类型通知做本地频控和 dedupe。Channel 分级也很重要：营销低重要度不能和连胜挽救抢同一优先级。监控上看送达、展示、点击、关闭和卸载率，而不是只看发送量。

【设计思想】通知是产品承诺，不是单纯技术通道

通知系统的核心不是“能发出去”，而是“在正确时间、以正确重要度、落到正确页面”。Android 的权限、Channel、Doze 和任务栈都会改变用户体验，客户端工程师必须把这些约束讲清楚。

【陷阱】把 notification payload 当成万能方案

如果你说“FCM 到了就在 `onMessageReceived` 里处理”，面试官很可能追问后台 notification payload。后台由系统直接展示时，App 不一定有机会做去重和本地判断。需要完整控制权的通知，应设计为 data payload 驱动的客户展示。

10.4 间隔重复练习的客户端存储与调度

设计题 #8 | 难度: **Medium**

[官方] / **[面经报告]** Birdbrain 是 Duolingo 自研个性化选题系统，在服务端计算 proficiency 并决定给用户出哪些题；**[高置信推断]** 因此客户端设计的核心不是本地跑 ML，而是存储、展示、同步和离线降级。

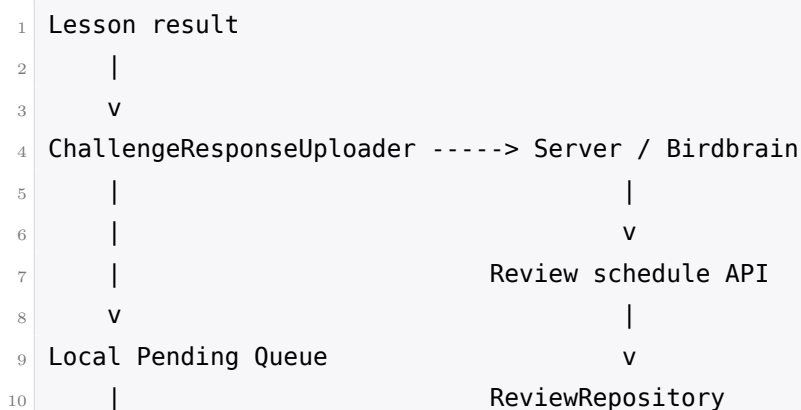
10.4.1 题面与常见变体

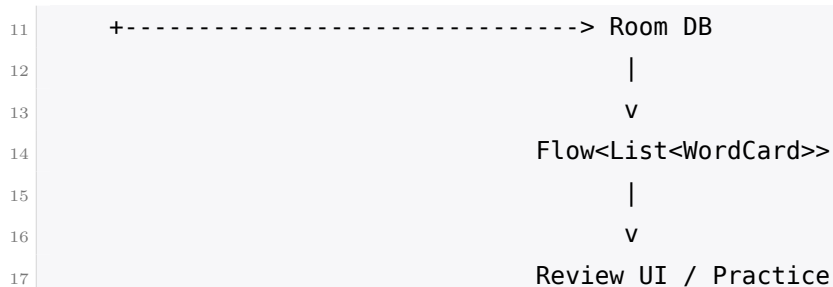
- 设计一个“今天该复习哪些词”的 Android 客户端模块。
- 离线时如何继续练习，联网后又如何接受服务端 schedule?
- 如果客户端本地算法和服务端 Birdbrain 给出的复习计划不同，谁说了算?

10.4.2 需求澄清：你该问什么

- “客户端是否要实现个性化模型?” 回答是：不跑 Birdbrain，服务端是权威；客户端只做离线兜底。
- “复习对象是词、句子还是技能?” 先以 WordCard 建模，但接口应允许 challengeId 或 skillId 扩展。
- “离线练习结果是否算用户资产?” 答题记录和课程进度是资产，要保守同步；review schedule 是系统建议，可被服务端覆盖。
- “是否需要实时 UI 更新?” Room + Flow 观察今天 due 的卡片即可。

10.4.3 架构总览





10.4.4 数据模型 / 关键接口

```

1  @Entity(
2      tableName = "word_cards",
3      indices = [Index(value = ["userId", "nextReviewAt"])]
4  )
5  data class WordCard(
6      @PrimaryKey val id: String,
7      val userId: String,
8      val wordId: String,
9      val nextReviewAt: Instant,
10     val easeFactor: Double,
11     val intervalDays: Int,
12     val lastResult: ReviewResult,
13     val source: ScheduleSource
14 )
15
16 @Dao
17 interface WordCardDao {
18     @Query("SELECT * FROM word_cards WHERE userId = :userId AND nextReviewAt <= :now ORDER BY nextReviewAt LIMIT :limit")
19     fun dueCards(userId: String, now: Instant, limit: Int): Flow<List<WordCard>>
20 }

```

10.4.5 关键机制逐个击破

职责边界。 [\[官方\]](#) / [\[面经报告\]](#) Birdbrain 的 proficiency 和选题在服务端计算。[\[高置信推断\]](#) Android 客户端如果试图复刻模型，会遇到模型版本、特征缺失、隐私、包体和一致性问题。正确边界是：客户端缓存服务端 schedule，展示“今天复习”，记录练习结果；离线时用简化 SM-2 风格算法给用户一个可用体验，但上线后以服务端 schedule 覆盖。

Room schema 与 Flow。 WordCard 表包含 userId、wordId、nextReviewAt、easeFactor、intervalDays、lastResult。[\[高置信推断\]](#) nextReviewAt 必须建索引，因为首页或练习入口会频繁查询“现在 due 的前 N 个”。Room 的 Flow 能自然驱动 UI：本地数据库更新后，复习列表自动刷新；进程被杀重启后，也能从 Room 恢复。

同步协议。 课程结束后, 客户端上传 challenge responses: 题目 ID、答题结果、耗时、是否使用提示等。服务端更新 proficiency 并返回新的 review schedule, 或客户端随后拉取。[\[高置信推断\]](#) pending queue 要保证至少一次上传, 带幂等 ID, 避免网络重试导致重复计入。拉取 schedule 时按 user/course 维度替换本地对应集合, 并记录 serverVersion。

离线降级与覆盖策略。 离线时, 客户端可以根据 lastResult 调整 interval: 答错则缩短, 答对则延长, 形成一个 SM-2 风格的简化 schedule。重点是联网后不要和服务端“合并”两个 schedule, 而是以服务端覆盖。原因: 课程进度、XP、购买资源是用户资产, 冲突时要保守合并; 复习调度是系统建议, 权威来自服务端模型。[\[高置信推断\]](#) 这与第八章离线课程进度的保守合并形成对照, 能体现你不是机械套同步模板。

数据量与查询性能。 每个用户几千到几万词卡都不算大, 但移动端查询必须稳定。[\[高置信推断\]](#) 用 (userId, nextReviewAt) 复合索引, 查询只取 due 的 limit, 不要一次加载所有卡片再在内存过滤。旧课程、重置进度、删除账号时应清理对应 userId/courseId 数据。多账号设备上所有表都必须带 userId, 避免串数据。

展示掌握度。 [\[高置信推断\]](#) 客户端不需要知道 Birdbrain 内部模型, 也不应暴露过细概率。可以展示服务端给的 bucket: 薄弱、一般、熟练, 或本地根据 nextReviewAt 与最近结果估算一个 UI 进度条。话术上要强调: 这是解释性展示, 不是客户端决策权。

10.4.6 边界情况

- 用户正在离线练习, 联网后 schedule 更新: 当前 session 不打断, 下一 session 使用服务端结果。
- 用户重置课程: 删除该课程 WordCard、pending queue 标记作废。
- 时区变化: nextReviewAt 用 UTC Instant 存, 展示时按本地时区格式化。
- 服务端返回空 schedule: 显示“今天没有复习”而不是错误。

【追问】“服务端和本地对该复习哪些词给出不同答案, 用户正在练到一半怎么办?”

不要中途换题, 避免破坏练习体验。当前 session 用启动时的本地快照完成; 结束后上传结果并拉取服务端 schedule, 下一轮再生效。如果服务端明确要求废弃某些题, 可在 session 结束后清理。

【追问】“本地兜底算法与服务端模型长期偏离会有什么问题?”

用户会练到不该练的词, 导致学习效率下降, 实验结果也会被污染。所以本地算法只应作为短期离线兜底, 并记录 source。设备重新联网后尽快上传离线结果, 让服务端重新计算, 客户端用服务端覆盖。

【追问】“如何不暴露模型细节又展示掌握度？”

让服务端返回粗粒度 bucket 或 display score，而不是完整模型特征。客户端只做颜色、文案和排序展示。这样既保护模型演进空间，也避免用户把一个精确小数误解成绝对能力评判。

【设计思想】资产与建议的同步策略不同

用户资产要保守合并，系统建议可以由权威覆盖。复习 schedule 属于“建议”，不是“用户拥有的进度”；这就是为什么它的离线冲突策略不同于 XP、连胜或课程完成记录。

【陷阱】在客户端复刻 Birdbrain

面试时不要说“我在 Android 端跑一个 ML 模型决定出题”。这会把模型版本、特征一致性、电量、包体和实验解释都变复杂。更好的边界是服务端智能、客户端可靠。

10.5 把一个 DuoState 式全局状态重构成 MVVM

设计题 #9 | 难度: **Hard**

[官方] Google Android Developers 博客公开描述 Duolingo Android Reboot：巨型全局可变状态 DuoState 被 XP、streak、lessons 等共享访问，ANR 每月恶化 5-10%，某次发布崩溃率增加 10%、慢帧 25%、入门设备课程启动慢 70%；团队停更功能 8 周，引入 MVVM、Dagger + Hilt、模块化、Repository pattern 和多线程，最终 ANR 降 41%、帧率不达标时间降 28%、滚动速度提升 40%。⁴ 这题官方基础最扎实，也是最能考察资深度的重构类设计题。

10.5.1 题面与常见变体

- 给你一个被 XP、连胜、课程进度、宝石共享读写的巨型可变状态对象，怎么拆？
- 如何从全局单例迁移到 MVVM + Repository + Hilt，同时不影响发版？
- PM 不愿停 8 周做重构，你如何用数据说服？

10.5.2 需求澄清：你该问什么

- “当前最痛的指标是什么？” ANR、崩溃、慢帧、课程启动耗时、开发 lead time，先建立基线。
- “DuoState 里有哪些域？” XP、Streak、Lesson、Leaderboard、Shop、Session 等按功能域拆。
- “是否允许停更或只能渐进？” 官方曾集中 8 周；面试中要同时给渐进方案与集中方案取舍。
- “团队规模和模块边界如何？” 如果多个团队共享状态，必须有编译期边界防回退。

10.5.3 架构总览

⁴<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

```

1 Before:
2   Any screen / manager / util
3       |
4       v
5   +-----+
6   | global mutable      |
7   | DuoState            |
8   | xp/streak/lessons   |
9   | gems/leaderboard    |
10  +-----+
11
12 After:
13   Screen -> ViewModel -> Domain Repository -> Data sources
14       |               |               |
15       v               v               v
16   StateFlow          single writer    Room / Network / Cache
17       |
18       v
19   immutable UiState
20
21 Gradle modules:
22   feature_lesson -> lesson_api -> lesson_impl
23   feature_shop   -> shop_api   -> shop_impl
24   app wires dependencies with Hilt

```

10.5.4 数据模型 / 关键接口

```

1 data class StreakState(
2     val current: Int,
3     val freezeAvailable: Boolean,
4     val lastUpdatedAt: Instant
5 )
6
7 interface StreakRepository {
8     val streak: StateFlow<StreakState>
9     suspend fun refresh()
10    suspend fun applyLessonCompleted(event: LessonCompleted)
11 }
12
13 @HiltViewModel
14 class HomeViewModel @Inject constructor(
15     private val streakRepository: StreakRepository,
16     private val sessionRepository: SessionRepository
17 ) : ViewModel()

```


10.5.5 关键机制逐个击破

先量化再动手。 [官方] 官方重构能被接受，是因为问题被量化：ANR 每月恶化、崩溃和慢帧明显上升、入门设备课程启动变慢。[高置信推断] 面试中你应先建立 dashboard：ANR rate、crash-free sessions、slow/frozen frames、lesson start p95、home scroll jank、cold start、开发周期和回滚次数。没有基线，重构就像“工程师洁癖”；有基线，重构才是业务风险控制。

按功能域拆 Repository。 不要按技术层先拆成 `NetworkManager`、`DatabaseManager`、`StateManager`，那只是把全局状态换个名字。[高置信推断] 应按业务域拆：`SessionRepository`、`StreakRepository`、`LeaderboardRepository`、`ShopRepository`。每个 Repository 拥有自己的数据源、缓存和写路径，对外暴露不可变状态流。这样团队边界也自然：Leaderboard 团队不应直接改 Streak 内部字段。

Hilt 与显式依赖。 [官方] 官方用 Dagger + Hilt 替代全局 DuoState。[高置信推断] 依赖显式化本身就消灭了一半耦合：过去任何类都能读写 `DuoState.gems`；现在构造函数里没有 `ShopRepository` 就无法访问宝石状态。谁读谁写从运行期约定变成编译期可见，code review 可以直接看到依赖扩散。

读写路径分离与单一写者。 全局可变对象最大的问题是 read-modify-write 竞态：A 屏刷新 XP，B 管理器同时改 streak，最后一次写覆盖前一次。[高置信推断] 每个状态域应该只有一个写者 Repository；UI 只收集 `StateFlow`，发出 intent，不直接 mutate。跨域事件如 `LessonCompleted` 通过明确方法或 domain event 分发，由各 Repository 自己更新本域，必要时用事务或有序队列保证一致性。

StateFlow 与不可变快照。 用 `MutableStateFlow` 内部维护状态，对外暴露 `StateFlow`；状态是 data class，不是可变对象。[高置信推断] UI 每次看到的是一个完整快照，Compose 或 View 层可以稳定 diff。回扣第七章/第四章的竞态原则：不要把同一个可变引用传给多个消费者再期待大家“别乱改”。

把工作移出主线程。 [官方] 官方重构包括 Repository pattern 与多线程，把工作移出主线程。[高置信推断] Repository 是天然边界：网络、数据库、JSON 解析、diff 计算都在 IO/Default dispatcher；ViewModel 只收集结果并暴露 UI state。所有 public suspend API 要说明线程语义，避免调用方在主线程做阻塞等待。

模块化与防回退。 [官方] 官方按功能模块化。[高置信推断] Gradle 模块可以采用 `feature_lesson` 依赖 `streak_api`，但不能依赖 `streak_impl` 的内部表或可变状态。用 `internal`、API/impl 分离、dependency analysis 和 lint 规则防止跨模块偷访问。否则两年后大家又会为了赶进度新建一个 `GlobalUserState`。

迁移策略：绞杀者与集中重构。 默认安全策略是 strangler：旧 DuoState 外包一层 adapter，新功能走 Repository，老功能按域迁移；feature flag 控制回退；一段时间内双读或影子比对，但尽量避免长期双写。**[高置信推断]** 然而官方选择停更 8 周集中重构，这并不幼稚。**[官方]** 当耦合是全局性的、任何渐进迁移都要长期维护复杂适配层和双写一致性时，集中重构可能总成本更低、风险窗口更短。高级回答要诚实讨论取舍，而不是机械说“大爆炸一定错”。

如何向 PM 论证 8 周。 **[官方]** 这直接对应 Take the Long View 与 Prioritize Ruthlessly。**[高置信推断]** 可以这样算账：ANR 每月恶化 5–10%，慢帧影响留存和课程完成；每个功能都要绕过 DuoState，开发 lead time 变长，回归率升高。若不还债，未来两个季度速度会继续下降；集中 8 周的目标不是“代码更漂亮”，而是把 ANR、慢帧、课程启动和发版回滚拉回可控区间。

10.5.6 边界情况

- 迁移中同一状态新旧两套来源不一致：定义权威来源和影子校验，不让 UI 随机读。
- 回滚：feature flag 回到旧路径，但数据库迁移要向后兼容。
- 团队继续加新全局变量：CI/lint 禁止，架构评审把关。
- 性能变差：每个迁移域都要看基线指标，不等到最后总验收。

【追问】“重构期间如何保证不出回归？”

先补 characterization tests，锁住旧行为；新 Repository 做 shadow read，与旧 DuoState 对比但不影响用户；按功能 flag 灰度；关键路径加 crash、ANR、slow frame、业务指标监控。迁移不是只靠人工 QA，而是每个域有可观测验收。

【追问】“如果不允许停更怎么办？”

采用 strangler：先冻结 DuoState 新增字段，建立 adapter；选痛点最大但边界较清晰的域先迁移，如 Streak；新功能只能接 Repository；旧功能逐步替换。代价是周期更长、适配层更复杂，所以要限制双写时间并定期删除旧路径。

【追问】“怎么防止两年后重新长出新的 DuoState？”

技术上用模块边界、internal、lint、dependency graph 和架构测试；流程上要求新共享状态必须有 owner、Repository API 和迁移计划；文化上在 code review 中拒绝“先放全局以后再说”。架构不是一次项目，而是一套防回退机制。

【设计思想】度量是架构债的入口和出口

架构债的偿还必须先有度量，否则你既说服不了别人，也验收不了自己。Duolingo 这次重构的说服力正来自 ANR、慢帧、滚动和启动时间这些可量化指标。

【要点】资深度信号

这题不要只背 MVVM、Hilt、Repository。资深候选人会讲：如何量化问题、如何划分所有权、如何迁移、如何回滚、如何防止组织在压力下重新制造全局状态。

Part V

项目、行为面与冲刺

Chapter 11

项目深挖、行为面与两周冲刺

本章把技术准备收束到面试当天：项目深挖怎么讲，行为面如何贴合 Duolingo 的官方 Operating Principles，两周内怎样把 Code Review 与 Design 练到可上场。**[官方]** 官方建议候选人 think out loud、先给实用的最简方案、与面试官协作、准备反问问题。¹

11.1 项目深挖轮

[面经报告] / **[高置信推断]** 项目深挖通常由 manager 或资深工程师主导，从你简历上的一个 Android 项目切入，逐层下钻到技术决策、权衡、影响和复盘。它不是“背项目介绍”，而是验证你是否真正在关键处做过判断。

常见下钻问题链如下：

- 为什么当时这么设计？
- 有哪些替代方案？为什么没选？
- 你如何衡量它成功了？指标是什么？
- 如果规模再大 10 倍，瓶颈会在哪里？
- 今天让你重做，你会改什么？

建议每个项目都按同一个骨架准备：

1. **背景与约束**：业务目标、用户痛点、时间、人力、遗留系统。
2. **我的具体角色**：说“我负责设计离线队列与冲突处理”，不要只说“我们做了一个功能”。
3. **关键技术决策**：方案 A、B、C 的取舍，尤其要说被否决的选项。
4. **量化结果**：崩溃率、ANR、启动时间、转化率、实验指标、开发效率。
5. **事后反思**：如果重做，会怎样降低复杂度或更早验证风险。

¹<https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/>

【要点】Duolingo 特别吃数据与实验

[官方] Duolingo 公开强调 Test It First、Learners First 与工程实验文化。你的每个项目最好能说出一个指标：不是“用户反馈不错”，而是“p95 启动时间下降 X%”“实验保留率无显著提升所以 kill”“离线失败率下降到 Y%”。没有指标的项目，也要补一个当时应该如何衡量的反思。

11.1.1 Android 项目示范讲法骨架

示例一：离线同步。 背景：弱网下课程结果丢失。我的角色：设计本地 pending queue、幂等 ID、重试和冲突处理。否决方案：直接依赖内存重试，因为进程被杀会丢；全量双向合并，因为课程完成是用户资产但某些推荐状态不是资产。结果：失败率、客服工单或重试成功率改善。反思：更早加入可观测性和回放测试。

示例二：性能重构。 背景：首页滚动掉帧或课程启动慢。我的角色：用 Perfetto/FrameMetrics 建基线，定位主线程 JSON 解析或 Bitmap 解码。否决方案：只加缓存掩盖问题；最终把数据准备移到 Repository 和后台线程。结果：慢帧、冷启动、ANR 指标变化。反思：建立性能预算和 CI 基准。

示例三：A/B 实验基础设施。 背景：移动端实验配置分散，曝光埋点不可信。我的角色：定义 assignment、exposure、local cache 与 kill switch。否决方案：每个 feature 自己拉实验配置，因为会造成口径不一致。结果：实验接入时间下降，曝光丢失率下降。反思：更早统一指标字典。

【陷阱】项目深挖两个致命伤

第一，把团队成果全部说成个人成果，会被追问击穿；第二，说不出任何被否决的方案，意味着你可能只是执行者，没有真正做技术决策。好的回答既诚实说明“我做了什么”，也能解释“为什么不是另一种做法”。

11.2 12 条 Operating Principles 与 STAR 故事

[官方] Duolingo 官方 Operating Principles 有 12 条。² 面试准备不需要为每条写一个独立故事，而是准备 6-8 个真实、可追问的故事，让一个故事覆盖多条原则。

²<https://blog.duolingo.com/operating-principles/>

原则	一句解读	该准备的故事
Learners First	用户学习价值高于短期指标	为用户利益反对某个指标或截止日期
Prioritize Ruthlessly	聚焦最重要的问题	主动砍 scope、保住核心路径
Take the Long View	为长期质量和学习效果投资	还技术债、改善架构或实验长期指标
Test It First	用实验验证假设	A/B 测试、灰度或失败实验复盘
Reduce Complexity	少做复杂系统，多做清晰边界	删除代码、合并流程、简化状态
Ship It	在不完美中交付价值	分阶段上线、用 feature flag 控风险
Strive for Excellence	对质量与细节较真	性能、无障碍、崩溃或体验打磨
Be Candid and Kind	诚实直接且尊重人	code review 或跨职能分歧
Explain Why	不只给结论，还解释理由	说服 PM/设计/团队接受取舍
Embrace Challenges	接下困难问题并成长	陌生技术、线上事故、复杂迁移
Never Settle on Talent	对团队能力有高标准	面试、带人、提升评审标准
All for One, and One for All	团队目标高于个人边界	跨团队救火、补位、共享工具

【陷阱】不要引用不存在的价值观

[官方] “Show, Don’t Tell” 不在 Duolingo 官方 12 条 Operating Principles 中。网上备考站常把它混进去；在面试中引用不存在的价值观是负向信号，说明你没有读官方材料。

11.2.1 故事矩阵

故事	可覆盖原则	必须准备的追问
性能重构	Take the Long View、Strive for Excellence、Explain Why	如何量化？如何说服 PM？
失败实验	Test It First、Learners First、Prioritize Ruthlessly	多快发现？kill 还是迭代？
离线同步事故	Embrace Challenges、Reduce Complexity、Ship It	如何回滚？如何防复发？
跨团队分歧	Be Candid and Kind、All for One、Explain Why	对方为什么反对？你让步了什么？
Code Review 发现重大 bug	Strive for Excellence、Never Settle on Talent	bug 如何证明？如何不伤人？
AI 工具质量问题	Embrace Challenges、Reduce Complexity、Strive for Excellence	你如何验证 AI 代码？流程怎么改？

11.2.2 STAR 范例骨架

【追问】模板：失败实验

S: 我们为了提升【指标】上线了【实验】。T: 我负责【Android 端实现/指标口径/灰度】。A: 我先定义 success metric 与 guardrail metric; 上线后第【N】天发现【代理指标】上升但【真实学习/留存/错误率】没有改善或变差; 我推动 kill/迭代, 并补上【流程改进】。R: 避免了【负面影响】, 沉淀了【实验检查清单】。这类故事可对应 Test It First、Learners First、Take the Long View。

【追问】模板：与 PM 或 Designer 分歧

S: 某功能在截止日前还有【性能/无障碍/离线】风险。T: 我需要既保护用户体验又不无限延期。A: 我把风险转成数据或 demo, 提出方案 A 立即 ship、方案 B 延后、方案 C flag 灰度; 我明确解释为什么。R: 团队选择【方案】, 结果【指标】。注意用“我如何让讨论更清晰”, 而不是“我赢了争论”。

【追问】示例：Android Code Review 重大 bug

S: 同事提交一个 streak 更新 PR, 正常路径测试通过。T: 我负责 review 状态一致性。A: 我发现进程被杀后 pending update 只在内存里, 可能导致 UI 显示已保住连胜但服务端未确认; 我写了一个重现步骤和小测试, 建议把事件持久化并加幂等 ID。R: 上线前避免了错误 streak 展示, 并把“进程死亡恢复”加入 review checklist。该示例是场景示范, 不要照抄成自己的经历。

11.3 高频行为问题与答法

11.3.1 Learners First 类

- 讲一个你为用户利益反对某个指标或截止日期的案例。考点：你是否敢于区分短期业务指标与真实用户价值；答案必须有数据、替代方案和沟通方式。
- 讲一个你用数据影响产品决策的案例。考点：不是只会写代码，而是能定义指标、解释结果、推动决策。
- Duolingo 什么做对了、什么你会改。考点：是否真的用过产品；建议结合 Android 体验，如提醒、课程流畅度、离线、无障碍，不要泛泛夸游戏化。

11.3.2 实验文化类

- 讲一个失败的实验。必须说清：多快发现，用什么指标发现，之后是 kill 还是迭代，以及你从中改了什么流程。Duolingo 重视 Test It First，失败实验不是丢脸，不能复盘才丢脸。
- 指标提升了但你怀疑用户其实没获益。这题在考代理指标与真实价值的区分。例：点击率涨了但学习完成率或长期留存没涨；短期 XP 增加但错误率上升。回答要落到 Learners First 与 Take the Long View。

- **如何决定哪些要 A/B 测试、哪些直接 ship？** 高风险行为改变、学习效果、付费和通知策略要测；明显 bug fix、崩溃修复、安全修复可直接 ship，但仍要监控。

11.3.3 协作类

- **讲你最好的结对编程经历。** 重点是你如何拆任务、共享上下文、让对方更强。
- **与 Designer 或 PM 的分歧如何解决？** 必须出现：先复述对方目标，再用数据/原型/用户影响比较方案。
- **一次改变你写代码方式的反馈。** 不要讲空泛“我更注重质量了”，要讲具体习惯如何改变，如更早写测试、更小 PR、更明确 owner。

11.3.4 AI 成熟度类

[面经报告] 2025 年新增高频问题包括：你如何在工作中使用 AI 工具？讲一次 AI 生成代码出质量问题、你发现并修复的经历。另有报告称出现实验性的 AI-Assisted Assessment 轮，约 1 小时。回答重点不是“我会用 AI 提效”，而是“我知道如何验证 AI 输出”。

- **你如何使用 AI 工具？** 说清边界：生成样板、测试草稿、解释陌生 API；敏感代码和架构决策仍由你负责。
- **AI 代码出质量问题怎么办？** 必须有具体 bug 类型、你如何通过测试/review/trace 发现、之后如何改 prompt 或流程。

11.3.5 Android 专项

- **你做过哪些影响整个 Android 团队技术方向的决策？** Senior 以上要讲技术领导力，不只是个人模块。
- **如何决定 MVVM 还是 MVI？** 回答应围绕状态复杂度、单向数据流、团队熟悉度、测试成本，不要站队宗教。
- **如何在 shipping velocity 与代码质量间平衡？** 直接对应 Ship It：用分阶段、feature flag、监控和明确的后续清债日期。
- **你用过 Duolingo 吗，哪个 Android 功能实现得最好/最差？** 结合真实体验和技术推断，如课程流畅度、通知权限、动画、离线。
- **如果设计 Duo 猫头鹰动画系统？** 讲资源缓存、状态机、降级、低端设备和实验配置。
- **如何确保移动端实验结果可信？** 讲 assignment 持久化、曝光时机、离线事件队列、版本过滤、guardrail 指标。

11.4 反问环节

[官方] 官方建议准备问面试官的问题。好的反问不是“你们文化好吗”，而是显示你理解 Android 团队的真实约束。

1. Compose 迁移到什么阶段？遗留 View 与 Compose 如何共存？这显示你懂 Android 架构迁移。
2. SDUI 目前的边界在哪里，哪些页面坚持原生？这连接官方 SDUI 博文与移动端取舍。
3. 实验数量如何影响移动端发版节奏和配置治理？这显示你关心实验可信度。
4. Android 团队如何度量 ANR、慢帧、启动和课程启动性能？这呼应 Android Reboot。
5. Code Review 更看重 bug、架构还是 Kotlin 惯用性？这帮助你理解质量文化。
6. On-call 或移动端线上事故如何分工？这显示你理解客户端也有生产责任。
7. Kotlin 100% 迁移后，detekt、ktlint、Android Lint 和自研规则如何协作？这引用官方 Kotlin 迁移材料。**[官方]**³
8. 对新入职 Android 工程师，前三个月最希望补上的产品或技术上下文是什么？这显示你想快速产生影响。
9. 移动端与 Learning/ML 团队如何定义客户端和服务端的边界？这适合 Birdbrain、个性化练习话题。
10. 团队如何处理技术债优先级，什么情况下会像 Android Reboot 那样集中投入？这显示你理解长期主义。

11.5 两周冲刺计划

假设每天 2-4 小时，本计划按读者优先级偏向 Code Review 与 Design。每天都必须有可验收产出，而不是“看了资料”。

³<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

天	任务	可验收产出
Day 1	全景浏览本书，精读官方面试博文	写出流程、轮次、移动端 design 评价轴一页纸
Day 2	精读 Frontend Prediction、SDUI、Android Reboot	各写 5 条可在面试引用的官方事实
Day 3	查错专项 ch04 5-6 题	每题写英文 review comment 与修复原则
Day 4	查错专项 ch05 5-6 题	建立 Kotlin 并发/生命周期错题表
Day 5	查错专项 ch06 5-6 题，并找真实 Kotlin PR 练习	完成 2 个 GitHub PR 的英文 review 草稿
Day 6	Kotlin 惯用性：Flow、coroutine、sealed class、Result	写 20 条 Kotlin review checklist
Day 7	算法轮复习：数组、哈希、堆、图、DP 各一题	每题 25 分钟限时并复盘复杂度
Day 8	设计题 ch08 前三题计时口述	每题手画架构图，录音 8-10 分钟
Day 9	设计题 ch08 第四题 + ch09 第五、第六题	写出 SDUI 与缓存的关键 trade-off 清单
Day 10	设计题 ch09 第七到第九题	每题准备 3 个 follow-up 答案
Day 11	结对编程环境演练：VS Code + Live Share，陌生 Kotlin 仓库限时加功能	60 分钟完成一个小改动并写复盘
Day 12	行为故事成稿	6-8 张 STAR 卡片，每张覆盖原则和追问
Day 13	全流程模拟：Code Review + Design + Behavioral	录屏或找人 mock，列出 10 个改进点
Day 14	复盘与休息	只看错题和卡片，整理面试反问，保证睡眠

【设计思想】冲刺期的验收标准

不要用“我复习了”作为完成标准。每一天都要产出可检查物：架构图、英文 review comment、STAR 卡片、录音、错题表。面试表现来自可复述、可追问、可迁移的材料。

11.6 资源清单

资源	来源	为什么值得读
https://blog.duolingo.com/interviewing-with-duolingos-engineering-team/	官方	面试流程、Design 轮定位与备考建议
https://blog.duolingo.com/operating-principles/	官方	12 条价值观，行为面必须以此为准则
https://blog.duolingo.com/frontend-prediction/	官方	乐观更新与三种回滚策略，移动端设计高频素材
https://blog.duolingo.com/server-driven-ui/	官方	SDUI 的响应结构、Actions、Conditions 与版本策略
https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html	官方/Google	Android Reboot、MVVM、Hilt、模块化和性能指标
https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/	官方	Kotlin 迁移、lint 链与工程文化
https://developer.android.com/topic/architecture	官方	Android 架构指南，MVVM/Repository/状态持有基础
https://developer.android.com/develop/ui/compose/side-effects	官方	Compose 副作用，回答状态与生命周期问题很有用
https://interviewing.io/companies/duolingo	社区	面试轮次和体验参考，需与官方信息交叉验证
https://www.programhelp.net/interview/duolingo-android-interview	社区	面经问题线索，不能当官方事实使用

结语

Duolingo Android 面试的核心不是背框架名，而是把“学习者价值、实验文化、移动端约束、工程质量”放进同一套叙事里。设计题要讲取舍，行为题要讲证据，项目深挖要讲你真实做过的决定。祝你把这些材料练到可以自然说出口。